Designing Data-Intensive Applications

The Big Ideas Behind Reliable, Scalable, and Maintainable Systems

Martin Kleppmann

Chapter-by-chapter notes on the author's argument

Density ceiling 1:3 (notes:book) · for personal study alongside the book

How to read these notes

Headings are the book's own, word for word. Text in quotation marks is the author's exact wording; everything else is close paraphrase that keeps his terms and steps of reasoning. Nothing is added from outside the book.

Blue box: "Place in the argument" — what the chapter title signals and why the chapter and its sub-headings come where they do. Gold box: how the chapter builds the author's case, linking back and forward. Gold labels in a Part overview are note headings, not the book's.

Where a passage matters in full, the notes point you to the section to read in the book.

Contents

Preface 4

PART I. Foundations of Data Systems 6

1. Reliable, Scalable, and Maintainable Applications 7
2. Data Models and Query Languages 12
3. Storage and Retrieval 20
4. Encoding and Evolution 27

PART II. Distributed Data 33

5. Replication 34
6. Partitioning 42
7. Transactions 46
8. The Trouble with Distributed Systems 56
9. Consistency and Consensus 65

PART III. Derived Data 77

10. Batch Processing 78
11. Stream Processing 88
12. The Future of Data Systems 98

The Architecture of the Book at a Glance

Part	Author's stated aim	Chapters
I. Foundations of Data Systems	"the fundamental ideas that apply to all data systems, whether running on a single machine or distributed across a cluster of machines"	1. Reliable, Scalable, and Maintainable Applications; 2. Data Models and Query Languages; 3. Storage and Retrieval; 4. Encoding and Evolution
II. Distributed Data	"what happens if multiple machines are involved in storage and retrieval of data?"	5. Replication; 6. Partitioning; 7. Transactions; 8. The Trouble with Distributed Systems; 9. Consistency and Consensus
III. Derived Data	integrating "several different datastores" into one coherent application architecture, separating systems of record from derived data	10. Batch Processing; 11. Stream Processing; 12. The Future of Data Systems

The book's purpose in the author's words

"The goal of this book is to help you navigate the diverse and fast-changing landscape of technologies for processing and storing data." It is "not a tutorial for one particular tool, nor is it a textbook full of dry theory"; instead it looks at the internals of successful data systems, with the aim of finding "useful ways of thinking about data systems—not just how they work, but also why they work that way, and what questions we need to ask".

Its stated justification: "behind the rapid changes in technology, there are enduring principles that remain true, no matter which version of a particular tool you are using."

Not summarised

Glossary, Index, Colophon, Copyright page, Dedication and the reference lists that end each chapter are reference material rather than the author's argument, so they are not summarised.

Preface

Place in the argument — why this chapter, why this title

The Preface frames the book before chapter 1: why, for whom, and with what limits. It promises to "dig deeper than buzzwords", hence the focus on internals and trade-offs. Headings: purpose, audience, scope, outline, sources, publisher, thanks.

Preface

- Engineers are "bombarded with a plethora of buzzwords" (NoSQL, Big Data, CAP theorem, MapReduce and more). Driving forces of recent developments: internet companies' huge data and traffic forcing new tools; businesses needing agility by "keeping development cycles short and data models flexible"; free and open source software; CPU clock speeds barely rising, so "parallelism is only going to increase"; IaaS letting small teams build distributed systems; high availability expected, downtime "increasingly unacceptable".
- Definition: **data-intensive** means data is the primary challenge (quantity, complexity, speed of change), as opposed to **compute-intensive**, where CPU cycles are the bottleneck.
- NoSQL gets attention, but queues, caches, search indexes and batch and stream frameworks matter too; applications combine them.
- Buzzwords show enthusiasm, but engineers need "a technically accurate and precise understanding" of trade-offs: "we have to dig deeper than buzzwords." Behind rapid change are "enduring principles that remain true".
- Goal: navigate a fast-changing landscape; not a tool tutorial nor a "textbook full of dry theory"; it studies successful data systems: internals, key algorithms, trade-offs, "not just how they work, but also why they work that way".
- Outcome: choose technology per purpose and combine tools; readers won't build their own storage engine, "but fortunately that is rarely necessary", yet gain intuition about what systems do.

Who Should Read This Book?

- For anyone building internet-using applications with a data backend; engineers, architects and technical managers "who love to code". Assumed: web experience, relational databases and SQL; language makes no difference.
- Valuable for scalable systems, highly available and "operationally robust" applications, easier long-run maintenance, or curiosity: "it's great fun to explore the bright thinking that went into their design."
- On "You're not Google or Amazon": there is truth, since unneeded scale is "a form of premature optimization". But choose the right tool; relational databases are "important but not the final word on dealing with data."

Scope of This Book

- No install or API instructions; principles, trade-offs and products' differing design decisions. Focus: architecture. Deployment, operations, security and management "deserve books of their own."
- "Big Data" is "so overused and underdefined" that the book uses single-node versus distributed, and online/interactive versus offline/batch.
- Bias toward free and open source software: source aids understanding, less vendor lock-in; proprietary software is covered where appropriate.

Outline of This Book

- Part I, fundamentals: Chapter 1 reliability, scalability, maintainability; 2 data models and query languages; 3 storage engines; 4 encoding and schema evolution.
- Part II, distributed data, "often necessary for scalability": replication (5), partitioning/sharding (6), transactions (7), distributed systems problems (8), consistency and consensus (9).
- Part III, derived datasets, since "there is no one database that can do everything well": batch (10), stream (11), and putting it together (12).

References and Further Reading

- Ideas come from talks, papers, code and “engineering folklore”; the book summarizes them with pointers. Chapter-end references (not summarised here) help go deeper.

O’Reilly Safari

Note

- Publisher’s Safari note; no argument.

How to Contact Us

- Publisher contact details; no argument.

Acknowledgments

- The book is “an amalgamation and systematization” of others’ ideas; “we have a huge amount to learn from things that have been done before.” Over 800 references.
- Thanks to named people, reviewers, editors, illustrators, family; errors are the author’s.

How this chapter builds the author’s case

The Preface sets the case for the whole book: tools change but principles endure, so chapters examine internals and trade-offs, bounded to architecture, for engineers who choose among tools.

PART I

Foundations of Data Systems

- The author's own description: "The first four chapters go through the fundamental ideas that apply to all data systems, whether running on a single machine or distributed across a cluster of machines".
- Chapter 1 "introduces the terminology and approach that we're going to use throughout this book", examining what is meant by reliability, scalability, and maintainability.
- Chapter 2 "compares several different data models and query languages", the "most visible distinguishing factor between databases from a developer's point of view".
- Chapter 3 "turns to the internals of storage engines and looks at how databases lay out data on disk".
- Chapter 4 "compares various formats for data encoding (serialization)", especially how they cope when "application requirements change and schemas need to adapt over time".
- The author closes the Part's description by pointing ahead: "Later, Part II will turn to the particular issues of distributed data systems."

The theme of the Part

- Single-machine ideas that hold for every data system: goals to aim for (Chapter 1), how data is modelled and queried (Chapter 2), how it is laid out on disk (Chapter 3), and how it is encoded as it changes over time (Chapter 4).

How the chapters build the theme

- Chapter 1 fixes the vocabulary and the three goals; Chapters 2 to 4 then move from the developer's view (models, queries) down to storage and out to encoding, each choice judged against those goals.

What the Part contributes to the book

- It is the base on which Part II (what happens if multiple machines are involved) and Part III are built.

CHAPTER 1

Reliable, Scalable, and Maintainable Applications

Place in the argument — why this chapter, why this title

The title names the three concerns the chapter is built around, in the order of its sections. It opens the book to "start by exploring the fundamentals of what we are trying to achieve", giving "the basics that we will need for later chapters". Order: why tools are lumped as data systems; then each concern, split into parts (kinds of faults; load and performance; operability, simplicity, evolvability); then Summary.

Chapter opening — untitled section before the first sub-heading in the book

- Alan Kay epigraph: the Internet "was done so well that most people think of it as a natural resource".
- Many applications are **data-intensive**, as opposed to **compute-intensive**: "Raw CPU power is rarely a limiting factor"; bigger problems are the amount, complexity and speed of change of data.
- Built from standard blocks: databases, caches (remember an expensive result), search indexes, stream processing (messages handled asynchronously), batch processing (periodically crunch accumulated data). This looks obvious only because data systems are "such a successful abstraction".
- But "reality is not that simple": different requirements give many databases, caching approaches and search index methods, and "it can be hard to combine tools when you need to do something that a single tool cannot do alone."
- The book is "a journey through both the principles and the practicalities of data systems"; this chapter covers reliable, scalable, maintainable systems, and later chapters go "layer by layer".

Thinking About Data Systems

- A database and a message queue both store data for some time, yet have "very different access patterns". Why one umbrella term, **data systems**?
- First: new tools "no longer neatly fit into traditional categories"; Redis is a datastore also used as a message queue, Kafka a message queue with database-like durability. "The boundaries between the categories are becoming blurred."
- Second: demanding applications exceed any single tool, so work is split into tasks and "those different tools are stitched together using application code", which normally must keep a cache (Memcached) or search server (Elasticsearch, Solr) in sync with the main database.

Figure 1-1. One possible architecture for a data system that combines several components .

- Shows how such components may be combined; details come later.
- The service's API usually hides the parts, making "a new, special-purpose data system from smaller, general-purpose components", maybe guaranteeing correct cache invalidation. "You are now not only an application developer, but also a data system designer."
- Design questions: keeping data correct and complete when things go wrong; good performance when parts are degraded; scaling; a good API.
- Three concerns "important in most software systems":
- **Reliability**: continue to work correctly (correct function at desired performance) despite adversity (hardware or software faults, human error).
- **Scalability**: reasonable ways of dealing with growth in data volume, traffic volume or complexity.
- **Maintainability**: many people over time, maintaining behavior and adapting, working "productively".
- These words are "often cast around without a clear understanding of what they mean"; techniques come in later chapters.

Reliability

- Expectations: the expected function; tolerance of user mistakes; good enough performance under expected load; no unauthorized access. All this is "working correctly"; reliability is "continuing to work correctly, even when things go wrong."

- Things that go wrong are **faults**; systems coping with them are **fault-tolerant** or **resilient**. The former is slightly misleading: tolerating every fault is not feasible (the Earth swallowed by a black hole would need web hosting in space), so only certain types of faults can be tolerated.
- A fault is "one component of the system deviating from its spec"; a failure is the whole system ceasing to provide the service. Fault probability cannot be zero, so design mechanisms "that prevent faults from causing failures".
- It can make sense to trigger faults deliberately, e.g. randomly killing processes: many critical bugs come from poor error handling, and this exercises the machinery. Netflix Chaos Monkey is an example.
- Prevention beats cure where no cure exists, e.g. security. The book mostly treats curable faults.

Hardware Faults

- Disks crash, RAM fails, blackouts, wrong cable unplugged; in big datacenters "these things happen all the time". Disk MTTF is about 10 to 50 years, so on 10,000 disks "we should expect on average one disk to die per day."
- First response is redundancy (RAID, dual power supplies, hot-swappable CPUs, backup generators): it "cannot completely prevent hardware problems from causing failures" but is well understood.
- It sufficed until recently; restoring a backup quickly made downtime tolerable.
- More machines raise fault rates proportionally; on AWS instances vanish without warning. Hence a move to tolerating loss of entire machines with software fault-tolerance, also allowing a **rolling upgrade**, one node at a time (see Chapter 4).

Software Errors

- Hardware faults are random and independent; software errors are systematic and "correlated across nodes", so they cause many more failures. Examples: a bug crashing every instance on a bad input (the leap second of June 30, 2012); a runaway process using a shared resource; a dependency that slows or returns corrupted responses; cascading failures.
- Bugs "lie dormant for a long time" until a circumstance shows an assumption about the environment, usually true, has stopped being true.
- "There is no quick solution": think about assumptions, test thoroughly, isolate processes, let processes crash and restart, monitor production, and have the system check its own guarantees (e.g. messages in equal messages out) and alert.

Human Errors

- Operators are human; one study found operator configuration errors the leading cause of outages, hardware only 10–25%. "The best systems combine several approaches": design to minimize opportunities for error (good abstractions make "the right thing" easy, but restrictive interfaces get worked around); decouple where mistakes happen from where they cause failures (sandboxes); test thoroughly at all levels; allow quick recovery (fast rollback, gradual rollout, tools to recompute data); detailed monitoring (telemetry); good management and training, beyond the book's scope.

How Important Is Reliability?

- Not only for nuclear plants: business-application bugs cost productivity and legal risk; ecommerce outages cost revenue and reputation. Even "noncritical" apps owe users, e.g. a parent's photos of their children in a corrupted database.
- Reliability may be sacrificed for a prototype or narrow margin, "but we should be very conscious of when we are cutting corners."

Scalability

- A reliable system may degrade under increased load (10,000 to 100,000 concurrent users, more data). **Scalability** is "a system's ability to cope with increased load", and not a one-dimensional label: ask what options exist "If the system grows in a particular way".

Describing Load

- Describe load with **load parameters**, depending on architecture: requests per second, read/write ratio, active users in a chat room, cache hit rate.

- Twitter (November 2012): post a tweet (4.6k requests/sec on average, over 12k at peak); view home timeline (300k requests/sec). 12,000 writes/sec would be easy; the challenge is **fan-out**. Two implementations:
- Approach 1: posting inserts the tweet into a global collection; reading a timeline looks up everyone followed, finds their tweets and merges them by time (a relational query, Figure 1-2).
- Approach 2: maintain a cache per user's home timeline, "like a mailbox of tweets"; posting inserts the tweet into each follower's cache, so reading is cheap because it was "computed ahead of time".

Figure 1-2. Simple relational schema for implementing a Twitter home timeline.

- The relational schema for approach 1.

Figure 1-3. Twitter's data pipeline for delivering tweets to followers, with load parameters as of November 2012 [16].

- The delivery pipeline for approach 2 with the load numbers.
- Approach 1 struggled with timeline reads, so Twitter switched to approach 2: published tweets are almost two orders of magnitude fewer than reads, so "it's preferable to do more work at write time and less at read time."
- Cost: a tweet reaches about 75 followers on average, so 4.6k tweets/sec become 345k timeline writes/sec; some users have over 30 million followers, within a five second target. The "distribution of followers per user" is thus the key load parameter.
- Twist: a hybrid; celebrities' tweets are merged at read time, delivering "consistently good performance". Revisited in Chapter 12.

Describing Performance

- Two questions: increasing a load parameter with resources unchanged, how is performance affected? Increasing it, how much must resources grow to keep performance unchanged? Both need performance numbers. Batch systems (Hadoop) care about **throughput**; online systems about **response time**, from request sent to response received.

Latency and response time

- **Response time** is what the client sees: **service time** plus network and queueing delays. **Latency** is "the duration that a request is waiting to be handled".
- Times vary every try, so response time is "a distribution of values that you can measure" (Figure 1-4: mostly fast, occasional outliers).

Figure 1-4. Illustrating mean and percentiles: response times for a sample of 100 requests to a service.

- Sample of 100 requests used to mark mean and percentiles.
- The mean "is not a very good metric" for typical time since it doesn't say how many users saw the delay. Use **percentiles**: the **median** (p50) means half of requests are faster; across several requests the chance at least one is slower is much greater than 50%.
- Higher percentiles (p95, p99, p999) show outliers: 95th at 1.5 s means 95 of 100 requests are faster.
- **Tail latencies** matter: Amazon sets internal requirements at the 99.9th percentile because the slowest requests come from the most valuable customers; a 100 ms rise cuts sales 1%; others report a 1-second slowdown cuts satisfaction 16%. The 99.99th was judged too expensive.
- Percentiles define **SLOs** and **SLAs**, e.g. median under 200 ms, 99th under 1 s, up 99.9% of the time, letting customers demand refunds.
- Queueing causes much high-percentile delay: a few slow requests hold up others (**head-of-line blocking**), so measure on the client side. Load-test clients must send independently of responses, or they "skews the measurements".

Percentiles in Practice

Figure 1-5. When several backend calls are needed to serve a request, it takes just a single slow backend request to slow down the entire end-user request.

- Shows one slow backend call slowing the whole request.
- When a request needs several backend calls, it waits for the slowest: **tail latency amplification**.

- Monitor percentiles over a rolling window; forward decay, t-digest or HdrHistogram approximate cheaply. averaging percentiles “is mathematically meaningless”; add histograms.

Approaches for Coping with Load

- “An architecture that is appropriate for one level of load is unlikely to cope with 10 times that load.”
- **Vertical scaling** (a stronger machine) versus **horizontal scaling** (a **shared-nothing architecture**): one machine is simpler but costly; good architectures mix.
- **Elastic** systems add resources automatically, useful for unpredictable load; manual ones are simpler with fewer operational surprises.
- Stateless services distribute easily, stateful ones add complexity, so the wisdom was one database node until forced. “It is conceivable that distributed data systems will become the default”, a theme for the rest of the book.
- “there is no such thing as a generic, one-size-fits-all scalable architecture” (**magic scaling sauce**): 100,000 requests/sec of 1 kB differs from 3 requests/min of 2 GB, though throughput is the same.
- Architectures rest on assumptions about common and rare operations; if wrong, effort is “at best wasted, and at worst counterproductive”. Yet general-purpose building blocks and patterns recur, which the book covers.

Maintainability

- Most software cost is “ongoing maintenance”. Legacy systems are disliked, and “Every legacy system is unpleasant in its own way”, but we can design to minimize pain via three principles:
- **Operability**: easy for operations to run the system smoothly.
- **Simplicity**: easy for new engineers to understand, removing complexity (not user interface simplicity).
- **Evolvability**: easy to change for unanticipated use cases (also extensibility, modifiability, plasticity).
- There are no easy solutions here either.

Operability: Making Life Easy for Operations

- “good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations”; humans set up automation. Operations teams handle monitoring, tracing problems, patches, capacity planning, deployment, migrations, security, predictable processes and organizational knowledge. “Good operability means making routine tasks easy”; data systems help via visibility, automation support, no dependency on individual machines, good documentation (“If I do X, Y will happen”), overridable defaults, self-healing with manual control, and predictable behavior.

Simplicity: Managing Complexity

- Large projects become a **big ball of mud**. Symptoms: state space explosion, tight coupling, tangled dependencies, inconsistent naming, hacks, special-casing.
- Complexity overruns budgets and invites bugs; “reducing complexity greatly improves the maintainability of software, and thus simplicity should be a key goal”.
- Simplicity means removing **accidental complexity** (Moseley and Marks: not inherent in the problem, arising only from the implementation), chiefly by **abstraction**: high-level languages hide machine code; SQL hides on-disk structures, concurrent clients and crash inconsistencies. Reuse raises quality.
- But “finding good abstractions is very hard”, especially for distributed algorithms; the book will look for them.

Evolvability: Making Change Easy

- Requirements are in “constant flux”. Agile, TDD and refactoring work at a small scale; the book seeks agility for a larger data system, e.g. “refactor” Twitter from approach 1 to 2.
- Ease of modification is linked to simplicity and abstractions, so the author uses the word **evolvability**.

Summary

- Functional requirements say what it does; nonfunctional ones are security, reliability, compliance, scalability, compatibility, maintainability. Covered here: reliability, scalability, maintainability.

- Reliability: correct work despite faults in hardware (random, uncorrelated), software (systematic) and humans; fault-tolerance can hide certain faults from users.
- Scalability: keeping performance good as load grows, described via Twitter and response time percentiles; capacity can be added to “remain reliable under high load”.
- Maintainability: better life for engineers and operations via abstractions and operability.
- “There is unfortunately no easy fix”, but patterns recur; next chapters analyze example data systems, and Part III patterns for multi-component systems like Figure 1-1.

Footnotes

- Notes on elasticity, fan-out (requests made to serve one request) and batch run time (skew, slowest task).

References

- The author ends the chapter with references (not summarised).

How this chapter builds the author’s case

This chapter fixes the three yardsticks later chapters apply. It begins with the claim that combining tools makes you a data system designer, then defines each yardstick with the author’s examples (Chaos Monkey, Twitter, Amazon). Forward references: Chapter 4 rolling upgrades, Chapter 12 Twitter, Part III composite systems.

CHAPTER 2

Data Models and Query Languages

Place in the argument — why this chapter, why this title

The title pairs two things: data models (how data is represented) and query languages (how it is asked for). The chapter is here because "Data models are perhaps the most important part of developing software", with a "profound effect" on how software is written and "how we think about the problem". This chapter covers "general-purpose data models for data storage and querying" (layer 2); Chapter 3 covers their implementation (layer 3).

Place in the argument — why this chapter, why this title

Order: relational versus document (argued with history); query languages (declarative versus imperative); graph-like models, for data with many many-to-many relationships; Summary.

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph (Wittgenstein): "The limits of my language mean the limits of my world."
- Layers: the developer models the real world as objects, data structures and APIs; stores them in a general-purpose model (JSON/XML documents, tables, a graph); the database engineers represent that as bytes in memory, on disk or on a network; hardware engineers represent bytes as electrical currents, pulses of light, magnetic fields.
- "For each layer, the key question is: how is it represented in terms of the next-lower layer?" Each layer hides the complexity below with "a clean data model", letting groups (database vendor engineers, application developers) work together.
- "every data model embodies assumptions about how it is going to be used": some usage is easy, some unsupported; some operations fast, some bad; some transformations natural, some awkward. Mastering even one takes effort, so "it's important to choose one that is appropriate to the application".
- Plan: compare relational, document and graph-based models, and query languages.

Relational Model Versus Document Model

- The best-known model is SQL's, based on Codd's 1970 relational model: data in relations (tables), each an unordered collection of tuples (rows).
- It was a theoretical proposal doubted for efficiency, but by the mid-1980s RDBMSes and SQL were the tools of choice for data with regular structure; dominance has lasted around 25–30 years, "an eternity in computing history".
- Roots: 1960s-70s business data processing, "mundane": transaction processing and batch processing. Other databases made developers think about internal representation; the relational goal was to hide it behind a cleaner interface.
- Competitors: network and hierarchical models, object databases, XML databases (niche). Each had hype, "but it never lasted".
- Relational databases "generalize very well" beyond their origin.

The Birth of NoSQL

- NoSQL is the latest attempt to overthrow relational dominance. The name "is unfortunate": a catchy Twitter hashtag for a 2009 meetup, later Not Only SQL. Driving forces: need for scalability beyond relational (very large datasets or write throughput); preference for free and open source software; specialized query operations the relational model supports poorly; frustration with restrictive schemas and a wish for a "more dynamic and expressive data model".
- Different applications need different technology, so relational databases will probably be used alongside nonrelational stores: **polyglot persistence**.

The Object-Relational Mismatch

- Application code is object-oriented, so SQL tables need "an awkward translation layer", the **impedance mismatch**. ORMs cut boilerplate but "can't completely hide the differences".

- Example: a résumé (LinkedIn profile). first_name and last_name appear once per user so are columns; positions, education and contact information are one-to-many, representable three ways: separate tables with foreign keys to users (Figure 2-1); structured datatypes and XML in later SQL versions, with querying and indexing inside (Oracle, DB2, SQL Server, PostgreSQL), and JSON datatypes (DB2, MySQL, PostgreSQL); or a JSON/XML text column the application interprets, with no querying inside it.

Figure 2-1. Representing a LinkedIn profile using a relational schema. Photo of Bill Gates courtesy of Wikimedia Commons, Ricardo Stuckert, Agência Brasil.

- The multi-table form.
- A résumé is mostly "a self-contained document", so JSON is appropriate, "much simpler than XML"; MongoDB and others support this model.

Example 2-1. Representing a LinkedIn profile as a JSON document

- The same profile as one JSON document, with positions, education and contact_info nested.
- Some feel JSON reduces the mismatch; Chapter 4 shows problems with JSON as an encoding; schema comes under "Schema flexibility in the document model".
- JSON has better **locality**: the relational form needs multiple queries or "a messy multi-way join", the JSON form "one query is sufficient". The one-to-many relationships imply a tree, which JSON makes explicit.

Figure 2-2. One-to-many relationships forming a tree structure.

- Shows the tree.

Many-to-One and Many-to-Many Relationships

- Why are region_id and industry_id IDs rather than text such as "Greater Seattle Area"? Free text suits free-text fields, but standardized lists chosen by drop-down or autocompleter give: consistent style and spelling; no ambiguity (same-named cities); easy updating (stored in one place); localization; and better search (the region list can encode that Seattle is in Washington).
- It is a question of duplication: with an ID the human-meaningful text is stored once; an ID has no meaning to humans so "it never needs to change", whereas duplicated text needs all copies updated, with write overhead and inconsistency risk. Removing such duplication is **normalization** (a footnote: duplicated values mean not normalized).

Note

- Administrators and developers argue over normalization and denormalization; judgment is suspended until Part III (caching, denormalization, derived data).
- Normalizing needs many-to-one relationships (many people in one region), which "don't fit nicely into the document model". Relational databases join easily; document databases need no joins for trees, and join support "is often weak".
- Without joins, emulate them in application code with multiple queries, shifting the work from database to application.
- Data "has a tendency of becoming more interconnected": organizations and schools as entities with their own pages; recommendations between users, which must reflect the author's updated photo and so reference the author's profile.

Figure 2-3. The company name is not just a string, but a link to a company entity. Screenshot of linkedin.com.

- Company as an entity.

Figure 2-4. Extending résumés with many-to-many relationships.

- Each dotted rectangle can be one document, but references to organizations, schools and users "require joins when queried".

Are Document Databases Repeating History?

- The debate on representing many-to-many relationships "goes back to the very earliest computerized database systems".

- IBM's IMS (released 1968, still maintained) used the **hierarchical model**, with "remarkable similarities to the JSON model": a tree of nested records. It was good for one-to-many, hard for many-to-many, with no joins; developers chose between denormalizing and resolving references manually. These 1960s-70s problems were "very much like" today's document database problems.
- Two solutions emerged: the relational model (which "took over the world") and the network model (faded into obscurity); the "great debate" lasted much of the 1970s and is worth revisiting.

The network model

- Standardized by CODASYL, a generalization of the hierarchical model: a record could have multiple parents (e.g. one "Greater Seattle Area" record linked to every resident), allowing many-to-one and many-to-many.
- Links were like pointers, not foreign keys; the only access was an **access path** from a root record. With many-to-many, several paths lead to one record and the programmer tracked them; even committee members compared it to navigating an n-dimensional data space.
- Manual paths suited 1970s hardware (slow tape seeks) but made code "complicated and inflexible"; changing paths meant rewriting handwritten query code, so changing the data model was difficult.

The relational model

- By contrast it "lay out all the data in the open": relations are collections of tuples, no nested structures or access paths; read any rows matching an arbitrary condition, look up by key, insert without worrying about foreign keys (footnote: constraints are optional, and joins happen at query time, whereas in CODASYL effectively at insert time).
- The query optimizer chooses the "access path" automatically, "so we rarely need to think about them". A new index is used by existing queries, making new features easier.
- Optimizers are "complicated beasts", but "you only need to build a query optimizer once"; handcoding is easier per query but "the general-purpose solution wins in the long run".

Comparison to document databases

- Documents revert to the hierarchical model in storing nested records inside the parent. For many-to-one and many-to-many, both use a unique identifier (foreign key or document reference) resolved at read time by join or follow-up queries. "To date, document databases have not followed the path of CODASYL."

Relational Versus Document Databases Today

- Fault-tolerance (Chapter 5) and concurrency (Chapter 7) are set aside.
- For documents: schema flexibility, better locality performance, closeness to application data structures. For relational: better joins and many-to-one, many-to-many support.

Which data model leads to simpler application code?

- If data is document-like (a tree of one-to-many, loaded at once), use documents; **shredding** into tables gives "cumbersome schemas and unnecessarily complicated application code".
- Limit: no direct reference to a nested item ("the second item in the list of positions for user 251", like an access path), usually fine if not deeply nested.
- Poor joins may not matter (e.g. analytics recording events by time). With many-to-many, denormalizing shifts consistency work to application code; emulated joins are more complex and slower.
- Verdict: "It's not possible to say in general which data model leads to simpler application code". For highly interconnected data, "the document model is awkward, the relational model is acceptable, and graph models … are the most natural."

Schema flexibility in the document model

- Most document databases and JSON support in relational ones enforce no schema; clients have no guarantees about fields. Calling them schemaless is misleading, since reading code assumes a structure: an implicit schema. Better terms: **schema-on-read** versus **schema-on-write**, like dynamic versus static type checking; "in general there's no right or wrong answer".
- Changing format (full name to first and last name): the document database just writes new documents and application code handles old ones (a code snippet). A relational database migrates (ALTER TABLE ADD COLUMN, then UPDATE; code shown).

- The bad reputation of schema changes is “not entirely deserved”: most run ALTER TABLE in milliseconds (MySQL copies the table). UPDATE on a large table is slow anywhere; if unacceptable, leave first_name NULL and fill at read time.
- Schema-on-read helps when items are heterogeneous, e.g. many object types not practical to put in separate tables, or structure determined by external systems you do not control; there a schema “may hurt more than it helps”; where records share structure, schemas document and enforce it. More in Chapter 4.

Data locality for queries

- A document is stored as one continuous string (JSON, XML, BSON), so if you need the whole document there is a locality advantage over multiple index lookups and disk seeks.
- It applies only if you need large parts at once: the whole document is loaded and usually rewritten on update. Hence keep documents small; these limits “significantly reduce the set of situations in which document databases are useful”.
- Locality is not unique to documents: Spanner interleaves child rows in a parent table, Oracle has multi-table index cluster tables, Bigtable column-families serve a similar purpose. More in Chapter 3.

Convergence of document and relational databases

- Most relational systems support XML and JSON; on the document side, RethinkDB supports joins and some MongoDB drivers resolve references client-side (likely slower).
- “relational and document databases are becoming more similar over time, and that is a good thing: the data models complement each other.” A footnote: Codd’s original model allowed “nonsimple domains” (nested relations). “A hybrid of the relational and document models is a good route for databases to take in the future.”

Query Languages for Data

- The relational model brought a new way to query: SQL is **declarative**, whereas IMS and CODASYL used **imperative** code (footnote: typically COBOL iterating record by record).
- Imperative example: a getSharks loop. Relational algebra: σ family = “Sharks” (animals), σ being the selection operator. SQL, which “followed the structure of the relational algebra fairly closely”: SELECT * FROM animals WHERE family = ‘Sharks’.
- Imperative: instructions “in a certain order”. Declarative: “you just specify the pattern of the data you want … but not how to achieve that goal”; the optimizer picks indexes, join methods and order.
- Why declarative: more concise; it “hides implementation details of the database engine”, allowing improvements without changing queries. E.g. reordering records to reclaim disk space is safe for SQL, which guarantees no ordering, but not for imperative code; SQL’s limits give “much more room for automatic optimizations”.
- Declarative languages suit parallel execution: CPUs gain cores, not clock speed; imperative code is hard to parallelize.

Declarative Queries on the Web

- The advantage is not limited to databases. Example: a page with navigation list items, where the “Sharks” item has class “selected”. To give the selected title a blue background, CSS `li.selected > p` declares the pattern; XSL with XPath `li[@class=’selected’]/p` does the same; both are declarative styling languages.
- The imperative DOM JavaScript version is “awful”: longer, harder to understand, and the blue is not removed when the class changes (CSS removes it automatically); using a new API such as getElementsByClassName means rewriting code, while browser vendors can speed up CSS and XPath without breaking compatibility.
- Conclusion: declarative CSS beats imperative JavaScript in browsers, and “declarative query languages like SQL turned out to be much better than imperative query APIs”.

MapReduce Querying

- MapReduce is a model for bulk processing across many machines, popularized by Google; MongoDB and CouchDB support a limited form for read-only queries over many documents (full description in Chapter 10).

- It is “neither a declarative query language nor a fully imperative query API, but somewhere in between”: code snippets called repeatedly by the framework, based on map (collect) and reduce (fold, inject).
- Example: a marine biologist counting shark sightings per month. PostgreSQL: filter family = ‘Sharks’, group by month, add up num_animals.
- MongoDB MapReduce: the filter is declarative (a MongoDB extension); map is called per matching document, emitting a key (year-month) and the animal count; pairs are grouped by key; reduce is called once per key and sums; output goes to monthlySharkReport. Two sample documents (3 and 4 animals, December 1995) give reduce(“1995-12”, [3, 4]) returning 7.
- Map and reduce must be **pure functions**: only their input, no extra queries, no side effects, so the database can run them anywhere, in any order, and rerun on failure; yet they are powerful.
- MapReduce is “a fairly low-level programming model for distributed execution”; SQL can run as MapReduce (Chapter 10), but nothing confines SQL to one machine, and MapReduce “doesn’t have a monopoly on distributed query execution”.
- Two coordinated functions are harder than one query, and a declarative language gives the optimizer more room, so MongoDB 2.2 added the declarative **aggregation pipeline** (same query shown with $match and $group), similar to a SQL subset with JSON syntax: “a matter of taste”. Moral: “a NoSQL system may find itself accidentally reinventing SQL, albeit in disguise.”

Graph-Like Data Models

- Many-to-many relationships distinguish data models. Mostly one-to-many (trees) or no relationships: the document model is appropriate. When many-to-many is common and connections grow complex, “it becomes more natural to start modeling your data as a graph”.
- A graph has **vertices** (nodes, entities) and **edges** (relationships, arcs). Examples: social graphs, the web graph, road or rail networks. Algorithms: shortest path in car navigation, PageRank on the web graph.
- Graphs can also hold completely different types of objects in one store: Facebook keeps one graph of people, locations, events, checkins and comments.
- Running example, Figure 2-5: Lucy from Idaho and Alain from Beaune, France, married and living in London.

Figure 2-5. Example of graph-structured data (boxes represent vertices, arrows represent edges).

- The graph used throughout the section.
- Two related structures are covered: the **property graph** (Neo4j, Titan, InfiniteGraph) and the **triple-store** (Datomic, AllegroGraph); three declarative languages: Cypher, SPARQL, Datalog; also imperative Gremlin and frameworks like Pregel (Chapter 10).

Property Graphs

- A vertex has a unique identifier, outgoing edges, incoming edges and key-value properties; an edge has an identifier, tail vertex, head vertex, a label for the kind of relationship, and properties. A graph store can be seen as two relational tables, one for vertices and one for edges (PostgreSQL json for properties); incoming or outgoing edges are found by querying head_vertex or tail_vertex.

Example 2-2. Representing a property graph using a relational schema

- The two-table schema with indexes on edge tails and heads.
- Aspects: any vertex can connect to any other, with no restricting schema; incoming and outgoing edges are found efficiently, so you can traverse both ways (hence both indexes); different labels store many kinds of information while keeping “a clean data model”. This gives flexibility: Figure 2-5 shows hard-to-express things in a relational schema: different regional structures (France has départements and régions; the US has counties and states), a country within a country, and varying granularity (Lucy’s residence a city, her birthplace only a state).
- Extension: allergens linked to foods and people. “Graphs are good for evolvability”.

The Cypher Query Language

- Cypher is a declarative language for property graphs, created for Neo4j (named after a character in The Matrix, unrelated to cryptographic ciphers).

- Example 2-3 inserts the left part of Figure 2-5; each vertex has a symbolic name (USA, Idaho) and arrow notation creates edges: (Idaho) -[:WITHIN]-> (USA).

Example 2-3. A subset of the data in Figure 2-5 , represented as a Cypher query

- Creates continent, country and state locations, Lucy, and the WITHIN and BORN_IN edges.
- Question: find people who emigrated from the United States to Europe: a BORN_IN edge to a location within the US and a LIVING_IN edge to one within Europe.

Example 2-4. Cypher query to find people who emigrated from the US to Europe

- A MATCH clause with arrow patterns, `:WITHIN*0..` chains and RETURN person.name; `(person) -[:BORN_IN]-> ()` binds the tail to person and leaves the head unnamed. The source reads it as a two-condition search ending in "return the name property".
- Execution: scan all people and check each, or start from the two Location vertices (via an index) and work backward along incoming edges. As is typical of declarative languages, "you don't need to specify such execution details": the optimizer picks the likely most efficient.

Graph Queries in SQL

- Graph data in a relational structure can be queried with SQL, "but with some difficulty": joins are normally known in advance, while graph queries traverse a variable number of edges, e.g. `:WITHIN*0..` ("follow a WITHIN edge, zero or more times"), since LIVES_IN may point to a street, city, region or state.
- Since SQL:1999, this is expressible with recursive common table expressions (WITH RECURSIVE; PostgreSQL, DB2, Oracle, SQL Server), "very clumsy in comparison to Cypher".

Example 2-5. The same query as Example 2-4 , expressed in SQL using recursive common table expressions

- Builds in_usa and in_europe by recursive traversal of within edges, then born_in_usa and lives_in_europe, then joins them.
- "If the same query can be written in 4 lines in one query language but requires 29 lines in another, that just shows that different data models are designed to satisfy different use cases."

Triple-Stores and SPARQL

- The triple-store model is "mostly equivalent" to the property graph, with different words, but its tools are worth knowing.
- All information is three-part statements (subject, predicate, object): in (Jim, likes, bananas), likes is the verb. The subject equals a vertex; the object is either a primitive value (then predicate and object are like a property key and value, (lucy, age, 33)) or another vertex (then the predicate is an edge, (lucy, marriedTo, alain)).

Example 2-6. A subset of the data in Figure 2-5 , represented as Turtle triples

- One triple per line. Vertices are written _:someName, meaningful only within the file. An edge predicate has a vertex object; a property predicate has a string literal.

Example 2-7. A more concise way of writing the data in Example 2-6

- Semicolons say several things about the same subject, so Turtle is "quite nice and readable".

The semantic web

- Triple-stores are independent of the semantic web (Datomic is one).
- The idea is "fundamentally a simple and reasonable idea": websites could publish machine-readable data as well as text and pictures; RDF was to publish data consistently so it combines into "a web of data", an internet-wide "database of everything".
- It was overhyped, showed no sign of being realized, and suffered "a dizzying plethora of acronyms, overly complex standards proposals, and hubris". Yet triples "can be a good internal data model for applications".

The RDF data model

- Turtle is a human-readable RDF format; RDF/XML does the same "much more verbosely" (Example 2-8); Turtle/N3 is preferable.

Example 2-8. The data of Example 2-7 , expressed using RDF/XML syntax

- The same data as nested XML.
- RDF's quirks come from internet-wide exchange: subject, predicate and object are often URIs (namespaces), so data from different sources does not conflict if they give different meanings to within or lives_in. The book uses non-resolvable urn:example: URIs.

The SPARQL query language

- SPARQL (SPARQL Protocol and RDF Query Language, "sparkle") predates Cypher, which borrowed its pattern matching, so they look similar; the emigration query is "even more concise" in SPARQL.

Example 2-9. The same query as Example 2-4 , expressed in SPARQL

- Variables start with a question mark; paths like :bornIn / :within* correspond to Cypher's pattern. Since RDF uses predicates for both properties and edges, the same syntax matches properties.
- "SPARQL is a nice query language—even if the semantic web never happens, it can be a powerful tool for applications to use internally."

Graph Databases Compared to the Network Model

- Are graph databases "the second coming of CODASYL in disguise"? "No." CODASYL's schema fixed which record type nests in which; any vertex can link to any other. CODASYL reached records via access paths; graphs by unique ID or index. CODASYL children were an ordered set; graph elements are unordered. CODASYL queries were imperative; graph databases mostly offer declarative Cypher or SPARQL.

The Foundation: Datalog

- Datalog is much older, studied in the 1980s, but "provides the foundation that later query languages build upon". Used in Datomic and Cascalog (a footnote: they use Clojure syntax; the book uses Prolog's).
- Its model generalizes triple-stores: predicate(subject, object).

Example 2-10. A subset of the data in Figure 2-5 , represented as Datalog facts

- The example data as facts such as within(usa, namerica) and born_in(lucy, idaho).

Example 2-11. The same query as Example 2-4 , expressed in Datalog

- Datalog takes "a small step at a time", defining **rules** for derived predicates (within_recursive and migrated), which can refer to other rules, like functions. Uppercase words are variables. A rule applies if all predicates on the right of :- match; then the left side is as though added to the database.
- Applying rules: name(namerica, 'North America') makes rule 1 produce within_recursive(namerica, 'North America'); rule 2 with within(usa, namerica) produces it for usa, then for idaho. Repeating yields all locations within a place (Figure 2-6); rule 3 finds people born in BornIn and living in LivingIn; querying with 'United States' and 'Europe' and leaving Who open yields the same answer as Cypher and SPARQL.

Figure 2-6. Determining that Idaho is in North America, using the Datalog rules from Example 2-11 .

- Shows the chain of rule applications.
- Different thinking, but "very powerful" since rules combine and are reused; less convenient for simple one-off queries, "but it can cope better if your data is complex."

Summary

- Wrap-up: data began as one big tree (hierarchical), poor for many-to-many, so the relational model was invented; NoSQL stores diverged in two directions (document databases target self-contained documents with rare relationships between them; graph databases "go in the opposite direction", where anything is potentially related to everything).
- All three (document, relational, graph) are widely used and "each is good in its respective domain". One can emulate another (graph data in a relational database), but the result is often awkward: "That's why we have different systems for different purposes, not a single one-size-fits-all solution."
- Document and graph databases typically enforce no schema, easing change, but the application still assumes a structure: explicit (on write) or implicit (on read).

- Each model has its query language: SQL, MapReduce, MongoDB aggregation pipeline, Cypher, SPARQL, Datalog; CSS and XSL/XPath are not database languages but parallel.
- Unmentioned models: genome sequence-similarity search (specialized software), particle physics at hundreds of petabytes (custom solutions), and full-text search, often used alongside databases (Chapter 3, Part III). Next: trade-offs in implementing these models.

Footnotes

- Footnotes are covered where they annotate.

References

- The author ends the chapter with references (not summarised).

How this chapter builds the author's case

The author's case: no model fits all, history repeats (hierarchical to relational; document stores meeting the same limits), and declarative languages leave the optimizer room. Forward links: Chapter 3 (implementation, locality), 4 (schema evolution, JSON), 5 and 7, 10 (MapReduce, Pregel), Part III (denormalization, derived data).

CHAPTER 3

Storage and Retrieval

Place in the argument — why this chapter, why this title

Title names the two jobs of a database: “when you give it some data, it should store the data, and when you ask it again later, it should give the data back to you.” Chapter 2 gave the developer’s side (data models, query languages); this chapter takes “the same from the database’s point of view”. Why care? You won’t write a storage engine, but “you do need to select a storage engine that is appropriate for your application”, and tuning needs “a rough idea of what the storage engine is doing under the hood.”

Place in the argument — why this chapter, why this title

Order: engines of familiar databases first (log-structured, then page-oriented B-trees), then the OLTP/analytics distinction, then column-oriented storage for analytics, then the Summary.

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph: if you keep things tidily ordered, you’re just too lazy to go searching.
- Promised: “a big difference between storage engines that are optimized for transactional workloads and those that are optimized for analytics”. First, engines in “traditional relational databases, and also most so-called NoSQL databases”: “log-structured storage engines, and page-oriented storage engines such as B-trees.”

Data Structures That Power Your Database

- The world’s simplest database: two Bash functions, db_set (appends key,value to a file) and db_get (grep for the key, last match via tail -n 1). A key-value store; the value could be a JSON document. Updates append, so the latest value is the last occurrence.
- db_set is fast “because appending to a file is generally very efficient.” Many databases use a **log**, “an append-only data file”; real ones add concurrency control, reclaiming disk space, and handling partially written records, “but the basic principle is the same.” Logs recur throughout the book.

Note

- “Log” here means “an append-only sequence of records”, possibly binary, not application logs.
- db_get scans the whole file: lookup cost O(n), double the records and a lookup takes twice as long.
- Remedy: an **index**, “an additional structure that is derived from the primary data”, metadata that “acts as a signpost”. Indexes affect only query performance, not contents.
- The trade-off: “well-chosen indexes speed up read queries, but every index slows down writes.” Appending is the simplest write, so databases don’t index everything by default; you choose from knowledge of query patterns.

Hash Indexes

- Key-value data is “a useful building block for more complex indexes.” Key-value stores resemble hash maps, so why not use one to index data on disk?
- Simplest strategy: in-memory hash map from every key to a byte offset in the append-only file (Figure 3-1), updated on each append; read = lookup offset, seek, read. This is “essentially what Bitcask (the default storage engine in Riak) does”: fast reads and writes provided all keys fit in RAM; values can exceed memory (one disk seek, or none if cached).
- Suits frequent updates to few distinct keys, e.g. a cat video’s play count.
- To avoid running out of disk: break the log into segments and run **compaction**, “throwing away duplicate keys in the log, and keeping only the most recent update for each key” (Figure 3-2). Compaction shrinks segments, so merge them at the same time (Figure 3-3); segments are never modified, the merged one goes to a new file, done in a background thread while reads and writes use old segments, then old files are deleted.
- Each segment has its own hash table; lookup checks the newest segment first, then older ones; merging keeps the number small.

- Real-implementation issues: binary format (length then raw string) instead of CSV; deletion via a special record, a **tombstone**, that tells merging to discard earlier values; restart loses hash maps, so Bitcask stores snapshots; crashes mid-append handled by checksums; usually one writer thread, with immutable segments read concurrently.
- Why append-only rather than update in place: “Appending and segment merging are sequential write operations, which are generally much faster than random writes”; “Concurrency and crash recovery are much simpler if segment files are append-only or immutable” (no file with part old and part new value spliced together); merging “avoids the problem of data files getting fragmented over time.”
- Limitations: “The hash table must fit in memory”; an on-disk hash map “requires a lot of random access I/O, it is expensive to grow when it becomes full, and hash collisions require fiddly logic”. “Range queries are not efficient”: you cannot scan keys kitty00000 to kitty99999, you’d look up each key individually. He promises a structure without these limits.

SSTables and LSM-Trees

- In Figure 3-3 order within a segment doesn’t matter apart from later values winning. Change: require key-value pairs “sorted by key”. This seems to break sequential writes; he returns to it.
- Called **Sorted String Table**, or SSTable; each key once per merged segment. Advantages over hash-indexed segments:
- Merging is “simple and efficient, even if the files are bigger than the available memory”, like mergesort (Figure 3-4): read input files side by side, copy the lowest key to the output, repeat. For a key in several segments, all values in one input segment are more recent than in the other (adjacent segments), so keep the most recent segment’s value.
- No index of all keys in memory (Figure 3-5): to find handiwork, jump to the offset of handbag and scan, since handiwork lies between handbag and handsome. The in-memory index “can be sparse: one key for every few kilobytes of segment file is sufficient”. Footnote: with fixed-size records you could binary search.
- Since reads scan several pairs in a range anyway, group them in a block and compress it; each index entry points at a block start, saving disk space and I/O bandwidth.

Constructing and maintaining SSTables

- Problem: writes arrive in any order. Sorting on disk is possible (see “B-Trees”) but “maintaining it in memory is much easier” with red-black or AVL trees.
- Steps: a write goes into an in-memory balanced tree (the **memtable**); past a threshold, typically a few megabytes, it is written as an SSTable, the newest segment, while writes continue to a new memtable; a read tries the memtable, then the newest segment, then older ones; in the background, merge and compaction discard overwritten or deleted values.
- A crash loses the memtable’s recent writes. Fix: a separate unsorted log, appended on every write; “its only purpose is to restore the memtable after a crash”, discarded when the memtable is written as an SSTable.

Making an LSM-tree out of SSTables

- This is used in LevelDB and RocksDB; similar engines in Cassandra and HBase, inspired by Google’s Bigtable paper (which introduced the terms SSTable and memtable). Originally the Log-Structured Merge-Tree (LSM-Tree) of O’Neil et al.; such engines are “often called LSM storage engines.”
- Lucene (Elasticsearch, Solr) uses a similar method: a term (word) maps to a postings list of document IDs, kept in SSTable-like files merged in the background.

Performance optimizations

- Lookups of nonexistent keys must check the memtable and all segments, so engines “often use additional Bloom filters”, a memory-efficient approximation of a set that can tell you a key does not appear.
- Compaction strategies: size-tiered (newer, smaller SSTables merged into older, larger) and leveled (key range split, data in “levels”, more incremental, less disk space). LevelDB and RocksDB: leveled; HBase: size-tiered; Cassandra: both.
- Conclusion: “keeping a cascade of SSTables that are merged in the background—is simple and effective”, works beyond memory, supports range queries, and sequential writes give “remarkably high write throughput.”

B-Trees

- Log-structured indexes “are not the most common type of index”; the **B-tree** (1970) is, standard in almost all relational databases. Like SSTables it keeps pairs sorted by key, “that’s where the similarity ends”.
- Log-structured: variable-size segments of several megabytes or more, written sequentially. B-trees: fixed-size blocks or **pages**, traditionally 4 KB, read or written one at a time, matching disks’ fixed-size blocks.
- Pages refer to each other by address, forming a tree (Figure 3-6): a root page, child references with boundary keys, down to a **leaf page** holding values inline or references. Example: key 251 lies between boundaries 200 and 300.
- **Branching factor**: references to child pages per page; six in the figure, in practice typically several hundred.
- Update: change the leaf and write it back. Insert: if the page is full, split into two half-full pages and update the parent (Figure 3-7). This keeps the tree **balanced**, depth O(log n); “A four-level tree of 4 KB pages with a branching factor of 500 can store up to 256 TB.” Footnote: deletion is more involved.

Making B-trees reliable

- The basic write overwrites a page in place, “in stark contrast to log-structured indexes” that never modify files. On SSDs, large blocks must be erased and rewritten.
- A split overwrites several pages; a crash midway corrupts the index (e.g. an orphan page). So implementations add a **write-ahead log** (WAL, also known as a redo log): every modification is appended there before being applied, and used to restore consistency after a crash.
- In-place updates also need latches (lightweight locks) for concurrency; log-structured approaches are simpler, merging in the background and atomically swapping segments.

B-tree optimizations

- Many optimizations developed over the years; “just a few”:
- Copy-on-write instead of WAL (LMDB): a modified page is written elsewhere and a new version of the parent pages points to it; also useful for concurrency control (see “Snapshot Isolation and Repeatable Read”).
- Abbreviate keys: interior keys “only need to provide enough information to act as boundaries”, so more keys per page, higher branching factor, fewer levels.
- Pages can be anywhere on disk, so a sorted range scan may need a seek per page; many implementations try to lay leaf pages out sequentially, “difficult to maintain that order as the tree grows”; LSM-trees keep sequential keys close more easily since they rewrite large segments.
- Sibling pointers on leaf pages allow ordered scans without returning to parents. Fractal trees borrow log-structured ideas to reduce seeks. Footnote: this variant is sometimes known as a B+ tree.

Comparing B-Trees and LSM-Trees

- Rule of thumb: “LSM-trees are typically faster for writes, whereas B-trees are thought to be faster for reads”, since LSM reads check several structures. But “benchmarks are often inconclusive and sensitive to details of the workload”: test with your workload.

Advantages of LSM-trees

- A B-tree writes data at least twice (WAL and page), writes whole pages for a few changed bytes. LSM-trees also rewrite via compaction: **write amplification**, “one write to the database resulting in multiple writes to the disk over the course of the database’s lifetime”, a concern on SSDs, and a direct cost in write-heavy applications.
- LSM-trees typically sustain higher write throughput, partly lower write amplification, partly sequential writes of compact files, important on magnetic drives.
- They compress better; B-trees waste space on fragmentation, LSM-trees rewrite SSTables to remove it.
- SSD firmware often uses log-structured algorithms internally, but lower write amplification and fragmentation still help.

Downsides of LSM-trees

- Compaction can interfere with ongoing reads and writes; throughput and average response time are usually fine, but at higher percentiles (see “Describing Performance”) latency can be high and B-trees more predictable.
- Disk bandwidth is shared between writes and compaction; if compaction cannot keep up, unmerged segments grow until the disk fills and reads slow. SSTable engines typically do not throttle writes, so “you need explicit monitoring”.
- B-tree advantage: each key in exactly one place, so range locks for transaction isolation attach directly to the tree (Chapter 7).
- B-trees are unlikely to go away; log-structured indexes are increasingly popular. “There is no quick and easy rule”, so test empirically.

Other Indexing Structures

- Key-value indexes are like a primary key index. Also common: **secondary indexes** (CREATE INDEX), crucial for joins. Built from a key-value index where keys aren’t unique: store a list of row identifiers per key, or append a row identifier to make keys unique. B-trees and log-structured indexes both work.

Storing values within the index

- The value is either the row itself or a reference into a **heap file** (no particular order), which avoids duplicating data across secondary indexes. Updating a larger value may move it: update all indexes or leave a forwarding pointer.
- To avoid the extra hop, store the row in the index: a **clustered index** (InnoDB primary key; SQL Server one per table). A compromise, the **covering index** (index with included columns), stores some columns so the index can “cover” a query.
- Duplication speeds reads but costs storage, write overhead, and effort to keep transactional guarantees.

Multi-column indexes

- **Concatenated index**: fields appended into one key, like a phone book on (lastname, firstname); “useless if you want to find all the people with a particular first name.”
- **Multi-dimensional indexes** matter for geospatial data: a map view is a two-dimensional range query on latitude and longitude. B-tree or LSM-tree gives one range “but not both simultaneously.” Options: space-filling curve plus B-tree, or R-trees (PostGIS).
- Also useful beyond geography: colors (red, green, blue), or (date, temperature) for 2013 observations between 25 and 30°C.

Full-text search and fuzzy indexes

- Earlier indexes assume exact data; fuzzy querying (misspellings, synonyms, grammatical variations, nearby words) needs other techniques. Lucene searches within an edit distance.
- Lucene’s in-memory index over its term dictionary is a finite state automaton similar to a trie (LevelDB uses a sparse key collection), transformable into a Levenshtein automaton. Other fuzzy techniques lean toward machine learning.

Keeping everything in memory

- Everything so far answers “the limitations of disks”, tolerated because disks are durable and cheaper per gigabyte. As RAM gets cheaper, **in-memory databases** appear.
- Memcached is for caching; others get durability via battery-powered RAM, change logs, snapshots, or replication. Writing a log to disk still counts: “the disk is merely used as an append-only log for durability”. VoltDB, MemSQL, Oracle TimesTen (relational), RAMCloud, Redis, Couchbase (asynchronous writes, weak durability).
- “Counterintuitively”, the speed is not from avoiding disk reads but from avoiding “the overheads of encoding in-memory data structures in a form that can be written to disk.” They also enable models hard to do on disk, e.g. Redis priority queues and sets.
- **Anti-caching** evicts least recently used records to disk while indexes stay in memory. Non-volatile memory may require further changes: “worth keeping an eye on”.

Transaction Processing or Analytics?

- A **transaction** began as a commercial transaction; the term stuck for "a group of reads and writes that form a logical unit."

Note

- It needn't be ACID; it "just means allowing clients to make low-latency reads and writes" as opposed to batch processing (Chapter 7, Chapter 10).
- Typical access: a small number of records looked up by key via an index, inserted or updated from user input: **online transaction processing (OLTP)**.
- **Data analytics** instead scans a huge number of records, reads a few columns, and computes aggregates (count, sum, average); example queries: total revenue of each store in January, how many more bananas than usual during a promotion, which baby food brand is most often bought with brand X diapers. These feed **business intelligence**; the pattern is called **online analytic processing (OLAP)**. Footnote: "online" is unclear.
- Table 3-1:

Property	OLTP	OLAP
Main write pattern	Random-access, low-latency writes from user input	Bulk import (ETL) or event stream
Primarily used by	End user/customer, via web application	Internal analyst, for decision support
What data represents	Latest state of data (current point in time)	History of events that happened over time
Dataset size	Gigabytes to terabytes	Terabytes to petabytes

- SQL served both at first, but from the late 1980s companies ran analytics on a separate database: the **data warehouse**.

Data Warehousing

- OLTP systems must be highly available and low-latency, so administrators resist expensive ad hoc analytic queries that "can harm the performance of concurrently executing transactions."
- A warehouse is a read-only copy of all OLTP data, extracted (dump or stream), transformed, cleaned and loaded: **Extract–Transform–Load (ETL)** (Figure 3-8). Large enterprises have them; small companies rarely, having fewer systems and less data.
- Main advantage: optimization for analytic access. The earlier indexing algorithms "work well for OLTP, but are not very good at answering analytic queries".

The divergence between OLTP databases and data warehouses

- Warehouses are mostly relational (SQL fits), with graphical tools for drill-down and slicing and dicing. Both look alike on the surface, but internals differ; vendors mostly pick one. SQL Server and SAP HANA are "increasingly becoming two separate storage and query engines". Commercial warehouses (Teradata, Vertica, ParAccel/RedShift) now meet open source SQL-on-Hadoop projects, some based on Google's Dremel.

Stars and Snowflakes: Schemas for Analytics

- Transaction processing uses diverse models; analytics has "much less diversity": a formulaic **star schema** (dimensional modeling).
- Figure 3-9 (grocery retailer): a central **fact table** (fact_sales), each row an event, here a purchase. Facts are individual events for maximum flexibility, so the table can reach tens of petabytes.
- Fact columns are attributes (price, cost) or foreign keys to **dimension tables**, the who, what, where, when, how and why of the event (dim_product; even dates, to encode holidays). The fact table in the middle with dimension tables around it looks like a star.
- **Snowflake schema**: dimensions broken into subdimensions; more normalized, but "star schemas are often preferred because they are simpler for analysts to work with."
- Tables are wide: fact tables often over 100 columns, dimension tables too (dim_store).

Column-Oriented Storage

- With trillions of rows, storage of facts is the challenge (dimension tables are usually millions of rows). A typical query accesses only 4 or 5 of 100+ columns.

Example 3-1. Analyzing whether people are more inclined to buy fresh fruit or candy, depending on the day of the week

- A join of fact_sales with dim_date and dim_product, for 2013 and two categories, summing quantity by weekday and category. It touches many rows but only three fact_sales columns. "How can we execute this query efficiently?"
- Row-oriented storage (OLTP, documents, as in Figure 3-1) must load all rows with 100+ attributes, parse, and filter: "That can take a long time."
- **Column-oriented storage**: "don't store all the values from one row together, but store all the values from each column together instead." Read only needed column files (Figure 3-10).

Note

- Applies to nonrelational data too: Parquet, based on Dremel.
- Column files hold rows in the same order, so the 23rd row is the 23rd entry of each.

Column Compression

- Columns "lends itself very well to compression" since values are repetitive. Particularly effective: **bitmap encoding** (Figure 3-11): n distinct values become n bitmaps, one bit per row (1 if the row has that value).
- If n is small, one bit per row; if bigger, bitmaps are **sparse** and can be run-length encoded, "remarkably compact".
- Queries suit it: WHERE product_sk IN (30, 68, 69) is a bitwise OR of three bitmaps; product_sk = 31 and store_sk = 3 is a bitwise AND, valid because the kth bit in one bitmap is the same row as the kth bit in another.

Column-oriented storage and column families

- Cassandra and HBase's column families (from Bigtable): "it is very misleading to call them column-oriented": columns of a row are stored together with a row key and no column compression. So the "Bigtable model is still mostly row-oriented."

Memory bandwidth and vectorized processing

- Disk-to-memory bandwidth is a big bottleneck but not the only one: also memory-to-CPU-cache bandwidth, branch mispredictions, pipeline bubbles, SIMD instructions.
- Columns suit this: iterate through compressed column chunks fitting the L1 cache in a tight loop with no function calls; compression fits more rows; bitwise AND and OR operate on compressed chunks directly: **vectorized processing**.

Sort Order in Column Storage

- Insertion order is easiest (append to each file), but we can impose an order, as with SSTables, as an indexing mechanism. Never sort columns independently; "the data needs to be sorted an entire row at a time", though stored by column.
- The administrator picks sort keys from common queries: date_key first for date ranges; product_sk second to group same-product sales on a day.
- Sorting helps compression: a first sort column with few distinct values gives long runs, so run-length encoding could shrink it to "a few kilobytes—even if the table has billions of rows". Strongest on the first key, weaker later, but "having the first few columns sorted is still a win overall."

Several different sort orders

- From C-Store, adopted by Vertica: data is replicated anyway, so store copies in different orders and pick the best for each query. Like secondary indexes in a row store, except there "normally aren't any pointers to data elsewhere, only columns containing values."

Writing to Column-Oriented Storage

- Optimizations suit read-only analyst queries but make writes harder: update in place is impossible with compressed columns, and inserting mid-table would rewrite all column files.

- Solution: LSM-trees. Writes go to an in-memory sorted store (row- or column-oriented), then merge into column files in bulk, "essentially what Vertica does". Queries combine disk and memory; the optimizer hides this, so changes are immediately visible.

Aggregation: Data Cubes and Materialized Views

- Columnar is not universal but "rapidly gaining popularity". For shared aggregates (COUNT, SUM, AVG, MIN, MAX): "Why not cache some of the counts or sums that queries use most often?"
- A **materialized view** is an actual copy of query results on disk, unlike a virtual view, which is expanded on the fly. As a denormalized copy it must be updated, making writes costlier, so rarely used in OLTP but sensible in read-heavy warehouses.
- A **data cube** (OLAP cube) is a grid of aggregates by dimensions (Figure 3-12): date by product, each cell a SUM of e.g. net_price; summing along rows or columns drops a dimension. Figure 3-9 has five dimensions (hypercube), same principle.
- Advantage: queries like total sales per store yesterday are precomputed. Disadvantage: less flexibility (no share of sales from items over $100, since price isn't a dimension). So warehouses keep raw data and use cubes "only as a performance boost for certain queries."

Summary

- Storage engines split into OLTP- and OLAP-optimized. OLTP systems are user-facing with huge request volume; each query touches few records by key via an index, and "Disk seek time is often the bottleneck". Warehouses are less well known, used by analysts, with far fewer but very demanding queries scanning millions of records; "Disk bandwidth (not seek time) is often the bottleneck", and column-oriented storage is increasingly popular.
- OLTP engines come from two schools: the log-structured school ("only permits appending to files and deleting obsolete files, but never updates a file that has been written": Bitcask, SSTables, LSM-trees, LevelDB, Cassandra, HBase, Lucene) and the update-in-place school (disk as fixed-size pages that can be overwritten; B-trees the biggest example). Log-structured engines "systematically turn random-access writes into sequential writes on disk", enabling higher write throughput. Then more complex indexes and in-memory databases.
- The warehouse detour shows why analytics differ: "when your queries require sequentially scanning across a large number of rows, indexes are much less relevant"; data must be encoded compactly, which column-oriented storage achieves.
- Payoff: you can pick the best tool and "imagine what effect a higher or a lower value may have" when tuning; the chapter gives "enough vocabulary and ideas that you can make sense of the documentation for the database of your choice."

Footnotes

- (i) fixed-size keys would allow binary search without an in-memory index; (ii) B-tree deletion is more involved; (iii) the leaf-linking variant is sometimes known as a B+ tree; (iv) "online" in OLAP is unclear.

References

- The author ends the chapter with references (not summarised).

How this chapter builds the author's case

The chapter answers Chapter 2 from the engine's side: from the append-only log to hash indexes, SSTables/LSM-trees and B-trees, with trade-offs and testing empirically as the lesson; then OLTP versus OLAP, star schemas, columns, compression, sort orders and LSM-style writes show why analytics needs different engines. Forward: Chapter 7 (transactions, range locks), Chapter 10 (batch processing); logs recur "several times in the rest of this book."

CHAPTER 4

Encoding and Evolution

Place in the argument — why this chapter, why this title

Title pairs two ideas: encoding data as bytes, and evolution of applications over time. Chapter 2's data models cope differently: relational has "exactly one schema in force at any one point in time"; schema-on-read a "mixture of older and newer data formats". This chapter asks how encodings let old and new code and data coexist, then how data flows between processes.

Place in the argument — why this chapter, why this title

Order: formats first (language-specific, JSON/XML, Thrift/Protocol Buffers, Avro, merits of schemas), because compatibility is a property of an encoding; then modes of dataflow (databases, services, message passing), because compatibility is "a relationship between one process that encodes the data, and another process that decodes it."

Chapter opening — untitled section before the first sub-heading in the book

- Applications inevitably change, and a feature change usually changes stored data. Code changes in a large application "often cannot happen instantaneously", so old and new code and data formats "may potentially all coexist in the system at the same time."
- Hence compatibility in both directions: newer code reads data written by older code (**backward compatibility**) and older code reads data written by newer code (**forward compatibility**). Backward is "normally not hard"; forward "can be trickier, because it requires older code to ignore additions made by a newer version of the code."
- A **rolling upgrade** (staged rollout) deploys to a few nodes at a time, "and thus encourages more frequent releases and better evolvability"; client-side, "you're at the mercy of the user".

Formats for Encoding Data

- Programs use (at least) two representations. In memory: structures optimized for the CPU, "typically using pointers"; for a file or network: a "self-contained sequence of bytes", since a pointer wouldn't make sense to any other process. So translation is needed: **encoding** (also serialization, marshalling) and **decoding** (parsing, deserialization, unmarshalling). Footnote: encoding has nothing to do with encryption.

Terminology clash

- Serialization also has a different meaning for transactions (Chapter 7); he sticks with encoding "overloading the word".

Language-Specific Formats

- Built-in support (Java Serializable, Ruby Marshal, Python pickle). Convenient, but with "a number of deep problems": the encoding is tied to one language, "committing yourself to your current programming language for potentially a very long time"; decoding must instantiate arbitrary classes, a security problem (an attacker can get "remotely executing arbitrary code"); versioning and efficiency are "often an afterthought" (Java's serialization is notorious for bad performance and bloat).
- Conclusion: "it's generally a bad idea to use your language's built-in encoding for anything other than very transient purposes."

JSON, XML, and Binary Variants

- JSON and XML are "widely known, widely supported, and almost as widely disliked." All three (with CSV) are textual and somewhat human-readable, but have subtle problems:
- Numbers are ambiguous: XML and CSV cannot distinguish a number from a string of digits; JSON has no integers versus floats and no precision. Integers greater than 2^53 become inaccurate in JavaScript, so Twitter's API returns tweet IDs twice, as number and string.
- No binary strings; Base64 is "somewhat hacky and increases the data size by 33%."

- XML and JSON have optional, complicated schemas; without them applications may hardcode encoding logic. CSV has no schema and vague escaping.
- They are good enough for many purposes, especially data interchange between organizations: “The difficulty of getting different organizations to agree on anything outweighs most other concerns.”

Binary encoding

- For internal data there is less pressure for a lowest-common-denominator format; at terabytes, format choice has big impact. JSON and XML use much space compared to binary, giving binary variants (MessagePack, BSON), none as widely adopted.
- They keep the data model; since “they don’t prescribe a schema, they need to include all the object field names within the encoded data.”

Example 4-1. Example record which we will encode in several binary formats in this chapter

- MessagePack bytes: 0x83 = object with three fields; 0xa8 = string of eight bytes, then the field name userName in ASCII; Martin gets prefix 0xa6; and so on.
- MessagePack (Figure 4-1) takes 66 bytes against 81 for textual JSON without whitespace. “It’s not clear whether such a small space reduction (and perhaps a speedup in parsing) is worth the loss of human-readability.”

Thrift and Protocol Buffers

- Apache Thrift (originally Facebook) and Protocol Buffers (Google), “based on the same principle”: both require a schema in an interface definition language (IDL), numbering each field and marking it required or optional, and both generate code.
- Thrift’s BinaryProtocol: 59 bytes (Figure 4-2); the big difference from Figure 4-1 is no field names but field tags, numbers from the schema: “Field tags are like aliases for fields”.
- CompactProtocol (Figure 4-3): 34 bytes, packing type and tag into one byte and using variable-length integers (1337 in two bytes, top bit signalling more bytes). Protocol Buffers (Figure 4-4): 33 bytes, similar bit packing.
- required versus optional makes no difference to encoding; required only enables a runtime check, “useful for catching bugs.”

Field tags and schema evolution

- **Schema evolution**: how do they keep backward and forward compatibility? An encoded record is a concatenation of fields, each with tag number and datatype; unset fields are omitted. So you can rename a field (names are never encoded) but “you cannot change a field’s tag, since that would make all existing encoded data invalid.”
- Adding a field with a new tag: old code ignores unknown tags (the datatype annotation says how many bytes to skip): forward compatible. Backward compatible as tags keep meaning, but a new field cannot be required: every field added after initial deployment “must be optional or have a default value.”
- Removing is the same with compatibility reversed: only optional fields can be removed, and a tag number can never be reused.

Datatypes and schema evolution

- Changing a datatype risks lost precision or truncation: widening 32-bit to 64-bit, old code truncates a value that doesn’t fit.
- Protocol Buffers has no list type but a repeated marker; the same tag appears several times, so an optional field can become repeated: old code reading new data sees only the last element.
- Thrift’s dedicated list datatype allows nested lists but not single-to-multi-valued evolution.

Avro

- Avro began in 2009 as a Hadoop subproject because Thrift was not a good fit. It uses a schema, with two languages: Avro IDL for humans and a JSON-based one for machines.
- The schema has “no tag numbers”. Encoding is 32 bytes, “the most compact of all the encodings we have seen” (Figure 4-5): “nothing to identify fields or their datatypes”, just values concatenated; strings are a length prefix and UTF-8; integers use variable-length encoding as in Thrift CompactProtocol.

- To parse, go through fields in schema order; data decodes correctly only if the reader uses "the exact same schema as the code that wrote the data".

The writer's schema and the reader's schema

- The **writer's schema** is the version the encoding application knows; the **reader's schema** is the one the decoding application expects.
- The key idea: the schemas don't have to be the same; "they only need to be compatible." The library resolves differences by comparing both side by side (Figure 4-6). Fields are matched by name; a writer-only field is ignored; a reader-only field gets the reader's declared default.

Schema evolution rules

- Forward compatibility: new schema as writer, old as reader. Backward: new schema as reader, old as writer. To stay compatible "you may only add or remove a field that has a default value." Adding a field without a default breaks backward compatibility; removing one breaks forward.
- Allowing null needs a **union type**, e.g. union { null, long, string }: "it helps prevent bugs by being explicit about what can and cannot be null." So Avro has no optional/required markers, but union types and defaults.
- Renaming uses aliases in the reader's schema: backward but not forward compatible; adding a union branch likewise.

But what is the writer's schema?

- The reader must know it, but including the schema with every record would make "all the space savings from the binary encoding futile." Three contexts: a large file of millions of records with one schema (Hadoop; Chapter 10), where the schema is included once at the start (object container files); a database, where each record starts with a version number and a list of schema versions is kept (Espresso); a bidirectional network connection, where schema version is negotiated at setup (Avro RPC).

Dynamically generated schemas

- Avro is friendlier to dynamically generated schemas: to dump a relational database, generate a record schema per table, each column a field. If the database schema changes, just regenerate; fields are matched by name.
- With Thrift or Protocol Buffers, field tags would likely be assigned by hand each time; this "simply wasn't a design goal" for them, whereas it was for Avro.

Code generation and dynamically typed languages

- Thrift and Protocol Buffers rely on code generation, useful in statically typed languages (type checking, IDE autocompletion); in dynamically typed ones there is little point, and for dynamically generated schemas it is "an unnecessarily obstacle".
- Avro's code generation is optional; an object container file embeds the writer's schema, so it is self-describing and can be opened like a JSON file (helpful for Apache Pig).

The Merits of Schemas

- Their schema languages are much simpler than XML Schema or JSON Schema.
- The ideas are not new: ASN.1 (1984) supports tag-number evolution but is "very complex and badly documented".
- Schema-based binary encodings have "a number of nice properties": more compact than "binary JSON" since they omit field names; the schema is valuable documentation that, being required for decoding, is surely up to date; a schema database lets you check compatibility before deployment; code generation allows compile-time type checking.
- In summary: "schema evolution allows the same kind of flexibility as schemaless/schema-on-read JSON databases provide… while also providing better guarantees about your data and better tooling."

Modes of Dataflow

- Compatibility is "a relationship between one process that encodes the data, and another process that decodes it." Who encodes and who decodes? Three ways: via databases, via service calls, via asynchronous message passing.

Dataflow Through Databases

- The writer encodes, the reader decodes. With a single process, storing something is "sending a message to your future self": backward compatibility is clearly necessary.
- Several processes usually access a database, perhaps old and new code during a rolling upgrade: forward compatibility is also often required.
- Snag: old code reads, updates and writes back a record containing a new field; "the desirable behavior is usually for the old code to keep the new field intact". The formats preserve unknown fields, but application-level care is needed (Figure 4-7): decoding into model objects and reencoding can lose the field. "Solving this is not a hard problem; you just need to be aware of it."

Different values written at different times

- A database may hold values from five milliseconds to five years ago. Code can be replaced in minutes but old data stays in its original encoding: "data outlives code". Migrating is expensive, so most databases avoid it.
- Schema evolution lets the whole database appear as if encoded with a single schema, though storing historical versions.

Archival storage

- Snapshots (backup, data warehouse loading) are typically encoded with the latest schema: "you might as well encode the copy of the data consistently." Written in one go and immutable, they suit Avro object container files or column-oriented Parquet (see "Column Compression").

Dataflow Through Services: REST and RPC

- Most common arrangement: **clients** and **servers**; the API a server exposes is a **service**. The web works this way (GET and POST over standardized HTTP, URLs, SSL/TLS, HTML).
- A server can be a client to another service: **service-oriented architecture (SOA)**, rebranded **microservices architecture**. Unlike databases, services expose predetermined inputs and outputs, giving encapsulation.
- Key design goal: services "independently deployable and evolvable", each owned by one team releasing frequently. So "old and new versions of servers and clients" run at once, and encodings must be compatible across API versions, "precisely what we've been talking about in this chapter."

Web services

- With HTTP as protocol it is a **web service**, used in several contexts: client apps calling a service over the internet; one service calling another in the same organization (middleware); or services of different organizations (public APIs, OAuth). Two approaches, REST and SOAP, "almost diametrically opposed in terms of philosophy" (footnote: HATEOAS provokes arguments).
- **REST** is "not a protocol, but rather a design philosophy that builds upon the principles of HTTP": simple data formats, URLs identifying resources, HTTP features for cache control, authentication and content type negotiation. Gaining on SOAP, and associated with microservices.
- **SOAP** is an XML-based protocol, aiming to be independent of HTTP, with a sprawling WS-* family of standards. Its WSDL-described API enables code generation (useful in static languages) but users rely heavily on tools and IDEs, and vendor interoperability causes problems. It "has fallen out of favor in most smaller companies" (footnote: SOAP is not required for SOA).

The problems with remote procedure calls (RPCs)

- Web services are "the latest incarnation of a long line of technologies" (EJB, RMI, DCOM, CORBA, each limited or complex). All rest on **remote procedure call (RPC)**, which makes a remote request look like a local function call: **location transparency**. "Although RPC seems convenient at first, the approach is fundamentally flawed", because a network request is very different from a local call:
- Unpredictable: requests or responses may be lost; "Network problems are common, so you have to anticipate them".
- A third outcome, a timeout: "you simply don't know what happened"; retrying may perform the action multiple times unless the protocol has deduplication (**idempotence**; Chapter 11).
- Latency is "wildly variable"; parameters must be encoded into bytes (problematic for larger objects); datatypes must be translated between languages.

- So “there’s no point trying to make a remote service look too much like a local object”. REST doesn’t hide being a network protocol.

Current directions for RPC

- RPC isn’t going away (Thrift, Avro, gRPC, Finagle, Rest.li). The new generation is more explicit that remote differs from local: futures (promises) encapsulate asynchronous actions that may fail and simplify parallel requests; gRPC supports streams; some provide service discovery (see “Request Routing”).
- Custom binary RPC can outperform JSON over REST, but REST suits debugging (browser, curl), is supported everywhere, and has a vast tool ecosystem. So REST is predominant for public APIs; RPC focuses on services of the same organization.

Data encoding and evolution for RPC

- Services must be changeable independently. Simplifying assumption versus databases: “all the servers will be updated first, and all the clients second”, so you need backward compatibility on requests and forward compatibility on responses. These properties are “inherited from whatever encoding it uses”: Thrift, gRPC and Avro RPC follow their encoding’s rules; SOAP’s XML schemas have “subtle pitfalls”; for REST, adding optional request parameters and new response fields is usually compatible.
- RPC often crosses organizational boundaries, so the provider cannot force clients to upgrade; compatibility may be needed “perhaps indefinitely”, and breaking changes mean multiple API versions side by side.
- “There is no agreement on how API versioning should work”: REST uses a version in the URL or Accept header, or a stored per-client version.

Message-Passing Dataflow

- Asynchronous message-passing is “somewhere between RPC and databases”: like RPC, a **message** is delivered with low latency; like databases, it goes via an intermediary, a **message broker** (message queue, message-oriented middleware), which stores it temporarily. Advantages over direct RPC: it buffers if the recipient is unavailable or overloaded; redelivers to a crashed process; spares the sender knowing the recipient’s IP address and port; allows one message to several recipients; and “logically decouples the sender from the recipient”.
- Unlike RPC, usually one-way. The pattern is **asynchronous**: the sender “simply sends it and then forgets about it.”

Message brokers

- A process sends to a named **queue** or **topic**; the broker delivers to one or more consumers or subscribers; many producers and consumers per topic.
- A topic is one-way, but a consumer may publish to another topic or a reply queue (request/response like RPC).
- A message is “just a sequence of bytes with some metadata”; with compatible encoding you can change publishers and consumers independently. A republishing consumer must preserve unknown fields (Figure 4-7).

Distributed actor frameworks

- The **actor model** is a concurrency model in a single process: logic in **actors** with private local state, communicating by asynchronous messages, delivery not guaranteed, one message at a time, so no threads, locking or deadlock to handle.
- Distributed actor frameworks scale it across nodes, with messages transparently encoded, sent and decoded. Location transparency works better than in RPC because actors already assume messages may be lost. It “essentially integrates a message broker and the actor programming model into a single framework”, yet rolling upgrades still require forward and backward compatibility. Akka defaults to Java serialization (no compatibility), Orleans to a format without rolling upgrades, and in Erlang OTP schema changes are “surprisingly hard”.

Summary

- Encodings affect efficiency and “more importantly also the architecture of applications and your options for deploying them.”

- **Rolling upgrades** encourage “frequent small releases over rare big releases” and make deployments less risky (faulty releases rolled back); “hugely beneficial for evolvability”.
- So we must assume nodes run different versions; all data must be backward compatible (new code reads old data) and forward compatible (old code reads new data). Recap of formats: language-specific encodings fail to provide compatibility; textual formats are vague about datatypes (numbers, binary strings); binary schema-driven formats give compact encoding and clear compatibility semantics, with the downside that “data needs to be decoded before it is human-readable”. Modes of dataflow: databases, RPC and REST (client encodes request, server decodes and encodes response), asynchronous message passing.
- Conclusion: “with a bit of care, backward/forward compatibility and rolling upgrades are quite achievable. May your application’s evolution be rapid and your deployments be frequent.”

Footnotes

- Seven short notes (e.g. encoding is not encryption; SOAP is not required for SOA).

References

- The author ends the chapter with references (not summarised).

How this chapter builds the author’s case

Chapter 1 set evolvability as a goal; Chapter 2 showed data models coping with schema change differently. Here he shows encodings (field tags in Thrift/Protocol Buffers, writer’s and reader’s schemas in Avro) are what make rolling upgrades and independent deployment possible, then follows them through databases, services and message passing. Cross-references: Chapters 7, 10, 11.

PART II

Distributed Data

- The author opens by turning the question from one machine to many: “Now, in Part II, we move up a level and ask: what happens if multiple machines are involved in storage and retrieval of data?”
- Reasons to distribute a database: scalability, fault tolerance/high availability, and latency.
- Scaling to higher load: the simplest approach is a more powerful machine (vertical scaling, shared-memory), but “the cost grows faster than linearly” and it is “limited to a single geographic location”; shared-disk is limited by “contention and the overhead of locking”.
- He focuses on shared-nothing architectures (horizontal scaling): “not because they are necessarily the best choice for every use case, but rather because they require the most caution from you, the application developer”, since “the database cannot magically hide these” constraints and trade-offs.
- Two ways data is distributed: replication, “Keeping a copy of the same data on several different nodes”, covered in Chapter 5, and partitioning, “Splitting a big database into smaller subsets called partitions”, covered in Chapter 6. They are “separate mechanisms, but they often go hand in hand”.
- Then “the difficult trade-offs”: transactions in Chapter 7, “as that will help you understand all the many things that can go wrong in a data system”, and “the fundamental limitations of distributed systems in Chapters 8 and 9”.

The theme of the Part

- What changes when data and work are spread over nodes: the developer must understand the constraints and trade-offs because the database cannot hide them.

How the chapters build the theme

- Chapters 5 and 6 give the two mechanisms (replication, partitioning). Chapter 7 gives transactions, to see what can go wrong and what can be done. Chapters 8 and 9 end the Part with the limits of distributed systems.

What the Part contributes to the book

- It carries the single-node foundations to a cluster and leads to Part III, which integrates several “(potentially distributed) datastores” into a larger system.

CHAPTER 5

Replication

Place in the argument — why this chapter, why this title

first chapter of Part II (distributed data). Title names the idea: “Replication means keeping a copy of the same data on multiple machines that are connected via a network.” Reasons are those of the Part II introduction. Scope: the dataset is “so small that each machine can hold a copy of the entire dataset”; Chapter 6 relaxes that.

Place in the argument — why this chapter, why this title

Order of sections: three algorithms from simplest to most permissive. Single-leader first, then its characteristic anomalies (replication lag), then multi-leader (no single write path, but write conflicts), then leaderless (no leader ordering at all, so the chapter ends with concurrency machinery). The Summary repeats this ladder.

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph (Douglas Adams): what “cannot possibly go wrong” is impossible to get at or repair when it does.
- Unchanging data is easy to replicate. “All of the difficulty in replication lies in handling changes to replicated data.” Three popular algorithms: **single-leader**, **multi-leader**, **leaderless**; “Almost all distributed databases use one of these three approaches.”
- Reasons to replicate: to keep data geographically close to your users (“reduce latency”); to keep working when parts have failed (“increase availability”); to “scale out the number of machines that can serve read queries”.
- Trade-offs: synchronous versus asynchronous, handling failed replicas; often configuration options, with similar principles across databases.
- The principles are old, since “the fundamental constraints of networks have remained the same”, but mainstream distributed databases are recent, hence “a lot of misunderstanding around issues such as eventual consistency”, made precise in “Problems with Replication Lag”.

Leaders and Followers

- Each node storing a copy is a **replica**. Every write must be processed by every replica, or they diverge. The most common solution is **leader-based replication** (active/passive, master-slave), Figure 5-1: one replica is the leader (master or primary) and clients must send writes to it, which first writes the data to its local storage. The others are followers (read replicas, slaves, secondaries, hot standbys); the leader sends each change as part of a “replication log or change stream”, and each follower applies all writes “in the same order”. Clients may read from the leader or any follower, but “writes are only accepted on the leader”.
- Built into many relational databases, some nonrelational ones, and message brokers such as Kafka.

Synchronous Versus Asynchronous Replication

- Figure 5-2: follower 1 synchronous (leader waits for its confirmation before reporting success and making the write visible), follower 2 asynchronous. Replication is normally under a second, but “there is no guarantee of how long it might take”; followers can lag minutes (recovering, near capacity, network problems).
- Synchronous: follower guaranteed up to date, but if it does not respond “the write cannot be processed” and the leader blocks. So all-synchronous is impractical; usually one follower is synchronous and the rest asynchronous, with promotion of an asynchronous one if it fails: **semi-synchronous**, guaranteeing an up-to-date copy on at least two nodes.
- Fully asynchronous: if the leader fails unrecoverably, unreplicated writes are lost, so “a write is not guaranteed to be durable, even if it has been confirmed to the client”. But the leader keeps processing writes even if all followers lag. Widely used despite weakened durability, especially with many or distant followers.

Research on Replication

- Researchers seek replication without data loss but with good performance; chain replication is a synchronous variant used in Microsoft Azure Storage. Replication consistency is tied to consensus (Chapter 9); this chapter covers the simpler common forms.

Setting Up New Followers

- To add replicas or replace failed ones. A plain file copy fails because data is in flux and "might not make any sense"; locking against writes defeats high availability. Setup steps: take a consistent snapshot of the leader, if possible without locking the whole database (as also required for backups); copy it to the new follower; the follower requests all changes since the snapshot, which needs the snapshot tied to an exact position in the replication log (PostgreSQL: log sequence number; MySQL: binlog coordinates); when it has processed the backlog, "it has caught up". It can usually be done without downtime; practical steps vary, from fully automated to an "arcane multi-step workflow".

Handling Node Outages

- Outages come from faults or planned maintenance; rebooting nodes without downtime is "a big advantage for operations and maintenance". Goal: keep the whole system running and the impact of an outage small.

Follower failure: Catch-up recovery

- A follower keeps a log of changes received, knows the last transaction processed, requests changes since from the leader, and catches up.

Leader failure: Failover

- Trickier: promote a follower, reconfigure clients, have other followers follow the new leader: **failover**, manual or automatic. Automatic steps: (1) determine the leader failed; there is no foolproof way, so most systems use a timeout (say 30 seconds without a response); (2) choose a new leader by election or a previously elected controller node; the best candidate is usually the replica with the most up-to-date changes, and agreeing on it "is a consensus problem" (Chapter 9); (3) reconfigure clients to write to the new leader, and ensure that an old leader that returns becomes a follower.
- "Failover is fraught with things that can go wrong". With asynchronous replication the new leader may lack the old leader's writes; the most common solution is to discard them, "which may violate clients' durability expectations". Especially dangerous with outside systems: in a GitHub incident a stale MySQL follower reused primary keys also used in Redis, so private data reached the wrong users. **Split brain**: two nodes both believe they are leader; if both accept writes without conflict resolution, "data is likely to be lost or corrupted". Timeout: longer means slower recovery; too short means unnecessary failovers, which make high load or network problems worse. There are "no easy solutions", so some teams prefer manual failover. These are "fundamental problems in distributed systems" (Chapters 8 and 9).

Implementation of Replication Logs

- Several methods are used; each is looked at briefly.

Statement-based replication

- Leader logs every statement (INSERT, UPDATE, DELETE) and followers execute it. It breaks down: nondeterministic functions such as NOW() or RAND() give different values on each replica; autoincrementing columns or statements depending on existing data (UPDATE … WHERE) must run in exactly the same order on each replica, limiting concurrent transactions; side effects (triggers, stored procedures, user-defined functions) may differ unless deterministic. Nondeterministic calls can be fixed by the leader, but "because there are so many edge cases, other replication methods are now generally preferred". VoltDB makes it safe by requiring deterministic transactions.

Write-ahead log (WAL) shipping

- The storage log (Chapter 3) is "an append-only sequence of bytes containing all writes"; the leader also sends it to followers, who build identical data structures (PostgreSQL, Oracle). In a log-structured engine the log is the main place for storage; in a B-tree every modification is first written to a write-ahead log so the index can be restored after a crash.
- Disadvantage: the log is low level (bytes changed in disk blocks), coupling replication to the storage engine, so versions typically cannot differ between leader and followers. Operational impact: a

newer-version follower allows zero-downtime upgrade (upgrade followers, then fail over); with WAL shipping upgrades require downtime.

Logical (row-based) log replication

- A **logical log** decoupled from the storage engine's (**physical**) representation: row-granularity records: an inserted row logs the new values of all columns; a deleted row logs enough to identify it (typically the primary key, else the old values of all columns); an updated row logs the identity plus the new values of changed columns. A transaction adds a commit record (MySQL binlog). More easily backward compatible (different versions or engines) and easier for external applications to parse, e.g., warehouses, custom indexes and caches: **change data capture** (Chapter 11).

Trigger-based replication

- Application-layer replication when more flexibility is needed: a subset of data, a different kind of database, or conflict resolution (see "Handling Write Conflicts"). Tools read the database log, or **triggers and stored procedures** run custom code on a change and log it to a table for an external process. Greater overheads and more bugs than built-in replication, but flexible.

Problems with Replication Lag

- Writes go through the leader but reads can use any replica; for mostly-read workloads this gives the read-scaling architecture. It "only realistically works with asynchronous replication", since synchronous replication to many followers is "very unreliable".
- Cost: asynchronous followers may be stale. This is temporary: they "eventually catch up", hence **eventual consistency**. "Eventually" is "deliberately vague"; there is no limit to how far a replica can fall behind. The **replication lag** may be a fraction of a second or minutes near capacity or with network trouble; then it is "a real problem for applications". Three examples follow.

Reading Your Own Writes

- Users submit data then view it; writes go to the leader, reads may use a follower. Figure 5-3: a read shortly after a write hits a stale replica and the data looks "lost".
- Needed: **read-after-write consistency** (read-your-writes): "if the user reloads the page, they will always see any updates they submitted themselves", with no promise about other users. Techniques: read from the leader what the user may have modified, otherwise from a follower; if most things are editable this negates read scaling, so instead read from the leader for one minute after the last update, or block followers more than one minute behind; or the client remembers the timestamp of its most recent write and the serving replica must reflect updates at least until then (logical timestamp or system clock). With several devices, **cross-device read-after-write consistency** adds issues: timestamp metadata must be centralized, and with replicas in several datacenters, requests that must reach the leader must be routed to its datacenter, even from devices on different networks.

Monotonic Reads

- Users may see things "moving backward in time" when reading different replicas. Figure 5-4: user 2345 reads from a low-lag follower, sees user 1234's comment, then reads a lagging follower and it disappears.
- **Monotonic reads**: "they will not read older data after having previously read newer data". Weaker than strong consistency, stronger than eventual. One way: each user always reads from the same replica (chosen by hash of user ID), rerouting if it fails.

Consistent Prefix Reads

- Causality violation: Mr. Poons asks, Mrs. Cake answers; an observer reading via followers with different lag sees the answer first (Figure 5-5), "very confusing".
- **Consistent prefix reads**: writes that happen in a certain order are seen in that order. A particular problem for partitioned databases (Chapter 6), where there is no global ordering. Solutions: write causally related data to the same partition (not always efficient), or algorithms that track causal dependencies ("The "happens-before" relationship and concurrency").

Solutions for Replication Lag

- Ask how the application behaves if lag reaches minutes or hours; if that is a bad experience, provide a stronger guarantee. “Pretending that replication is synchronous when in fact it is asynchronous is a recipe for problems down the line.” Application-level fixes are “complex and easy to get wrong”.
- Hence transactions: stronger guarantees so the application is simpler. Many distributed databases abandoned them, claiming eventual consistency is inevitable; “There is some truth in that statement, but it is overly simplistic”. Developed in Chapters 7 and 9 and Part III.

Multi-Leader Replication

- Single-leader downside: “there is only one leader, and all writes must go through it”. Natural extension: more than one node accepts writes, each forwarding changes to all others: **multi-leader** (master-master, active/active); each leader is also a follower of the others.

Use Cases for Multi-Leader Replication

- Within one datacenter the benefits rarely outweigh the complexity.

Multi-datacenter operation

- A leader per datacenter (Figure 5-6), ordinary leader-follower inside, leaders replicate to each other. Versus single-leader: (1) performance, writes are local and the inter-datacenter delay is hidden; (2) outages, each datacenter continues independently; (3) network problems, asynchronous replication tolerates them better, whereas single-leader writes are synchronous over the link.
- Big downside: concurrent modification in two datacenters, so “those write conflicts must be resolved”. A “somewhat retrofitted feature”, with pitfalls (autoincrementing keys, triggers, integrity constraints): “dangerous territory that should be avoided if possible”.

Clients with offline operation

- Calendar apps must work offline: each device’s database is a leader with asynchronous sync, lag possibly days; multi-datacenter replication “taken to the extreme”, and tricky (CouchDB is designed for it).

Collaborative editing

- Etherpad, Google Docs: edits apply locally and replicate asynchronously. Locking the document avoids conflicts, equivalent to single-leader with transactions. For faster collaboration, make changes tiny (a keystroke) and skip locking, which brings all the challenges of multi-leader, including conflict resolution.

Handling Write Conflicts

- “The biggest problem with multi-leader replication is that write conflicts can occur”. Figure 5-7: a wiki title changed A to B by user 1 and A to C by user 2; both succeed locally, and the conflict is detected on replication. Single-leader does not have this problem.

Synchronous versus asynchronous conflict detection

- Single-leader: the second writer blocks or aborts. Multi-leader: both succeed, detection is late, “it may be too late to ask the user”. Synchronous detection loses the point of multi-leader: “you might as well just use single-leader replication”.

Conflict avoidance

- Simplest: route all writes for a record through the same leader (e.g., a user’s “home” datacenter); “a frequently recommended approach”. It breaks down when the designated leader must change (datacenter failure, user moved).

Converging toward a consistent state

- Single-leader: the last write determines the value. Multi-leader has no defined ordering: leader 1 sees B then C, leader 2 C then B; neither is “more correct”, and applying writes as seen leaves C and B, “not acceptable”. The database must resolve conflicts in a **convergent** way: all replicas reach the same final value. Ways: give each write a unique ID and pick the highest as the winner, discarding the others; with a timestamp this is **last write wins** (LWW), “dangerously prone to data loss”; give each replica a unique ID and let higher-numbered replicas win, which also implies data loss; merge the

values (e.g., “B/C”); or record the conflict in an explicit data structure and resolve it later in application code, perhaps by prompting the user.

Custom conflict resolution logic

- Resolution depends on the application, so tools accept application code, run on write (a handler called when a conflict is detected in the log; it runs in the background and must be quick, e.g., Bucardo Perl) or on read (conflicting versions stored, returned on the next read for the application or user to resolve; CouchDB).
- Resolution applies per row or document, not per transaction.

Automatic Conflict Resolution

- Custom code is error-prone: Amazon’s cart kept added but not removed items, so removed items reappeared. Research into automatic resolution: **conflict-free replicated datatypes** (CRDTs), structures for sets, maps, ordered lists, counters that can be edited concurrently and “automatically resolve conflicts in sensible ways” (some in Riak 2.0); mergeable persistent data structures, which track history like Git and use a three-way merge; and operational transformation, the algorithm behind Etherpad, designed for concurrent editing of an ordered list of characters. This work is “still young” but could make multi-leader much simpler.

What is a conflict?

- Obvious: the same field set to two values (Figure 5-7). Subtle: room booking, where overlapping bookings made on two leaders conflict even though availability was checked. “There isn’t a quick ready-made answer”; see Chapter 7 for more examples and Chapter 12 for scalable approaches.

Multi-Leader Replication Topologies

- A **replication topology** is the paths along which writes propagate (Figure 5-8). All-to-all is the most general; MySQL supports only circular by default; star has one root forwarding to all (generalizes to a tree).
- In circular and star topologies, writes are tagged with the identifiers of nodes passed, and a node ignores a change bearing its own identifier, preventing infinite loops. One failed node can interrupt the flow, with manual reconfiguration; denser topologies avoid a single point of failure.
- All-to-all: messages “overtake” others (Figure 5-9): leader 2 may receive an update before the insert it depends on. A causality problem like “Consistent Prefix Reads”; timestamps do not suffice because “clocks cannot be trusted to be sufficiently in sync” (Chapter 8). Version vectors can order events (“Detecting Concurrent Writes”), but implementations are poor: BDR lacks causal ordering, and Tungsten “doesn’t even try to detect conflicts”. Read the documentation and test.

Leaderless Replication

- Here any replica accepts writes directly; no leader orders writes. Early systems were leaderless; the idea returned after Amazon’s Dynamo. Riak, Cassandra, Voldemort are **Dynamo-style**. A client or a coordinator sends writes to several replicas, but the coordinator “does not enforce a particular ordering of writes”, with “profound consequences”.

Writing to the Database When a Node Is Down

- Three replicas, one down: leaderless has no failover. Figure 5-10: the client writes to all three in parallel; two ok responses count as success and the miss is ignored. A returning node is stale, so reads go to several nodes in parallel, and version numbers pick the newer value.

Read repair and anti-entropy

- Two mechanisms. **Read repair**: a client reading sees a stale response (version 6 on replica 3, 7 elsewhere) and writes the newer value back; good for frequently read values. **Anti-entropy**: a background process copying missing data, in no particular order, possibly with significant delay. Without it (Voldemort) rarely read values have reduced durability.

Quorums for reading and writing

- With n replicas, writes need w confirmations and reads query r nodes (n = 3, w = 2, r = 2). as long as w + r > n, “we expect to get an up-to-date value when reading”, since the read set overlaps the write set: **quorum** reads and writes (Figure 5-11). Common: n odd, w = r = (n + 1) / 2 rounded up. Few writes and many reads may use w = n and r = 1: fast reads but one failed node blocks writes.

- More than n nodes may exist in the cluster, but a value lives on only n (partitioning, Chapter 6). Below w or r available nodes, operations error; the cause of unavailability does not matter.

Limitations of Quorum Consistency

- With w + r > n reads generally return the latest value. Quorums "are not necessarily majorities"; only overlap matters. With w + r ≤ n reads are likelier stale but latency is lower and availability higher.
- With w < n writes still work when a node is unavailable; with r < n so do reads; n = 3, w = 2, r = 2 tolerates one unavailable node, n = 5, w = 3, r = 3 tolerates two. Requests go to all n replicas in parallel; w and r say how many must report success.
- Even with w + r > n there are edge cases: with a sloppy quorum the w writes may land on different nodes than the r reads; concurrent writes have no clear order, so the only safe solution is merging, and LWW can lose writes "due to clock skew"; a write concurrent with a read may be reflected on only some replicas; a write that succeeded on fewer than w replicas is not rolled back, so a reported failure may still be returned by later reads; a node with a new value restored from an old replica may drop the count below w; and timing edge cases. Dynamo-style databases suit eventual consistency; "it's wise to not take them as absolute guarantees". You usually do not get read-your-writes, monotonic or consistent prefix reads; "Stronger guarantees generally require transactions or consensus".

Monitoring staleness

- Leader-based lag is measured by subtracting log positions. Leaderless has no fixed order, and with only read repair a value may be "ancient". "Eventual consistency is a deliberately vague guarantee, but for operability it's important to be able to quantify "eventual.""

Sloppy Quorums and Hinted Handoff

- Quorums tolerate failed or slow nodes without failover, suiting high availability and low latency. But a network interruption can leave a client unable to reach a quorum, though nodes are alive: "they might as well be dead". In a large cluster the designers face a trade-off: return errors for all requests that cannot reach a quorum of w or r nodes, or accept writes anyway on reachable nodes that aren't among the n usual home nodes. accepting writes on reachable nodes outside the n "home" nodes is a **sloppy quorum** (like sleeping on the neighbor's couch when locked out).
- When the interruption heals, the stand-in nodes send writes to the home nodes: **hinted handoff**. This raises write availability, but even with w + r > n the latest value may be outside n. "a sloppy quorum actually isn't a quorum at all in the traditional sense": only durability, no read guarantee until handoff completes.

Multi-datacenter operation

- Leaderless tolerates conflicts, interruptions, latency spikes. Cassandra and Voldemort: n spans datacenters, the client waits for a local quorum, remote writes often asynchronous. Riak: n is local, cross-datacenter replication asynchronous like multi-leader.

Detecting Concurrent Writes

- Concurrent writes conflict even with strict quorums (and during read repair or hinted handoff). Events arrive in different orders at different nodes; Figure 5-12: node 1 misses B's write, node 2 gets A then B, node 3 gets B then A. Overwriting on receipt would leave overwriting on receipt leaves nodes "permanently inconsistent" (node 2 has B, others A). Replicas must converge; most implementations are poor, so "you—the application developer—need to know a lot about the internals".

Last write wins (discarding concurrent writes)

- Keep only the most "recent" value. That is "actually quite misleading": neither client knew of the other, so the writes are **concurrent** and unordered. Forcing a timestamp order is **last write wins** (LWW), the only method in Cassandra, optional in Riak.
- It converges "at the cost of durability": successful concurrent writes are "silently discarded", even non-concurrent ones ("Timestamps for ordering events"). Perhaps fine for caching; otherwise "a poor choice". Safe only with immutable write-once keys (Cassandra UUID keys).

The "happens-before" relationship and concurrency

- "An operation A happens before another operation B if B knows about A, or depends on A, or builds upon A in some way." Two operations are concurrent "if neither happens before the other (i.e., neither knows about the other)". Examples: in Figure 5-9 the insert by A happens before B's increment because B builds on A's value, so B is "causally dependent on A"; in Figure 5-12 each client starts without knowing of the other's operation, so there is no causal dependency. Three possibilities: A before B, B before A, concurrent. Later overwrites earlier; concurrent means conflict.

Concurrency, Time, and Relativity

- Concurrency is not "at the same time"; clocks make that hard to tell (Chapter 8). Operations are concurrent "if they are both unaware of each other, regardless of the physical time". Like special relativity, but network problems can make operations concurrent even when light would have allowed influence.

Capturing the happens-before relationship

- Start with one replica. Figure 5-13: two clients concurrently add to a shopping cart, five writes; Figure 5-14 graphs which operation depended on which. Clients are never fully up to date, but old versions are overwritten and "no writes are lost".
- The server decides concurrency from version numbers, without interpreting the value The cart run: milk gets version 1; eggs (unaware) version 2, stored beside milk; [milk, flour] sent with version 1 supersedes [milk] but is concurrent with [eggs] (version 3); [eggs, milk, ham] sent with version 2 gets version 4; the final merge with bacon is sent with version 3. Algorithm: the server keeps a version number per key, increments it on every write; a read returns all values not overwritten plus the latest version, and a client must read before writing; a write includes the prior version and merges all values received; the server may overwrite all values with that version or below but "must keep all values with a higher version number". A write carrying a prior read's version says what it is based on; a write without one "is concurrent with all other writes" and becomes a sibling on later reads.

Merging concurrently written values

- No data is silently dropped, but clients must merge concurrent values, which Riak calls **siblings**; this is the multi-leader conflict problem. LWW loses data. For the cart, take the union: [milk, flour, eggs, bacon] and [eggs, milk, ham] give [milk, flour, eggs, bacon, ham].
- With removals the union resurrects removed items, so deletion leaves a versioned marker, a **tombstone**. Merging in application code is error-prone; Riak's CRDTs merge siblings automatically, "including preserving deletions".

Version vectors

- With multiple replicas, use "a version number per replica as well as per key". Each replica increments its own number on writes and tracks others' numbers. The collection is a **version vector** (dotted version vector in Riak 2.0). Sent to clients on read and back on write (Riak: **causal context**), it separates overwrites from concurrent writes, and makes it safe to read from one replica and write to another; siblings may result, but "no data is lost as long as siblings are merged correctly".

Version vectors and vector clocks

- Version vector is sometimes called **vector clock**, not quite the same; "when comparing the state of replicas, version vectors are the right data structure to use".

Summary

- Replication serves: keeping the system running through machine or datacenter loss; working through network interruptions; data near users; higher read volume. Simple goal, yet "a remarkably tricky problem", requiring care about concurrency and faults.
- Three approaches: single-leader (reads on any replica may be stale), multi-leader (several leaders stream changes to each other), leaderless (write to several, read from several to correct stale nodes). Single-leader is popular as "fairly easy to understand" with no conflicts; the others are more robust to faults, interruptions and latency spikes "at the cost of being harder to reason about and providing only very weak consistency guarantees".
- Synchronous versus asynchronous matters under faults: promoting an asynchronously updated follower can lose committed data. Consistency models under lag: users see their own data (read-

after-write), do not see time go backward (monotonic reads), see causally sensible order (consistent prefix reads).

- Multi-leader and leaderless raise concurrency conflicts; an algorithm decides happened-before versus concurrent, and concurrent updates can be merged. Next: splitting a large dataset into partitions.

Footnotes

- Notes on standby terms, fencing (STONITH), eventual consistency, DynamoDB versus Dynamo, and **strict quorum**.

References

- The chapter ends with references (not summarised).

How this chapter builds the author's case

The chapter builds its case as a ladder: each step removes a limit (one write path, one ordering) and pays in weaker guarantees. Back: Part II introduction, Chapter 3 (log, "Hash Indexes" tombstones). Forward: Chapter 6 (partitions), 7 (transactions), 8 (clocks), 9 (consensus), 11 (change data capture), 12 (conflict detection), Part III.

CHAPTER 6

Partitioning

Place in the argument — why this chapter, why this title

second chapter of Part II. Chapter 5 gave replication, "having multiple copies of the same data on different nodes"; for "very large datasets, or very high query throughput, that is not sufficient". The title names the remedy: break the data into partitions, also known as sharding. The sections run from how records are assigned (key-value data), to the complication of secondary indexes, to moving partitions (rebalancing), to finding them (request routing).

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph (Grace Murray Hopper): "We must state relationships, not procedures."
- Partitioning is the counterpart to replication, introduced as breaking data into partitions.

Terminological confusion

- What is called a partition here is a shard in MongoDB, Elasticsearch and SolrCloud, a region in HBase, a tablet in Bigtable, a vnode in Cassandra and Riak, a vBucket in Couchbase; "partitioning is the most established term, so we'll stick with that."
- Each record belongs to exactly one partition; "each partition is a small database of its own", though operations may touch several.
- Main reason: **scalability**. Partitions sit on different nodes of a shared-nothing cluster, spreading data over disks and load over processors. Single-partition queries scale by adding nodes; complex queries can be parallelized, "although this gets significantly harder".
- Pioneered in the 1980s, rediscovered by NoSQL and Hadoop; the fundamentals apply to transactional and analytic workloads alike.

Partitioning and Replication

- Usually combined: each record is in one partition but stored on several nodes for fault tolerance. With leader-follower replication (Figure 6-1) each node leads some partitions and follows others.
- Chapter 5 applies equally to partitions; the partitioning scheme is "mostly independent" of the replication scheme, so replication is ignored here.

Partitioning of Key-Value Data

- Goal: spread data and query load evenly; with a fair share, 10 nodes handle 10 times the data and throughput. Unfair partitioning is **skewed**; a partition with disproportionately high load is a **hot spot**.
- Random assignment avoids hot spots but then "you have to query all nodes in parallel". Assume a key-value model accessed by primary key, like an alphabetically sorted paper encyclopedia.

Partitioning by Key Range

- Assign a continuous range of keys to each partition, like encyclopedia volumes (Figure 6-2); knowing the boundaries (and partition-to-node assignment) lets you go directly to the node. Ranges need not be evenly spaced, since data is not (volume 1 holds A and B, volume 12 holds T to Z); boundaries must adapt to the data, chosen manually or automatically (see "Rebalancing Partitions").
- Keys sorted within a partition make range scans easy (sensor data keyed by year-month-day-hour-minute-second; fetch one month).
- Downside: hot spots. With timestamp keys all writes hit today's partition. Fix: prefix with the sensor name, which spreads writes but requires a separate range query per sensor.

Partitioning by Hash of Key

- Because of skew risk, many datastores hash the key. A good hash function takes skewed data and makes it uniformly distributed (e.g., 32-bit, 0 to 2^32 − 1). It need not be cryptographically strong; built-in language hashes may be unsuitable, as the same key may hash differently in different processes.
- Each partition gets a range of hashes (Figure 6-3). This distributes keys fairly; boundaries can be evenly spaced or pseudorandom (sometimes called consistent hashing).

Consistent Hashing

- Defined by Karger et al. for caches such as a CDN: random boundaries avoid central control or consensus. “consistent” has nothing to do with replica consistency or ACID consistency. It “doesn’t work very well for databases”, so rarely used; “it’s best to avoid the term consistent hashing and just call it hash partitioning instead.”
- Cost of hashing: efficient range queries are lost, since adjacent keys are scattered. MongoDB in hash mode sends range queries to all partitions.
- Cassandra’s compromise: a compound primary key whose first column is hashed and the other columns serve as a concatenated index for sorting. No range over the first column, but a fixed first column allows an efficient range scan. This gives an elegant one-to-many model, e.g., (user_id, update_timestamp): one user’s updates in a time interval, sorted, on a single partition.

Skewed Workloads and Relieving Hot Spots

- Hashing cannot avoid hot spots when all requests are for one key, e.g., a celebrity user causing “a storm of activity”; identical IDs hash the same.
- Most systems cannot compensate automatically, so “it’s the responsibility of the application to reduce the skew”: add a random number (two digits give 100 keys) to a hot key. Cost: reads must read all 100 keys and combine, and you must track which keys are split, since it makes sense only for the few hot keys. “you need to think through the trade-offs for your own application”.

Partitioning and Secondary Indexes

- So far it was key-value with primary-key access. A secondary index “usually doesn’t identify a record uniquely but rather is a way of searching for occurrences of a particular value” (all actions by user 123, articles containing hogwash, red cars). Bread and butter of relational databases, common in document databases, avoided by HBase and Voldemort for complexity, added by Riak, and “the raison d’être” of Solr and Elasticsearch.
- They “don’t map neatly to partitions”. Two main approaches: document-based and term-based.

Partitioning Secondary Indexes by Document

- Used-car site (Figure 6-4): partition by document ID (0 to 499 in partition 0, etc.), secondary indexes on color and make, updated automatically, e.g., color:red.
- Each partition keeps its own index covering only its documents, so a write touches only one partition: a **local index** (versus global).
- Reads need care: red cars appear in both partitions, so the query goes to all partitions and results are combined: **scatter/gather**, expensive and prone to tail latency amplification. Still widely used. Vendors advise structuring partitioning so queries hit one partition, not always possible, especially with multiple secondary indexes in one query.

Partitioning Secondary Indexes by Term

- A **global index** covers all partitions but cannot sit on one node (bottleneck), so it is itself partitioned, differently from the primary key (Figure 6-5): colors a to r in partition 0, s to z in partition 1. **Term-partitioned**: the term (e.g., color:red, from full-text indexes) determines the partition. Partition by the term for range scans (e.g., asking price) or by a hash for even load.
- Advantage: efficient reads, a request goes only to the partition holding the term. Downside: writes are slower and more complicated, since one document may affect several index partitions.
- An always-current index would need a distributed transaction (Chapters 7 and 9), not supported everywhere, so updates are often asynchronous: a read just after a write may miss it. DynamoDB: normally within a fraction of a second, longer in faults. Implementation returns in Chapter 12.

Rebalancing Partitions

- Things change: query throughput rises (add CPUs), dataset size rises (add disks and RAM), a machine fails (others take over). Moving load between nodes is **rebalancing**. Minimum requirements: afterwards load is “shared fairly”; the database keeps accepting reads and writes meanwhile; “No more data than necessary should be moved”.

Strategies for Rebalancing

- There are a few ways of assigning partitions to nodes.

How not to do it: hash mod N

- Why not hash(key) mod N? “if the number of nodes N changes, most of the keys will need to be moved”. Example: hash 123456 is on node 6 with 10 nodes, node 3 with 11, node 0 with 12. “We need an approach that doesn’t move data around more than necessary.”

Fixed number of partitions

- Create many more partitions than nodes, e.g., 1,000 partitions on 10 nodes. A new node steals a few from every node (Figure 6-6); removal reverses it. Only whole partitions move; the key-to-partition assignment is unchanged. Transfer takes time, so the old assignment serves meanwhile. More partitions can go to more powerful nodes.
- The number is usually fixed at setup (splitting is operationally harder), so it is the maximum number of nodes and must be set high enough for growth, but each partition has overhead. Hard if dataset size varies: partitions grow with the data; too large makes rebalancing and recovery expensive, too small adds overhead; best when “just right”.

Dynamic partitioning

- Fixed boundaries with key range partitioning could leave all data in one partition, and manual reconfiguration is tedious. So HBase and RethinkDB split a partition exceeding a configured size (HBase default 10 GB) into two, and merge a shrunken one with a neighbor, like a B-tree top level. Halves can move to other nodes (HBase via HDFS).
- Advantage: the number of partitions adapts to data volume. Caveat: an empty database starts with one partition, so one node handles all writes; **pre-splitting** (HBase, MongoDB) needs knowing the key distribution. Also works for hash partitioning (MongoDB since 2.4 for both).

Partitioning proportionally to nodes

- Dynamic: partitions proportional to dataset size; fixed: partition size proportional to dataset; both independent of node count. Third option (Cassandra, Ketama): a fixed number of partitions per node. Partition size grows with data while nodes stay fixed, shrinks when nodes are added, so stays fairly stable.
- A new node randomly picks existing partitions to split and takes half of each. Random splits can be unfair but average out (Cassandra: 256 per node by default); Requires hash partitioning; closest to the original consistent hashing, and newer hash functions achieve a similar effect with less metadata.

Operations: Automatic or Manual Rebalancing

- A gradient from fully automatic to fully manual; Couchbase, Riak and Voldemort suggest an assignment that an administrator must commit.
- Automatic is convenient but “unpredictable”: rebalancing is expensive and can overload the network or nodes. Combined with automatic failure detection it is dangerous: an overloaded slow node is judged dead, load is moved away, making things worse, “potentially causing a cascading failure”. So “it can be a good thing to have a human in the loop”.

Request Routing

- Open question: after rebalancing, how does a client know which node to connect to for key “foo”? It is **service discovery**, not limited to databases;
- Approaches (Figure 6-7): (1) clients contact any node (e.g., round-robin load balancer), which handles the request or forwards it to the right node; (2) a routing tier, a “partition-aware load balancer”, handles no requests itself; (3) clients are partition-aware and connect directly. The key problem: how does the component making the routing decision (node, routing tier or client) learn about assignment changes? All participants must agree, which is a consensus problem, hard to implement (Chapter 9).
- Many systems use a coordination service such as ZooKeeper (Figure 6-8): nodes register, ZooKeeper holds the authoritative mapping, and the routing tier subscribes and is notified of changes. HBase, SolrCloud, Kafka use it; MongoDB has its own config server.
- Cassandra and Riak use a gossip protocol: any node can take a request and forward it (approach 1 in Figure 6-7), more complexity in nodes but no external dependency. Couchbase does not rebalance automatically, using a routing tier called moxi. Client IP addresses change slowly, so DNS often suffices.

Parallel Query Execution

- So far simple single-key reads/writes (and scatter/gather), the level of most NoSQL datastores. MPP relational products for analytics support joins, filtering, grouping, aggregation: the optimizer breaks the query into stages and partitions run in parallel on nodes, benefiting large scans. See Chapter 10.

Summary

- Partitioning is necessary when storing and processing on one machine is no longer feasible. The goal: spread data and query load evenly and avoid hot spots, by choosing a suitable scheme and rebalancing as nodes change.
- Two approaches. **Key range partitioning**: sorted keys, efficient range queries, but hot-spot risk when close keys are accessed often; partitions usually rebalanced dynamically by splitting. **Hash partitioning**: "destroys the ordering of keys, making range queries inefficient, but may distribute load more evenly"; commonly a fixed number of partitions assigned several per node, moved whole when nodes change; dynamic partitioning also possible. Hybrids are possible with a compound key.
- Secondary indexes also need partitioning. **Document-partitioned** (local) indexes live with the primary key and value: one partition updated on write, but a read needs scatter/gather across all partitions. **Term-partitioned** (global) indexes are partitioned separately by indexed value: a write updates several index partitions, but a read can be served from a single partition. Routing ranges from partition-aware load balancing to parallel query engines.
- "By design, every partition operates mostly independently", which allows scale; but writes to several partitions are hard to reason about (what if one succeeds and another fails?), to be addressed in the following chapters.

Footnotes

- Notes: partitioning is unrelated to network partitions (Chapter 8); a hand-made key-value secondary index risks inconsistency (see "The need for multi-object transactions").

References

- The chapter ends with references (not summarised).

How this chapter builds the author's case

This chapter builds on Chapter 5: replication copies, partitioning splits, and the two combine, with Chapter 5's lessons applying per partition. Back-links: Part II introduction (shared nothing), Chapter 5 (consistent prefix reads across partitions, consistency terms), Chapter 3 (SSTables, B-trees, indexes). Forward-links: Chapter 7 (multi-partition writes and transactions), Chapter 8 (network partitions), Chapter 9 (consensus for routing), Chapter 10 (parallel queries), Chapter 12 (term-partitioned indexes).

CHAPTER 7

Transactions

Place in the argument — why this chapter, why this title

The title names the mechanism that "for decades" has simplified fault handling: "a way for an application to group several reads and writes together into a logical unit". The author asks what safety guarantees a database gives and at what cost. It covers single-node and distributed databases; challenges "that arise only in distributed systems" go to Chapter 8 (distributed transactions: Chapter 9).

Place in the argument — why this chapter, why this title

The order descends from claim to mechanism: the slippery concept (ACID, multi-object operations); weak isolation levels in rising strength, each exposing an anomaly the last allows; then serializability and its three implementations, answering "why isn't everyone using it?"; then the Summary.

Chapter opening — untitled section before the first sub-heading in the book

- Faults: the database or application may fail "at any time", networks are interrupted, clients overwrite each other, a client may read data "only partially been updated", and "race conditions between clients can cause surprising bugs". "Implementing fault-tolerance mechanisms is a lot of work": careful thinking and a lot of testing.
- A **transaction** groups reads and writes into a logical unit: it either succeeds (**commit**) or fails (**abort**, **rollback**), and "the application can safely retry". The application "doesn't need to worry about partial failure".
- "Transactions are not a law of nature; they were created with a purpose, namely to simplify the programming model". The database handles certain errors and concurrency issues: "safety guarantees".
- Not every application needs them; "some safety properties can be achieved without transactions". To decide, understand what they guarantee and cost; the chapter goes "especially deep in the area of concurrency control".

The Slippery Concept of a Transaction

- Most relational databases, and some nonrelational ones, follow the style of IBM System R (1975); for 40 years the idea is virtually the same, "uncannily similar".
- In the late 2000s NoSQL databases offered new data models and replication and partitioning by default; "Transactions were the main casualty", abandoned or redefined as "a much weaker set of guarantees".
- Hype says transactions are "the antithesis of scalability"; vendors say they are essential. "Both viewpoints are pure hyperbole."
- "Like every other technical design choice, transactions have advantages and limitations", so the guarantees are examined in normal and extreme but realistic circumstances.

The Meaning of ACID

- **ACID**: Atomicity, Consistency, Isolation, Durability, coined in 1983 by Härder and Reuter to give precise terminology for fault-tolerance mechanisms.
- In practice "one database's implementation of ACID does not equal another's"; "The high-level idea is sound, but the devil is in the details." "ACID has unfortunately become mostly a marketing term."

Atomicity

- Atomic means something that cannot be broken into smaller parts. In ACID, "atomicity is not about concurrency"; that is the I.
- ACID atomicity covers a client making several writes when a fault (crash, network interruption, disk full, constraint violation) hits partway: the transaction is aborted and the database must discard or undo its writes.

- Otherwise it is hard to know which changes took effect, and retrying risks duplicate or incorrect data. After an abort the application "can be sure that it didn't change anything, so it can safely be retried".
- "The ability to abort a transaction on error and have all writes from that transaction discarded is the defining feature of ACID atomicity"; "abortability" might be a better term.

Consistency

- "The word consistency is terribly overloaded", with at least four meanings: replica consistency and eventual consistency (Chapter 5), consistent hashing (partitioning), the CAP theorem's linearizability, and in ACID "an application-specific notion of the database being in a "good state.""
- ACID consistency: **invariants** (e.g. credits and debits balance) must always be true; a transaction starting from a valid database and preserving validity keeps them satisfied.
- That depends on the application's invariants; "it's the application's responsibility to define its transactions correctly". "If you write bad data that violates your invariants, the database can't stop you."
- Atomicity, isolation and durability are database properties; consistency is the application's. Thus "the letter C doesn't really belong in ACID."

Isolation

- Clients accessing the same records cause race conditions. Figure 7-1: two clients each read a counter, add 1 and write back; 42 should become 44 but reaches 43.

Figure 7-1. A race condition between two clients concurrently incrementing a counter.

- Isolation means concurrent transactions "cannot step on each other's toes". Textbooks formalize it as **serializability**: each transaction can pretend to be the only one, and the result is as if they ran serially.
- Serializable is rarely used in practice (performance penalty). Oracle 11g's "serializable" is really snapshot isolation, "a weaker guarantee than serializability"; weaker forms come in "Weak Isolation Levels".

Durability

- "Durability is the promise that once a transaction has committed successfully, any data it has written will not be forgotten, even if there is a hardware fault or the database crashes."
- Single node: written to nonvolatile storage, usually with a write-ahead log. Replicated: copied to some number of nodes. The database must wait for these before reporting commit. Per "Reliability", "perfect durability does not exist".
- Each technique falls short: a disk is "inaccessible until you either fix the machine"; a correlated fault can knock out all replicas, losing in-memory data; async replication can lose recent writes; SSDs may violate guarantees ("even fsync isn't guaranteed to work correctly"); firmware bugs and undetected corruption occur, and may reach replicas and backups.

Replication and Durability

- "There are only various risk-reduction techniques, including writing to disk, replicating to remote machines, and backups", to be used together.

Single-Object and Multi-Object Operations

- Atomicity (all-or-nothing, no partial failure) and isolation (others see all or none of a transaction's writes) assume **multi-object transactions**, needed when pieces of data must stay in sync.
- Email example: a counted `unread_flag` query may be too slow, so an unread counter is stored (denormalization) and must be updated with each message. Figure 7-2: user 2 sees an unread message but a counter of zero because the increment has not yet happened; isolation gives both or neither.

Figure 7-2. Violating isolation: one transaction reads another transaction's uncommitted writes (a "dirty read").

Figure 7-3. Atomicity ensures that if an error occurs any prior writes from that transaction are undone, to avoid an inconsistent state.

- Figure 7-3: if the counter update fails, mailbox and counter diverge; atomically, the transaction aborts and the email is rolled back.
- Relational databases usually group operations by the client's TCP connection (BEGIN TRANSACTION to COMMIT).
- Many nonrelational databases lack grouping; a multi-put "doesn't necessarily mean it has transaction semantics" and may leave a partial update.

Single-object writes

- Atomicity and isolation matter for one object too (a 20 KB JSON document): if the network drops after 10 KB, is the fragment stored? If power fails mid-overwrite, are old and new values spliced? Can a reader see a partial value? Engines "almost universally" provide them per object: a crash-recovery log, a lock per object.
- Atomic increment removes the read-modify-write cycle of Figure 7-1; compare-and-set writes only if the value is unchanged. They prevent lost updates.
- But they are not transactions: "lightweight transactions" or "ACID" is "misleading". A transaction groups "multiple operations on multiple objects into one unit of execution".

The need for multi-object transactions

- Distributed stores dropped them because they are hard across partitions and get in the way of very high availability or performance, but "there is nothing that fundamentally prevents transactions in a distributed database" (Chapter 9).
- Some uses need only single-object operations; in many others writes must be coordinated: foreign keys must stay valid, denormalized documents updated together, and secondary indexes (separate objects) kept in step. Without them "error handling becomes much more complicated without atomicity, and the lack of isolation can cause concurrency problems" (alternatives: Chapter 12).

Handling errors and aborts

- A key feature is abort and safe retry; ACID databases would "rather abandon the transaction entirely than allow it to remain half-finished".
- Leaderless-replication stores work "best effort": they won't undo what they have done, so recovery is the application's job.
- ActiveRecord and Django don't retry aborted transactions: "the whole point of aborts is to enable safe retries." Retrying "isn't perfect": a lost commit acknowledgement means running twice "unless you have an additional application-level deduplication mechanism"; on overload it "will make the problem worse"; retry only transient errors, since after a permanent one "a retry would be pointless"; outside side effects (an email) can repeat.

Weak Isolation Levels

- Transactions on different data can run in parallel; races arise when one reads data another modifies, or both modify the same data.
- Such bugs are hard to test for ("when you get unlucky with the timing") and reason about: "any piece of data could unexpectedly change at any time".
- Serializable isolation would let you pretend there is no concurrency, but it has a performance cost, so systems use weaker levels that "protect against some concurrency issues, but not all", causing subtle bugs.
- Such bugs have "caused substantial loss of money" and corrupted customer data. "Use an ACID database if you're handling financial data!" "misses the point": many "ACID" databases use weak isolation. We need "a good understanding of the kinds of concurrency problems that exist".

Read Committed

- "The most basic level of transaction isolation is read committed", with two guarantees: no dirty reads (only committed data is read) and no dirty writes (only committed data is overwritten).

No dirty reads

- Reading uncommitted data is a **dirty read**; writes become visible only on commit, all at once (Figure 7-4: user 2 still reads 2 until user 1 commits x = 3). Otherwise a transaction sees some updates but not others (Figure 7-2), "confusing to users", and may see data "never actually committed" after a rollback, where "reasoning about the consequences quickly becomes mind-bending".

Figure 7-4. No dirty reads: user 2 sees the new value for x only after user 1's transaction has committed.

No dirty writes

- A later write overwriting an uncommitted one is a **dirty write**; usually the second write is delayed until the first transaction ends. Figure 7-5: Alice and Bob buy the same car (listing write, invoice write); the sale goes to Bob, the invoice to Alice. "Read committed prevents such mishaps", but not Figure 7-1: the second write follows a commit, "it's not a dirty write", yet it is incorrect.

Figure 7-5. With dirty writes, conflicting writes from different transactions can be mixed up.

Implementing read committed

- Default in Oracle 11g, PostgreSQL, SQL Server 2012 and others.
- Read locks would make read-only transactions wait behind one long write, harming "operability". So most databases keep the old committed value and the new value, and readers get the old one until commit.

Snapshot Isolation and Repeatable Read

- Read committed gives "much stronger guarantees than you can get from a system that has no transactions". Yet "there are still plenty of ways in which you can have concurrency bugs".
- Figure 7-6: Alice has $1,000 over two accounts of $500; during a $100 transfer she sees $500 and $400, so "$100 has vanished into thin air".

Figure 7-6. Read skew: Alice observes the database in an inconsistent state.

- This is a **nonrepeatable read** or **read skew** (re-reading account 1 gives $600), "considered acceptable under read committed" since each value was committed.

Note

- Here "skew" means **timing anomaly**.
- For Alice it passes on reload, but some cases cannot tolerate it. Backups take hours while writes continue, so a restore makes inconsistencies "permanent". Analytic queries and integrity checks "are likely to return nonsensical results" if they see different points in time.
- **Snapshot isolation** is the common solution: each transaction sees all data committed at its start, whatever changes later. A "boon for long-running, read-only queries such as backups and analytics".

Implementing snapshot isolation

- Write locks prevent dirty writes, but reads need none: "readers never block writers, and writers never block readers".
- It generalizes Figure 7-4: several committed versions are kept side by side, **multi-version concurrency control (MVCC)**. Read committed needs only two versions, but engines usually use MVCC for it too: "read committed uses a separate snapshot for each query, while snapshot isolation uses the same snapshot for an entire transaction".
- Figure 7-7 (PostgreSQL): each transaction gets an always-increasing transaction ID (**txid**), tagging its writes.

Figure 7-7. Implementing snapshot isolation using multi-version objects.

- An update is a delete plus a create: transaction 13 deducts $100 from account 2, leaving a $500 row marked deleted by 13 and a $400 row created by 13.

Visibility rules for observing a consistent snapshot

- Transaction IDs decide what a reader sees. At the start the database lists transactions in progress: (1) their writes "are ignored, even if the transactions subsequently commit"; (2) aborted transactions' writes are ignored; (3) writes by later transaction IDs are ignored; (4) all other writes are visible. An

object is visible if its creator had committed when the reader started and it is not deleted (or the deleter had not committed). Transaction 12 sees $500 in account 2.
- A long-running transaction may read values long overwritten; since new versions are created rather than updating in place, a consistent snapshot has "only a small overhead".

Indexes and snapshot isolation

- Either the index points to all versions and queries filter out invisible ones, or (CouchDB, Datomic, LMDB) append-only/copy-on-write B-trees copy modified pages up to the root; each write creates a new root, a consistent snapshot, needing background compaction.

Repeatable read and naming confusion

- Oracle calls snapshot isolation "serializable"; PostgreSQL and MySQL call it "repeatable read", because the SQL standard, based on System R's 1975 definitions, lacks snapshot isolation and defines a superficially similar repeatable read, so they can claim compliance.
- The standard's definition "is flawed—it is ambiguous, imprecise, and not as implementation-independent as a standard should be"; DB2 uses the term for serializability. "Nobody really knows what repeatable read means."

Preventing Lost Updates

- So far we looked at what read-only transactions see; dirty writes were only one write-write conflict. The best known other is the **lost update**: a **read-modify-write cycle** run concurrently, where the later write "clobbers" the earlier one. It arises with counters or balances, local changes to a complex value (a list in a JSON document), or two users saving a whole wiki page. Several solutions exist.

Atomic write operations

- Atomic updates, e.g. `UPDATE counters SET value = value + 1 WHERE key = 'foo';`, are "usually the best solution". Not all writes fit (wiki editing).
- Usually an exclusive lock on read (**cursor stability**), or a single thread. ORMs make unsafe cycles easy to write by accident.

Explicit locking

- If atomic operations don't suffice, lock the objects explicitly. In a multiplayer game, moves must obey rules you "cannot sensibly implement as a database query"; Example 7-1 locks the figure.

Example 7-1. Explicitly locking rows to prevent lost updates

- It works, but "it's easy to forget to add a necessary lock somewhere in the code, and thus introduce a race condition".

Automatically detecting lost updates

- Alternative: run cycles in parallel and, if the transaction manager detects a lost update, abort and retry. This pairs efficiently with snapshot isolation: PostgreSQL, Oracle and SQL Server detect it; MySQL/InnoDB's repeatable read does not.

Compare-and-set

- Without transactions, update only if the value is unchanged since read, else retry. Wiki example: update `WHERE content = 'old content'`.
- If the WHERE clause reads an old snapshot, lost updates may not be prevented: "Check whether your database's compare-and-set operation is safe before relying on it."

Conflict resolution and replication

- Locks and compare-and-set assume "a single up-to-date copy of the data"; multi-leader and leaderless replication replicate asynchronously, so they do not apply (see "Linearizability").
- Instead, concurrent writes create conflicting versions (**siblings**) merged afterwards by application code or special data structures.
- Atomic operations work if **commutative** (counter increment, adding to a set): Riak 2.0 datatypes merge updates so none is lost. **Last write wins (LWW)** "is prone to lost updates" and is the default in many replicated databases.

Write Skew and Phantoms

- Dirty writes and lost updates hit the same objects; “that is not the end of the list”. Doctors’ on-call shifts: at least one must stay on call; Alice and Bob both request leave at once (Figure 7-8).

Figure 7-8. Example of write skew causing an application bug.

- Each checks that two or more are on call; under snapshot isolation both see 2, each updates their own record, both commit, none is on call: “Your requirement of having at least one doctor on call has been violated.”

Characterizing write skew

- **Write skew** is neither dirty write nor lost update (different objects), yet “definitely a race condition”: serially, the second doctor would be refused.
- It generalizes lost updates: two transactions read the same objects, then update some (maybe different) ones; if the same, you get a dirty write or lost update.
- Options are “more restricted”: atomic single-object operations and automatic lost-update detection don’t help; it needs “true serializable isolation”. A constraint would span multiple objects, which most databases lack (triggers or materialized views may do). Second best: explicitly lock the rows the transaction depends on (select the on-call doctors `FOR UPDATE`, then update).

More examples of write skew

- Four more: meeting room booking (Example 7-2): check for overlaps, then insert; snapshot isolation allows a conflicting concurrent insert, so you need serializable isolation.

Example 7-2. A meeting room booking system tries to avoid double-booking (not safe under snapshot isolation)

- Multiplayer game: Example 7-1’s lock doesn’t stop two figures going to one position; a unique constraint may help, otherwise you are “vulnerable to write skew”.
- Usernames (a unique constraint is “a simple solution here”) and double-spending: two concurrent spending items can make the balance negative, “but that neither transaction notices the other”.

Phantoms causing write skew

- The pattern: (1) a SELECT checks a requirement by searching for matching rows; (2) the application decides to go ahead or abort; (3) it writes and commits; (4) the write changes the precondition of step 2. In the doctor case the write modifies a row the query returned, so locking works. The other four “check for the absence of rows matching some search condition, and the write adds a row matching the same condition”; with no rows, SELECT FOR UPDATE “can’t attach locks to anything”.
- A write changing another transaction’s search result is a **phantom**. Snapshot isolation avoids phantoms in read-only queries, but in read-write transactions they cause “particularly tricky cases of write skew”.

Materializing conflicts

- Perhaps “artificially introduce a lock object”: a table of rooms and 15-minute periods; a booking locks the matching rows, then checks and inserts. The table is “purely a collection of locks”.
- This **materializing conflicts** turns a phantom into a lock conflict on concrete rows, but it is “hard and error-prone” and “ugly to let a concurrency control mechanism leak into the application data model”: “a last resort”. “A serializable isolation level is much preferable in most cases.”

Serializability

- Some races are prevented by read committed and snapshot isolation, others not; write skew and phantoms are tricky: “It’s a sad situation”. Isolation levels are hard to understand and inconsistently implemented; from the code it is hard to tell whether it is safe at a given level; and there are no good tools to detect race conditions (testing is hard as problems are nondeterministic). Since the 1970s the researchers’ answer has been “use serializable isolation!”
- It is “usually regarded as the strongest isolation level”: parallel execution gives the end result of serial execution, so correct transactions stay correct and “the database prevents all possible race conditions”.

- Why isn't everyone using it? Three techniques, for now on a single node (distributed: Chapter 9): literally executing transactions in a serial order; two-phase locking, for decades the only viable option; and optimistic techniques such as serializable snapshot isolation.

Actual Serial Execution

- Simplest: remove concurrency, one transaction at a time on a single thread; the isolation is "by definition serializable".
- Designers decided only around 2007 it was feasible: RAM became cheap enough to keep the active dataset in memory, and OLTP transactions are usually short with few reads and writes, while read-only analytic queries can run on a snapshot outside the serial loop. Used in VoltDB/H-Store, Redis, Datomic.
- It avoids locking overhead but is limited to one CPU core, so transactions must be structured differently.

Encapsulating transactions in stored procedures

- Early on a transaction was to cover a whole user flow, e.g. booking an airline ticket. But "humans are very slow to make up their minds and respond", so OLTP keeps transactions short; on the web "a transaction does not span multiple requests".
- Yet transactions stayed interactive, statement by statement; with serial execution throughput "would be dreadful".
- Hence serial systems "don't allow interactive multi-statement transactions": the application submits the whole transaction ahead of time as a **stored procedure** (Figure 7-9), fast if data is in memory (footnote: **anti-caching** otherwise).

Figure 7-9. The difference between an interactive transaction and a stored procedure (using the example transaction of Figure 7-8).

Pros and cons of stored procedures

- In the SQL standard (SQL/PSM) since 1999 but with "a somewhat bad reputation": vendor languages that look "ugly and archaic"; code in the database is harder to debug, deploy, test and monitor; a bad procedure hurts more since one database serves many application servers. Modern ones use general languages: VoltDB Java or Groovy, Datomic Java or Clojure, Redis Lua.
- VoltDB replicates by running the same procedure on each replica, so procedures must be **deterministic**; current time needs special APIs.

Partitioning

- Serial execution limits throughput to one core; read-only transactions can run elsewhere with snapshot isolation, but write-heavy applications hit "a serious bottleneck".
- Partition (Chapter 6; VoltDB): single-partition transactions give each core its own thread, so throughput scales linearly.
- Multi-partition transactions need lock-step coordination and are "vastly slower" (VoltDB: about 1,000 cross-partition writes per second, not raised by adding machines). Key-value data partitions easily; secondary indexes need cross-partition coordination.

Summary of serial execution

- It is "a viable way of achieving serializable isolation within certain constraints": every transaction small and fast, since one slow one stalls everything; the active dataset must fit in memory; write throughput low enough for one CPU core, or partitioned without cross-partition coordination, which has "a hard limit".

Two-Phase Locking (2PL)

- "For around 30 years, there was only one widely used algorithm for serializability in databases: two-phase locking (2PL)." Footnote: sometimes called SS2PL.

2PL is not 2PC

- 2PL and two-phase commit "are completely different things" (2PC: Chapter 9).

- Locks already prevent dirty writes; 2PL is "much stronger": many readers while nobody writes, but any write (modify or delete) needs exclusive access. A reader blocks a later writer, and a writer blocks a later reader, until commit or abort.
- "Writers don't just block other writers; they also block readers and vice versa", unlike snapshot isolation's mantra. Because it is serializable, it prevents all earlier races, lost updates and write skew included.

Implementation of two-phase locking

- Used in MySQL (InnoDB) and SQL Server serializable and DB2 repeatable read. Each object has a lock in **shared mode** or **exclusive mode**. To read, acquire it shared (several may hold it; wait if exclusive is held); to write, acquire exclusive (no other lock may exist); shared may upgrade to exclusive; locks are held to the end of the transaction. Hence "two-phase": acquire while executing, release all at the end.
- Many locks make **deadlock** easy; the database detects it and aborts one transaction, which the application retries.

Performance of two-phase locking

- The big downside: "transaction throughput and response times of queries are significantly worse under two-phase locking than under weak isolation", partly lock overhead, "more importantly due to reduced concurrency".
- Databases built for interactive use set no transaction duration limit, so waits are unbounded, and queues form even with short transactions. Hence "quite unstable latencies"; one slow transaction can "cause the rest of the system to grind to a halt".
- Deadlocks are "much more frequent" than under read committed; each retry redoes all the work.

Predicate locks

- A serializable database must prevent phantoms: in Example 7-2, once one transaction searched a room and window, another may not insert or update a conflicting booking, though other rooms or times are fine.
- A **predicate lock** works like the shared/exclusive lock but belongs to all objects matching a search condition (room 123, noon to 1 pm), not one row. A reader takes a shared predicate lock on its query conditions; a writer must check whether the old or new value matches an existing predicate lock and wait for its holder.
- "The key idea" is that it applies even to objects not yet in the database (phantoms). With it, 2PL prevents all write skew and becomes serializable.

Index-range locks

- Predicate locks "do not perform well" with many locks, so most 2PL databases use **index-range locking** (**next-key locking**), "a simplified approximation of predicate locking".
- Attached to an index entry (room_id 123) or a range of a time-based index (noon to 1 p.m.), the approximation makes a conflicting writer meet the shared lock and wait. Less precise but cheaper: "a good compromise". With no suitable index it falls back to a shared lock on the whole table, bad for performance but "a safe fallback position".

Serializable Snapshot Isolation (SSI)

- The "bleak picture": serializable implementations that don't perform well (2PL) or scale well (serial), versus fast weak levels prone to races. Are they "fundamentally at odds"? "Perhaps not": SSI gives full serializability with "only a small performance penalty compared to snapshot isolation".
- First described in 2008; used in PostgreSQL since 9.1 and similarly FoundationDB. It is young and "still proving its performance in practice", but could "become the new default in the future".

Pessimistic versus optimistic concurrency control

- 2PL is **pessimistic**: if anything might go wrong, wait. Serial execution is "pessimistic to the extreme", an exclusive lock on the whole database or partition, offset by making transactions fast.
- SSI is **optimistic**: transactions continue "in the hope that everything will turn out all right"; at commit the database checks whether isolation was violated and, if so, aborts. "Only transactions that executed serializably are allowed to commit."

- It “performs badly if there is high contention”, and retries add load near maximum throughput; with spare capacity and low contention it tends to beat pessimistic control. Commutative atomic operations reduce contention.
- SSI builds on snapshot isolation, the main difference from earlier optimistic techniques, adding an algorithm for “detecting serialization conflicts among writes and determining which transactions to abort”.

Decisions based on an outdated premise

- The database can’t know how the application uses a result, so it assumes a changed premise may invalidate the writes: “there may be a causal dependency between the queries and the writes”. It must detect possible outdated premises and abort. There are two cases: detecting reads of a stale MVCC object version (the uncommitted write occurred before the read), and detecting writes that affect prior reads (the write occurs after the read).

Detecting stale MVCC reads

- Figure 7-10: transaction 43 sees Alice on_call = true because 42 is uncommitted; by 43’s commit, 42 has committed, so 43’s premise is false.

Figure 7-10. Detecting when a transaction reads outdated values from an MVCC snapshot.

- The database tracks writes ignored through MVCC visibility and, at commit, aborts if any has committed.
- Why not abort at once? A read-only transaction needn’t abort; the database doesn’t yet know whether it will write; and 42 may still abort or be uncommitted. “By avoiding unnecessary aborts, SSI preserves snapshot isolation’s support for long-running reads from a consistent snapshot.”

Detecting writes that affect prior reads

- Second case: another transaction modifies data after it was read (Figure 7-11).

Figure 7-11. In serializable snapshot isolation, detecting when one transaction modifies another transaction’s reads.

- Like index-range locks “except that SSI locks don’t block other transactions”. Transactions 42 and 43 both search shift 1234; the index entry (or table level) records both read it, kept only until all concurrent transactions finish.
- A writer checks the indexes for recent readers; the lock “acts as a tripwire”, notifying them that their reads may be out of date.
- 43 notifies 42 and vice versa. 42 commits first, successfully, since 43’s write has not taken effect; when 43 commits, 42’s write has, so 43 must abort.

Performance of serializable snapshot isolation

- Granularity of tracking is a trade-off: detailed tracking aborts precisely but costs bookkeeping; coarser is faster but “may lead to more transactions being aborted than strictly necessary”. PostgreSQL proves some overwritten reads still serializable to cut aborts.
- Versus 2PL: no blocking, “writers don’t block readers, and vice versa”, so latency is “much more predictable and less variable”, and read-only queries need no locks, “very appealing for read-heavy workloads”.
- Versus serial execution: not limited to one core; FoundationDB distributes conflict detection across machines and allows serializable multi-partition transactions.
- The abort rate “significantly affects” performance, so SSI “requires that read-write transactions be fairly short” (long read-only ones may be okay), but it is probably less sensitive to slow transactions than 2PL or serial execution.

Summary

- “Transactions are an abstraction layer that allows an application to pretend that certain concurrency problems and certain kinds of hardware and software faults don’t exist.” A large class of errors reduces to a transaction **abort** and retry.
- Not all applications suffer all problems: single-record access can probably manage without; for complex patterns transactions “can hugely reduce the number of potential error cases”. Without them, denormalized data “can easily go out of sync with the source data”.

- Isolation levels: read committed, snapshot isolation (sometimes called repeatable read), serializable, characterized by races. Dirty reads (prevented by read committed and stronger) and dirty writes (by almost all implementations); read skew (snapshot isolation, usually via MVCC); lost updates (some snapshot isolation implementations automatically, others a manual SELECT FOR UPDATE); write skew ("Only serializable isolation prevents this anomaly"); phantom reads (straightforward ones by snapshot isolation, those in write skew by index-range locks).
- Weak levels leave the developer to handle others manually; "Only serializable isolation protects against all of these issues." Three implementations: serial execution (simple, effective if each transaction is very fast and throughput fits one core); two-phase locking (the standard for decades, often avoided for performance); SSI ("a fairly new algorithm that avoids most of the downsides of the previous approaches", optimistic, checked at commit).
- The examples were relational, but transactions are valuable "no matter which data model is used" (see "The need for multi-object transactions"). Distributed transactions "open a new set of difficult challenges", for the next two chapters.

Footnotes

- Folded into the sections above; Hellerstein said the C in ACID was "tossed in to make the acronym work".

References

- The chapter ends with a list of references (not summarised).

How this chapter builds the author's case

The chapter turns the "harsh reality" of faults (Chapters 5 and 6) into one abstraction, the transaction with abort and retry, then refuses to treat ACID as a slogan: consistency belongs to the application and "ACID" is mostly marketing. Each weak level is introduced by the anomaly the one below allows, until the "sad situation" makes the case for serializability and three implementations weighed by performance, ending on SSI as the promising answer. Back to Chapters 5 and 6; forward to Chapter 8 (distributed challenges), Chapter 9 (distributed transactions, 2PC) and Chapter 12 (alternatives).

CHAPTER 8

The Trouble with Distributed Systems

Place in the argument — why this chapter, why this title

The title names the chapter's job: to show what goes wrong. The epigraph on the "laggy" network signals the first villain. Kleppmann says earlier chapters (failover, replication lag, weak isolation) "have still been too optimistic"; here he turns "our pessimism to the maximum". It sets up Chapter 9, which gives algorithms that provide guarantees despite all this. His roadmap fixes the order: networks, then clocks and timing, then how to reason about knowledge and truth, ending with the system model that makes reasoning possible.

Chapter opening — untitled section before the first sub-heading in the book

- Earlier chapters handled things going wrong; understanding edge cases makes us better at handling them. Now: "assume that anything that can go wrong will go wrong" (a footnote's one exception: faults are non-Byzantine).

> *"there are lots of new and exciting ways for things to go wrong"*

- Aim: a taste of problems in practice and "an understanding of the things we can and cannot rely on"; engineers must meet users' guarantees "in spite of everything going wrong". Chapter 9 gives algorithms; first, the challenges.
- Roadmap: networks, clocks and timing issues and how far they are avoidable, then how to think about the state of a system and reason about what happened.

Faults and Partial Failures

- One computer is deterministic: it works or it doesn't; hardware faults usually mean total failure. This is deliberate: a computer should crash rather than return a wrong result, hiding fuzzy physical reality behind an idealized model of mathematical perfection.
- Several computers on a network: "we have no choice but to confront the messy reality of the physical world" (Coda Hale's anecdote: partitions, PDU, switch, rack, DC power failures, a truck hitting the HVAC).
- A **partial failure**: some parts broken unpredictably while others work. They are **nondeterministic**: an operation across nodes may sometimes work, sometimes fail, and you may not know which, as message time is nondeterministic too.

> *"This nondeterminism and possibility of partial failures is what makes distributed systems hard to work with"*

Cloud Computing and Supercomputing

- Spectrum: at one end high-performance computing (HPC), supercomputers with thousands of CPUs for scientific computing such as weather forecasting; at the other **cloud computing**, "not very well defined" but associated with multi-tenant datacenters, commodity computers on an IP network, elastic/on-demand allocation and metered billing; enterprise datacenters lie between. Very different fault handling follows. Supercomputer: checkpoint to durable storage; if a node fails, stop the whole cluster, repair, restart from the last checkpoint. It is "more like a single-node computer": partial failure escalates into total failure.
- Internet services, the book's focus, look very different: many are **online**, needing low-latency service at any time, so stopping the cluster for repair is not acceptable, whereas **offline** (batch) jobs can be stopped and restarted. Supercomputers use specialized, quite reliable nodes with shared memory and RDMA and special network topologies (meshes, toruses); cloud nodes are commodity machines, equivalent performance at lower cost but "higher failure rates", on IP/Ethernet Clos topologies. Geographically distributed deployments communicate over the internet, "slow and unreliable compared to local networks".
- "The bigger a system gets, the more likely it is that one of its components is broken"; with thousands of nodes assume "something is always broken". Giving up as error handling makes a large system spend its time recovering rather than doing useful work. Tolerating failed nodes helps operations: a rolling upgrade (Chapter 4) restarts one node at a time, and a badly performing VM can be killed and replaced.

- Hence accept partial failure and build fault tolerance into software: “build a reliable system from unreliable components”, knowing there is no perfect reliability.
- Even a few nodes: sooner or later some part faults; fault handling is part of design and operators must know the expected behaviour. Test by artificially creating even unlikely faults.

> *“suspicion, pessimism, and paranoia pay off”*

Building a Reliable System from Unreliable Components

- A system is not merely as reliable as its weakest link: it is “an old idea in computing” to build reliable systems on a less reliable base.
- Examples: error-correcting codes transmit data accurately over a channel that occasionally gets bits wrong; IP is unreliable (it may drop, delay, duplicate, or reorder packets) and TCP on top ensures missing packets are retransmitted, duplicates eliminated and order restored.
- But there is a limit: error-correcting codes cannot beat a swamped signal; TCP hides loss, duplication and reordering but “cannot magically remove delays in the network”.
- The higher level is still useful: the remaining faults are easier to reason about (“The end-to-end argument” follows up).

Unreliable Networks

- Part II’s focus is **shared-nothing systems**: own memory and disk, the network the only link. Dominant for internet services: cheap, commodity cloud, reliability through geographically distributed datacenters.
- The internet and most datacenter networks are **asynchronous packet networks**: no guarantee of when a message arrives, or whether at all.

Figure 8-1. If you send a request and don’t get a response, it’s not possible to distinguish whether (a) the request was lost, (b) the remote node is down, or (c) the response was lost.

- The figure shows that these cases look identical to the sender: the request lost or queued for later; the remote node failed or temporarily stopped responding (e.g. a long GC pause); the request processed but the response lost (misconfigured switch) or delayed.
- The only evidence is a response, which may itself be lost or delayed.
- The usual remedy is a **timeout**, but you still don’t know whether the remote node got the request.

Network Faults in Practice

- Despite decades of experience, problems are “surprisingly common”, even within one company’s datacenter. A medium-sized datacenter had about 12 faults a month, half disconnecting one machine, half a rack.
- Redundant gear doesn’t reduce faults as much as hoped, since it doesn’t guard against human error such as misconfigured switches. EC2 is notorious for glitches; private networks are stabler; nobody is immune (a switch upgrade can delay packets over a minute; sharks bite cables).
- An interface may drop inbound packets yet send outbound: a link working one way need not work the other.

Network partitions

- A **network partition** (netsplit) means part of the network is cut off; the book says **network fault** to avoid confusion with Chapter 6’s partitions (shards).
- Even if rare, faults can occur: “Whenever any communication happens over a network, it may fail”. Undefined, untested handling can deadlock the cluster even after recovery, or delete all your data.
- Handling needn’t mean tolerating: showing users an error may be valid. But know how software reacts and ensure recovery; deliberately trigger faults (Chaos Monkey).

Detecting Faults

- Many systems must detect faulty nodes: a load balancer must take a dead node “out of rotation”; single-leader replication must promote a follower if the leader fails. Network uncertainty makes it hard. Explicit feedback sometimes exists: a reachable machine with no listening process refuses TCP connections (RST/FIN), though a crash mid-request leaves you not knowing how much was processed; a script can tell others a process crashed (HBase does this); switch management interfaces reveal link failures (unavailable over the internet or in shared datacenters); a router may

reply ICMP Destination Unreachable but has no magic detection either. Feedback can't be counted on: TCP may acknowledge a packet and the application crash before handling it. For certainty you need a positive response from the application itself.

- Assume no response: retry, wait for the timeout, eventually declare the node dead.

Timeouts and Unbounded Delays

- How long a timeout? "There is unfortunately no simple answer." Long means a long wait; short risks declaring a merely slow node dead.
- Premature death declaration: the node's action (e.g. sending an email) may be done twice by its replacement; moving its load onto others can cause a **cascading failure** (extreme: all nodes declare each other dead).
- Fictitious system: delivery within d or lost, handling within r; then 2d + r is a reasonable timeout. Real systems have neither: asynchronous networks have **unbounded delays**, servers can't promise maximum handling time. Fast most of the time isn't enough: "it only takes a transient spike in round-trip times to throw the system off-balance".

Network congestion and queueing

- As road trips vary with traffic, packet delay varies mostly due to queueing: several senders to one destination are queued by the switch (**network congestion**) and if the queue fills the packet is dropped and resent "even though the network is functioning fine". At the destination, if all CPU cores are busy the OS queues the request for an arbitrary time; in virtualized environments the OS is paused for tens of milliseconds while another VM runs and data is buffered. TCP flow control (backpressure) limits the sender's own rate, adding queueing at the sender.

Figure 8-2. If several machines send network traffic to the same destination, its switch queue can fill up. Here, ports 1, 2, and 4 are all trying to send packets to port 3.

- The figure shows several senders overfilling one switch queue. TCP also retransmits after a timeout; the application sees not the loss but the resulting delay.

TCP Versus UDP

- Latency-sensitive apps (VoIP) use UDP: a trade-off between reliability and variable delay. Good "in situations where delayed data is worthless": fill the slot with silence; "The retry happens at the human layer instead."
- Queueing delays vary widely near maximum capacity. In shared clouds a **noisy neighbor** (e.g. MapReduce saturating links) makes delays highly variable.
- So choose timeouts experimentally: measure round-trip distributions over time and machines. Better, measure response times and **jitter** and adjust automatically, e.g. the Phi Accrual failure detector (Akka, Cassandra); TCP retransmission timeouts work similarly.

Synchronous Versus Asynchronous Networks

- Why not make the network reliable in hardware? Compare the fixed-line telephone network, extremely reliable. A call establishes a **circuit**: fixed, guaranteed bandwidth along the whole route. ISDN: 4,000 frames per second, 16 bits per frame per direction, i.e. 16 bits every 250 microseconds.
- This is **synchronous**: no queueing, so end-to-end latency is fixed, a **bounded delay**.

Can we not simply make network delays predictable?

- A circuit is reserved bandwidth; a TCP connection opportunistically uses what's available and uses none when idle. Ethernet and IP are packet-switched, with queueing and unbounded delays.
- Why? They are optimized for **bursty traffic**. For a file over a circuit you must guess: too low is slow, too high and the circuit can't be set up. "TCP dynamically adapts the rate of data transfer to the available network capacity."
- Hybrids (ATM, InfiniBand's link-level flow control) reduce but don't remove delays; QoS plus admission control can emulate circuits or give statistically bounded delay.

Latency and Resource Utilization

- Variable delay follows from dynamic resource partitioning. A wire for 10,000 calls is static: your slot stays reserved even when 9,999 are idle. The internet shares dynamically: queueing, but maximal

utilization, so each byte is cheaper. CPUs likewise: threads wait in run queues, but hardware is better used (a motive for virtual machines).

- Latency guarantees need static partitioning at lower utilization, i.e. more cost.

> *"Variable delays in networks are not a law of nature, but simply the result of a cost/benefit trade-off."*

- Such QoS isn't enabled in multi-tenant datacenters, public clouds or the internet, so assume congestion and unbounded delays: "there's no "correct" value for timeouts—they need to be determined experimentally."

Unreliable Clocks

- Applications use clocks for durations (examples 1–4: has this request timed out; 99th percentile response time; queries per second over five minutes; time a user spent on the site) and points in time (5–8: article publication, when a reminder email is sent, cache entry expiry, a log timestamp). Receipt is always later than sending but by an unknown amount, so ordering across machines is hard.
- Each machine has its own quartz clock, not perfectly accurate; the Network Time Protocol (NTP) synchronizes to some degree against servers that get time from e.g. GPS.

Monotonic Versus Time-of-Day Clocks

- Computers have at least two kinds, for different purposes.

Time-of-day clocks

- Return date and time per a calendar (**wall-clock time**): seconds since the **epoch**, January 1, 1970 UTC, not counting leap seconds. Usually NTP-synchronized, but may be forcibly reset and appear to jump back, so unsuitable for measuring elapsed time. Resolution was historically coarse.

Monotonic clocks

- Suitable for durations (timeouts, response times); "guaranteed to always move forward". Only differences mean anything; values from two computers can't be compared.
- Per-CPU timers: take monotonicity "with a pinch of salt". NTP may slew the rate (up to 0.05% by default) but can't make it jump. In a distributed system they're usually fine for elapsed time since they assume no synchronization.

Clock Synchronization and Accuracy

- Time-of-day clocks need an external source, and hardware clocks and NTP "can be fickle beasts":
- Quartz drift varies with temperature; Google assumes 200 ppm, i.e. 6 ms for a resync every 30 seconds or 17 seconds for once a day.
- A clock too far from NTP may refuse to sync or be forcibly reset, so applications may see time go backward or jump forward; a firewalled-off node may go unnoticed.
- NTP accuracy is limited by network delay: one experiment achieved a minimum error of 35 ms over the internet, with spikes to around a second; some NTP servers are wrong by hours. Clients are robust (several servers, outliers ignored), but "somewhat worrying to bet the correctness of your systems on the time that you were told by a stranger on the internet".
- Leap seconds make a minute 59 or 61 seconds long and have crashed many large systems; the best handling may be to make NTP servers "lie" by smearing the adjustment over a day.
- In virtual machines the clock is virtualized; a VM paused for tens of milliseconds looks like the clock jumping forward.
- On devices you don't control, users may set the clock wildly wrong, so you probably cannot trust it.
- Good accuracy is possible with resources: MiFID II requires high-frequency trading funds to be within 100 microseconds of UTC, via GPS, the Precision Time Protocol and careful monitoring. A misconfigured NTP daemon or firewall lets drift error grow quickly.

Relying on Synchronized Clocks

- Pitfalls: a day may not have 86,400 seconds, clocks may move backward, nodes differ. As with networks, software must be prepared for incorrect clocks.
- Incorrect clocks go unnoticed: a bad CPU or network fails visibly, a bad quartz or NTP client leaves things "fine". The result is more likely "silent and subtle data loss than a dramatic crash". So monitor offsets and declare drifting nodes dead.

Timestamps for ordering events

- Tempting but dangerous: who wrote first? Figure 8-3: multi-leader replication (like Figure 5-9). Client A writes x = 1 on node 1; replicated to node 3; client B increments x on node 3 (x = 2); both replicated to node 2.

Figure 8-3. The write by client B is causally later than the write by client A, but B's write has an earlier timestamp.

- Writes are tagged by the origin node's time-of-day clock. Skew under 3 ms, yet x = 1 gets 42.004 s and x = 2 gets 42.003 s. Node 2 drops x = 2: B's increment is lost.
- This is **last write wins** (LWW), widely used (Cassandra, Riak); client timestamps don't change the problems: writes can silently vanish (a lagging-clock node cannot overwrite a fast-clock node's values until the skew elapses, "arbitrary amounts of data" dropped without error); LWW cannot distinguish sequential writes from truly concurrent ones, so version vectors are needed; and two nodes may generate the same timestamp, needing a tiebreaker (e.g. a large random number) that can also violate causality. Even tight NTP could have a packet sent at 100 ms arrive at 99 ms.
- Better NTP? "Probably not": accuracy is limited by round-trip time and drift; the clock must be much more accurate than network delay.
- Safer: **logical clocks**, counters measuring only relative order; time-of-day and monotonic clocks are **physical clocks**.

Clock readings have a confidence interval

- Fine resolution isn't accuracy: drift is milliseconds even with local NTP every minute; over the internet tens of milliseconds, spiking over 100 ms. A reading is "more like a range of times, within a confidence interval" (e.g. 95% sure of 10.3 to 10.5 s past the minute); at +/– 100 ms microsecond digits are "essentially meaningless".
- Uncertainty is computable (manufacturer error for GPS/atomic clocks; drift plus server uncertainty plus round-trip), but most APIs hide it: "five milliseconds or five years". Exception: Google's TrueTime API in Spanner returns [earliest, latest].

Synchronized clocks for global snapshots

- Snapshot isolation usually uses monotonically increasing transaction IDs: a counter suffices on one node. Distributed, a global ID needs coordination, must reflect causality, and becomes an "untenable bottleneck". (Footnote: Snowflake-style IDs can't guarantee causal order.)
- Timestamps would have the right properties given good synchronization; the trouble is uncertainty. Spanner uses TrueTime intervals: if A's and B's intervals don't overlap, B definitely happened after A.
- So Spanner "deliberately waits for the length of the confidence interval" before committing a read-write transaction, keeping uncertainty small with a GPS receiver or atomic clock per datacenter (about 7 ms). Active research, not yet mainstream outside Google.

Process Pauses

- Single leader per partition: how does it know it is still leader? Obtain a **lease**, "similar to a lock with a timeout", renewed periodically.
- The book's loop: if the lease has under 10 seconds left, renew; if valid, process the request. Flaw one: expiry is set by another machine and compared to the local clock. Flaw two, even with monotonic clocks: it assumes little time passes between check and processing. If the thread pauses 15 seconds, the lease has expired and another node leads, yet "there is nothing to tell this thread that it was paused for so long".
- Is that crazy? "Unfortunately not". Causes: stop-the-world GC pauses, pauses that have "sometimes been known to last for several minutes", and even "concurrent" collectors like CMS must stop the world sometimes, so "we must assume the worst"; VM suspend/resume and live migration, or a laptop lid closed; context switches and hypervisor switches (**steal time**) under heavy load; synchronous disk I/O, even hidden (the Java classloader), network filesystems such as EBS; paging and **thrashing** (so paging is often disabled on servers); SIGSTOP (Ctrl-Z), possibly sent accidentally. All can preempt a thread unnoticed. Mutexes, semaphores, atomic counters and blocking queues don't carry over: no shared memory, only an unreliable network.
- A node must assume it can be paused at any point; the world moves on and may declare it dead.

Response time guarantees

- Pauses can be eliminated "if you try hard enough". Where missing a deadline fails the whole system (aircraft, rockets, cars) we have **hard real-time** systems.

Is real-time really real?

- Embedded real-time means designed and tested to meet timing guarantees in all circumstances, unlike the vague web use (Chapter 11). E.g. an airbag must not wait for a GC pause.
- It needs a real-time operating system (RTOS), documented worst-case times, restricted dynamic memory allocation and heavy testing. So it is very expensive and restricts languages and tools. It is not "high-performance": throughput may be lower.
- For most server-side systems it is not economical, so they suffer pauses and clock instability.

Limiting the impact of garbage collection

- Mitigate without real-time guarantees: treat GC pauses like brief planned outages; the runtime warns the application, which stops sending requests to the node, lets it finish, then collects. This hides pauses and reduces high percentiles (used in some financial trading systems).
- Variant: collect only short-lived objects and restart processes periodically, shifting traffic first like a rolling upgrade. These cannot fully prevent pauses but reduce their impact.

Knowledge, Truth, and Lies

- No shared memory, only messages over an unreliable network, with partial failures, unreliable clocks and pauses. A node "cannot know anything for sure—it can only make guesses based on the messages it receives (or doesn't receive)"; network problems can't be told from node problems.
- Philosophical questions follow, but we needn't find the meaning of life: state assumptions (the **system model**) and design to meet them. Algorithms can be proved correct in a model, so reliable behaviour is achievable even with few guarantees, though not straightforward. Chapter 9 gives examples.

The Truth Is Defined by the Majority

- Scenarios: a node that receives but whose outgoing messages are dropped is declared dead ("I'm not dead!"); a node that notices the fault is still wrongly declared dead; a node after a one-minute stop-the-world GC pause resumes "as if nothing had happened", unaware a minute passed.

> *"a node cannot necessarily trust its own judgment of a situation"*

- No system can rely on one node. Many algorithms use a **quorum**: voting, with decisions needing a minimum number of votes. If a quorum declares a node dead it is dead, and it must step down.
- Usually an absolute majority of more than half: three nodes tolerate one failure, five tolerate two. It is safe because there can be only one majority. (Chapter 9 returns to this.)

The leader and the lock

- Often there must be only one of something: one leader per partition (avoiding split brain), one holder of a lock on a resource, one user registering a username. A node believing it is "the chosen one" doesn't mean a quorum agrees; it may have been demoted after a GC pause or network interruption. Acting on may make the system do something incorrect.
- Figure 8-4: a lock-service lease protecting a file (HBase used to have this bug).

Figure 8-4. Incorrect implementation of a distributed lock: client 1 believes that it still has a valid lease, even though it has expired, and thus corrupts a file in storage.

- The figure shows the "Process Pauses" problem: client 1 pauses, its lease expires, client 2 gets it and writes; client 1 resumes believing it still holds the lease and writes too, corrupting the file.

Fencing tokens

- The simple remedy is **fencing**: each lock or lease grant returns a **fencing token**, a number that increases with every grant; every write must include it.

Figure 8-5. Making access to storage safe by allowing writes only in the order of increasing fencing tokens.

- Client 1 gets token 33, pauses; client 2 gets 34 and writes. Client 1 then writes with 33 and the storage server, having processed 34, rejects it. ZooKeeper's zxid or cversion can serve.
- The resource itself must check tokens: "it is not sufficient to rely on clients checking their lock status themselves". Server-side checking is good: services should protect themselves from accidentally abusive clients.

Byzantine Faults

- Fencing blocks inadvertent error, but a node deliberately lying could fake a token.

> *"In this book we assume that nodes are unreliable but honest"*

- Slow, silent or outdated nodes are fine, but responses are the "truth" as best they know. If nodes may "lie" (arbitrary or corrupted responses), that is a **Byzantine fault**, and consensus there is the **Byzantine Generals Problem**.

The Byzantine Generals Problem

- It generalizes the **Two Generals Problem**: two generals agree a plan by messengers that may be delayed or lost (Chapter 9). Byzantine version: n generals, some traitors sending fake messages, unknown in advance. The name comes from Byzantine as excessively complicated, devious, not from history; Lamport was advised against "The Albanian Generals Problem".
- A system is **Byzantine fault-tolerant** if it works correctly despite malfunctioning nodes or attackers. Relevant in specific circumstances: aerospace, where radiation can corrupt memory or registers and a failure would be very expensive, so flight control must tolerate Byzantine faults; and multiple organizations where some participants may cheat, e.g. Bitcoin and blockchains getting mutually untrusting parties to agree "without relying on a central authority". But usually safely ignorable here: your nodes are controlled by your organization, radiation is low, and protocols are complicated and costly.
- Web apps must expect malicious clients, hence input validation, sanitization and output escaping, but the server is simply the authority; peer-to-peer networks, lacking one, need BFT more.
- A bug is a Byzantine fault, but BFT needs a supermajority of more than two-thirds correct (four nodes, one faulty) and so independent implementations. A compromised node means all compromised since software is shared, so "authentication, access control, encryption, firewalls" remain the main protection.

Weak forms of lying

- Still, guard against weak "lying": invalid messages from hardware, bugs or misconfiguration. Not full BFT, but "simple and pragmatic steps toward better reliability" (examples: checksums in the application-level protocol, since corrupted packets sometimes evade TCP and UDP checksums; sanitizing user inputs for range and string size; basic sanity-checking even behind a firewall; configuring NTP clients with several servers so a majority range excludes a wrong one as an outlier).

System Model and Reality

- Algorithms must not depend on hardware and software details, so we define a **system model**, "an abstraction that describes what things an algorithm may assume".
- Three timing models: the **synchronous model** assumes bounded delay, pauses and clock error (not exact synchronization); unrealistic. **Partial synchrony**: synchronous most of the time but sometimes exceeding bounds, arbitrarily; realistic. **Asynchronous model**: no timing assumptions, not even a clock, so no timeouts; very restrictive.
- Node models: **crash-stop** (fails only by crashing, gone forever); **crash-recovery** (may return, with stable storage surviving, memory lost); **Byzantine** (arbitrary, including deception).
- Most useful: partially synchronous with crash-recovery faults. How do algorithms cope?

Correctness of an algorithm

- Correctness is defined by **properties**, as a sorted list has each element smaller than its right neighbour. For fencing tokens: no two requests return the same value; if x completed before y began then t_x < t_y; a non-crashing requester eventually gets a response.

- Correct in a model means the properties always hold in all situations the model allows. But if all nodes crash, nothing works, so how does this make sense?

Safety and liveness

- Uniqueness and monotonic sequence are **safety** properties; availability is **liveness**, often containing "eventually" (eventual consistency is liveness). Informally "nothing bad happens" and "something good eventually happens", but "it's best to not read too much into those informal definitions"; the real ones are mathematical. If a safety property is violated we can point at the moment it broke and "the violation cannot be undone—the damage is already done"; a liveness property may not hold now but "there is always hope" it will be satisfied in the future.
- Safety must hold always, in all situations of the model, even if all nodes crash. Liveness may carry caveats: a response only if a majority haven't crashed and the network eventually recovers. Partial synchrony requires eventual return to a synchronous state.

Mapping system models to the real world

- "the messy facts of reality come back to bite you again": disks may be corrupted or wiped, firmware may fail to recognize drives. Quorums rely on a node remembering what it stored; amnesia breaks correctness. A "mostly survives" storage model is harder to reason about.
- Theory may assume things don't happen, yet implementations may need code for the "impossible", even printf("Sucks to be you") and exit(666): "arguably the difference between computer science and software engineering".
- Models are "incredibly helpful" for reducing complexity to faults we can reason about. A proof isn't a guarantee for an implementation but is "a very good first step". "Theoretical analysis and empirical testing are equally important."

Summary

- Problems covered: a packet sent may be lost or arbitrarily delayed, as may the reply, so without one you can't know whether it got through; a node's clock may be significantly out of sync, jump, and its error interval is unknown; a process may pause at any point, be declared dead, and come back "without realizing that it was paused".
- Partial failure is "the defining characteristic of distributed systems": interaction with other nodes may fail, go slow or time out; we build tolerance into software.
- Detection is hard: algorithms rely on timeouts, which can't distinguish network from node failure and may falsely suspect nodes. A degraded "limping" node (Gigabit NIC dropping to 1 Kb/s) can be harder than a cleanly failed one.
- Tolerance is hard too: no global variable, shared memory or common knowledge; nodes can't agree on the time. Major decisions need a quorum.
- If you can avoid "opening Pandora's box" and keep things on one machine, it is generally worth doing so. But fault tolerance and low latency, like scalability, need distribution (Part II introduction).
- Unreliability is not a law of nature: hard real-time and bounded delays exist but are expensive and lower utilization; most systems choose cheap and unreliable. Distributed systems can run forever by handling faults at the node level, "at least in theory" (a bad configuration rollout still topples them).
- The chapter has been all about problems; the next moves to solutions.

Footnotes

- Six footnotes (non-Byzantine assumption, TCP keepalive, ATM, BGP peering, CLOCK_REALTIME, Snowflake IDs), covered where they arise.

References

- The chapter ends with references (not summarised).

How this chapter builds the author's case

This chapter turns the optimism of the replication, partitioning and transaction chapters into pessimism, showing deeper causes: unreliable networks, clocks and unnoticed pauses. It ties back to Chapter 5 (multi-leader conflicts and LWW), Chapter 6 (shards), Chapter 7 (snapshot isolation) and to "Reliability" and Part II's introduction. It looks forward to Chapter 9 (quorums, consensus, the Two Generals Problem, safety and liveness in the partially synchronous crash-recovery model), Chapter 10 and Chapter 11. The case: since faults are a cost/benefit trade-off, build on explicit system models and quorums, not on one node's belief, a clock, or a timeout.

CHAPTER 9

Consistency and Consensus

Place in the argument — why this chapter, why this title

The title pairs consistency guarantees with consensus (getting nodes to agree). It follows Chapter 8, whose faults (lost, reordered or delayed packets, approximate clocks, pausing or crashing nodes) are all assumed, and uses the method of Chapter 7: "find some general-purpose abstractions with useful guarantees, implement them once, and then let applications rely on those guarantees". The sections climb a ladder: linearizability, the strongest simple model; ordering and causality, weaker and cheaper; total order broadcast, linking ordering to consensus; then consensus itself, with 2PC as stepping stone and ZooKeeper-style services as the practical use. Consensus comes late because "appreciating the subtleties requires some prerequisite knowledge".

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph (Jay Kreps): "Is it better to be alive and wrong or right and dead?"
- Simplest handling of faults is to let the whole service fail and show an error; if unacceptable, we must tolerate faults, "keeping the service functioning correctly, even if some internal component is faulty".
- A key one is **consensus**: "getting all of the nodes to agree on something", a "surprisingly tricky problem" under network faults and process failures.
- Use: if a single-leader database's leader dies, the remaining nodes use consensus to elect a new one. Two nodes both believing they lead is **split brain**, which "often leads to data loss".
- Order: first the range of guarantees and abstractions, then algorithms for consensus. We need the scope of what can and cannot be done; formal models and proofs are out of scope, so "we will stick with informal intuitions".

Consistency Guarantees

- Two nodes looked at the same moment will likely differ, because writes arrive at different times, whatever the replication method.
- Most replicated databases give at least **eventual consistency**: stop writing, wait "some unspecified length of time", and all reads return the same value. "A better name for eventual consistency may be convergence".
- It is very weak: nothing about when, and until then "reads could return anything or nothing"; you may not see your own write if the read goes to another replica.
- Hard for developers, since a variable in a single-threaded program doesn't behave so. Bugs are subtle and hard to find by testing; edge cases appear only under a fault (e.g. network interruption) or at high concurrency.
- Stronger models "don't come for free" (worse performance or less fault tolerance) but are easier to use correctly; knowing several lets you choose.
- Versus isolation levels: "mostly independent concerns". Isolation avoids races between concurrent transactions; distributed consistency "coordinating the state of replicas in the face of delays and faults".
- The areas are "deeply linked": first linearizability, "one of the strongest consistency models in common use"; then ordering, "particularly around causality and total ordering"; then atomic commit of distributed transactions, which "will finally lead us toward solutions for the consensus problem".

Linearizability

- In an eventually consistent database two replicas may give two answers. Wouldn't it be simpler if there appeared to be only one copy of the data?
- That is **linearizability** (atomic consistency, strong consistency, immediate consistency, external consistency): make the system appear as if there were only one copy and all operations atomic. Once a client completes a write, all clients reading must see it, never a stale cache or replica: "linearizability is a recency guarantee".

- Figure 9-1: Alice and Bob check the 2014 World Cup final. Alice refreshes, sees the winner and tells Bob; Bob reloads, hits a lagging replica and sees the game still on. Bob reloaded after hearing Alice, so expects a result at least as recent as hers; the stale result violates linearizability.

What Makes a System Linearizable?

- Figure 9-2: three clients read and write key x, called a **register**. Each bar runs from request sent to response received; a client only knows processing happened sometime in between. (Footnote i: a global clock is assumed, fine for analysis if the algorithm cannot access it.)
- Operations: read(x) ⇒ v and write(x, v) ⇒ r (r is ok or error).
- x starts at 0 and C writes 1 while A and B read. A's first read completes before the write begins, "so it must definitely return the old value 0"; A's last read begins after the write completed, so must return 1 (the write was processed between its start and end, the read likewise). Reads overlapping the write "might return either 0 or 1".
- Not sufficient: readers could see the value flip back and forth during a write, which is not a single copy. (Footnote ii: such a register is a regular register.)
- Added constraint (Figure 9-3): at some point between start and end of the write x atomically flips from 0 to 1; once one read returns 1, "all subsequent reads must also return the new value", even if the write is incomplete. A reads 1, B starts later so must read 1, as with Alice and Bob.
- Figure 9-4 adds cas(x, v old, v new) ⇒ r, atomic compare-and-set: if x equals v old it is set to v new, else left unchanged with an error. A vertical line in each bar marks when the operation takes effect; joined in order they must form a valid register history (every read returns the most recent write).
- B's read returns 1 though D's write of 0 and A's write of 1 were sent later: fine, the three are concurrent and B's request may have been delayed. Returning 1 before A got its ok is also fine; that ok was delayed. No transaction isolation is assumed (C reads 1 then 2); cas detects concurrent change (B's and C's succeed, D's fails).
- The requirement: the joining lines "always move forward in time (from left to right), never backward". This gives recency: once a value is written or read, later reads see it until overwritten. B's final read (shaded) is not linearizable: concurrent with C's cas from 2 to 4 it could return 2, but A already read 4, so B "is not allowed to read an older value than A".

Linearizability Versus Serializability

- Easily confused (both suggest a sequential order) but "two quite different guarantees".
- Serializability: isolation property of transactions over multiple objects; behaves as some serial order (literally serial, or concurrent while preventing conflicts by locking or aborting), possibly different from actual order.
- Linearizability: recency guarantee on reads and writes of one register. No grouping into transactions, so it doesn't prevent write skew unless extra measures such as materializing conflicts.
- Both together: **strict serializability** or **strong one-copy serializability** (strong-1SR). 2PL and actual serial execution are typically linearizable.
- Serializable snapshot isolation is not: "by design, it makes reads from a consistent snapshot", which excludes writes newer than the snapshot.

Relying on Linearizability

- Outdated sports scores do no harm, but "there a few areas in which linearizability is an important requirement for making a system work correctly".

Locking and leader election

- Single-leader replication needs exactly one leader (no split brain). One way is a lock acquired by starting nodes; the winner leads. "it must be linearizable: all nodes must agree which node owns the lock; otherwise it is useless".
- ZooKeeper and etcd use consensus to give linearizable operations fault-tolerantly. (Footnote iii: reads may be stale by default; etcd has quorum reads, ZooKeeper sync().) But "a linearizable storage service is the basic foundation for these coordination tasks".

Constraints and uniqueness guarantees

- Enforcing a uniqueness constraint as data is written (username, email, file path), so one of two concurrent creators gets an error, needs linearizability. It resembles a lock on the username, or an atomic compare-and-set of the username to a user ID if not taken.
- Likewise: bank balance never negative, stock not oversold, seat not double-booked; each needs "a single up-to-date value" all nodes agree on.
- Sometimes constraints can be loose (overbooked flight: compensate customers); then linearizability may not be needed (see "Timeliness and Integrity").
- But "a hard uniqueness constraint, such as the one you typically find in relational databases, requires linearizability". Foreign key or attribute constraints need not.

Cross-channel timing dependencies

- In Figure 9-1 Bob noticed staleness only because of an extra channel (Alice's voice). Computer analogue (Figure 9-5): a web server writes an uploaded photo to file storage and, once written, queues a resize instruction to the resizer.
- If storage is linearizable this works. If not, the queue (steps 3-4) may beat the storage's replication, so the resizer (step 5) may see an old version or nothing; with an old version, full-size and resized images become permanently inconsistent.
- Cause: two channels, storage and queue; without recency, races are possible, like replication versus audio in Figure 9-1.
- "Linearizability is not the only way of avoiding this race condition, but it's the simplest to understand." If you control the extra channel, use approaches like "Reading Your Own Writes", at the cost of complexity.

Implementing Linearizable Systems

- Simplest is a single copy, but it can't tolerate faults. So use replication; compare the Chapter 5 methods:
- Single-leader (potentially linearizable): reads from the leader or synchronously updated followers have "the potential to be linearizable", but not every one is, by design (snapshot isolation) or concurrency bugs. Reading from the leader assumes you know who it is; a delusional leader likely violates linearizability. Async failover may lose committed writes, violating durability and linearizability. (Footnote iv: a leader per partition doesn't affect this single-object guarantee; cross-partition transactions differ.)
- Consensus algorithms (linearizable) resemble single-leader replication but "contain measures to prevent split brain and stale replicas", so can implement linearizable storage safely (ZooKeeper, etcd).
- Multi-leader (not linearizable): "generally not linearizable"; asynchronous multi-node writes produce conflicts, "an artifact of the lack of a single copy of the data".
- Leaderless (probably not linearizable, Dynamo-style): claims of strong consistency via quorums ($w + r > n$) are "not quite true". Last write wins using time-of-day clocks (e.g. Cassandra) is almost certainly nonlinearizable (clock skew); sloppy quorums ruin any chance. Even strict quorums can fail:

Linearizability and quorums

- Figure 9-6: x=0, writer sends 1 to all three replicas ($n = 3$, $w = 3$). A reads a quorum of two ($r = 2$) and sees 1 on one node; B, starting after A completes, reads another quorum of two and gets 0 from both. $w + r > n$ holds yet it is not linearizable: Alice and Bob again.
- Only reads and writes can be so made; a linearizable compare-and-set "requires a consensus algorithm". It is "safest to assume that a leaderless system with Dynamo-style replication does not provide linearizability".

The Cost of Linearizability

- Multi-leader is often a good choice for multi-datacenter (Figure 9-7). If the link between datacenters breaks (each internally fine), multi-leader datacenters carry on and queue writes for later exchange.
- With single-leader, writes and linearizable reads must go to the leader's datacenter. Clients in the follower datacenter can't write or read linearizably, only read (possibly stale) from the follower. If the application needs linearizability it becomes unavailable there; clients reaching the leader are unaffected.

The CAP theorem

- “any linearizable database has this problem, no matter how it is implemented”, on any unreliable network, even within one datacenter. The trade-off: if linearizability is required and replicas are disconnected, some “cannot process requests while they are disconnected” and wait or error, becoming unavailable; if not required, each replica can process independently (e.g. multi-leader), staying available but not linearizable (footnote v: called CP and AP, a flawed classification best avoided).
- Applications not needing linearizability tolerate network problems better. This is the **CAP theorem**, named by Eric Brewer in 2000, though known to designers since the 1970s.

The Unhelpful CAP Theorem

- “pick 2 out of 3” is misleading: partitions are a kind of fault, so you have no choice about them. With a working network, a system can offer linearizability and total availability; during a fault choose. Better: “Consistent or Available when Partitioned”. A more reliable network makes the choice less often, but it is inevitable.
- Availability has contradictory definitions and the formal one differs from the usual; many “highly available” systems fail CAP’s definition. “CAP is best avoided.”
- Formally it is narrow: one consistency model (linearizability), one fault (network partitions; footnote vi explains the term), nothing on delays or dead nodes. So “little practical value for designing systems”; superseded by more precise results, “of mostly historical interest today”.

Linearizability and network delays

- CAP can’t justify dropping linearizability in multi-core RAM (cores have caches and store buffers, so no linearizability without a fence), as communication within a computer is reliable. The reason is “performance”, not fault tolerance, and likewise for many distributed databases. “Linearizability is slow—and this is true all the time, not only during a network fault.” Attiya and Welch prove response time is at least proportional to the uncertainty of network delays; no faster algorithm exists, though weaker models can be much faster. Chapter 12 discusses avoiding linearizability without sacrificing correctness.

Ordering Guarantees

- A linearizable register means operations execute in a well-defined order (the line in Figure 9-4). Ordering recurs throughout the book, hinting at a fundamental idea, with “deep connections between ordering, linearizability, and consensus”. It is more abstract but clarifies what systems can and cannot do.
- Recap: the leader in single-leader replication determines write order; without one, conflicts arise from concurrent operations. Serializability is behaving as if transactions ran in some sequential order. Timestamps and clocks (Chapter 8) are “another attempt to introduce order into a disorderly world”.

Ordering and Causality

- Ordering helps preserve **causality**: cause before effect, message sent before received, question before answer. Chains of causally dependent operations define the **causal order**, “what happened before what”.
- Earlier causality cases: Figure 5-5, an answer seen before its question (“a causal dependency between the question and the answer”); Figure 5-9, writes overtaking others so a replica sees an update to a nonexistent row; happened-before (“B might have known about A, or built upon A, or depended on A”; concurrent means “neither knew about the other”); a consistent snapshot is “consistent with causality” (an answer implies its question present), and read skew violates causality; write skew (Figure 7-8, going off call) depends causally on observing who is on call, which SSI detects by tracking causal dependencies; Alice and Bob and the image resizer are causality violations.
- A system obeying it is **causally consistent**. Snapshot isolation is: if you see data you also see data that causally precedes it.

The causal order is not a total order

- A **total order** compares any two elements (13 > 5). Sets are **partially ordered**: {a, b} and {b, c} are **incomparable**.

- Linearizability has a total order (Figure 9-4's timeline). Operations are concurrent if neither happened before the other: causally related events are ordered, concurrent ones incomparable. So causality defines a partial order.
- Hence a linearizable datastore has no concurrent operations: one timeline, one copy, each request atomic at a single point. Concurrency means the timeline branches and merges (Figure 5-14; arrows show causal dependencies), like Git histories.

Linearizability is stronger than causal consistency

- "linearizability implies causality": with several channels (Figure 9-5) causality is preserved automatically, without passing timestamps around. That makes it simple and appealing.
- But it can harm performance and availability, especially across networks with delays; so some systems abandoned it and became harder to work with.
- Middle ground: a system can be causally consistent without the hit, and "the CAP theorem does not apply". Causal consistency is "the strongest possible consistency model that does not slow down due to network delays, and remains available in the face of network failures".

Capturing causal dependencies

- Only key ideas. Concurrent operations may be processed in any order, causally ordered ones in that order on every replica: a replica must first process all causally preceding operations, else the later waits.
- We must describe a node's "knowledge": if it had seen X when issuing write Y, they may be causally related (like asking whether a CEO knew X when deciding Y).

Sequence Number Ordering

- Tracking all causal dependencies can be impractical (clients read lots before writing; unclear which reads matter; large overhead). Better: sequence numbers or timestamps, possibly from a **logical clock**, an algorithm generating numbers, typically with counters incremented per operation.
- They are compact and give a total order. We can make it "consistent with causality": if A causally precedes B, A gets a lower number; concurrent operations are ordered arbitrarily. It imposes more ordering than causality requires. (Footnote vii: an inconsistent total order, e.g. random UUIDs, is easy but useless.)
- With single-leader replication the log, numbered by a leader counter, is a total order consistent with causality; a follower applying it in order is causally consistent even if lagging.

Noncausal sequence number generators

- Without a single leader (multi-leader, leaderless, partitioned) methods: each node generates its own numbers (one odd, one even, or reserved bits holding a node identifier); a time-of-day clock timestamp (high resolution may totally order operations, as in last write wins); preallocated blocks (A takes 1 to 1,000, B 1,001 to 2,000). They scale better than one counter and give unique, approximately increasing numbers, but "the sequence numbers they generate are not consistent with causality": nodes process different numbers of operations, so with an odd and an even number you "cannot accurately tell which one causally happened first"; clock skew can give a causally later operation a lower timestamp (Figure 8-3); a later operation may get a number from a lower block. (Footnote viii: physical timestamps can be consistent with causality, as in Spanner, which waits out clock uncertainty; most clocks lack that metric.)

Lamport timestamps

- A simple causally consistent method, proposed in 1978 by Leslie Lamport. Each node has a unique ID and an operation counter; the timestamp is the pair (counter, node ID), unique even when counters match (Figure 9-8).
- Total order: greater counter wins; ties go to the greater node ID. No relation to physical time.
- Key idea: every node and client tracks the maximum counter seen and includes it on every request; a node receiving a greater maximum immediately raises its counter. In Figure 9-8 client A gets 5 from node 2, sends it to node 1 (counter 1), which jumps to 5, so the next operation is 6. Every causal dependency thus increases the timestamp.
- Unlike version vectors, which distinguish concurrent from causally dependent operations, Lamport timestamps always enforce total order and can't tell. Their advantage is compactness.

Timestamp ordering is not sufficient

- Not quite enough for common problems. Example: a username must uniquely identify an account. Seemingly, the lower timestamp wins.
- That works after the fact, once all creation operations are collected. But a node must decide now, not knowing whether another node is concurrently creating the name or what timestamp it will assign.
- Checking every other node means that if one is unreachable "this system would grind to a halt"; not the fault tolerance we need.
- The problem: the total order "only emerges after you have collected all of the operations". For uniqueness, "you also need to know when that order is finalized". That idea is captured by **total order broadcast**.

Total Order Broadcast

- On one CPU core a total order is execution order; distributed, agreement is tricky. Timestamp ordering is less powerful than single-leader replication (for uniqueness it tolerates no faults).
- Single-leader sequences operations on one leader core. The challenge: scaling beyond one leader's throughput, and failover. This problem is **total order broadcast** or atomic broadcast (footnote ix: "atomic" is confusing, unrelated to ACID; total order multicast is a synonym).

Scope of ordering guarantee

- Partitioned databases with a leader per partition often order only per partition, so no cross-partition guarantees (consistent snapshots, foreign keys). Total ordering across partitions "requires additional coordination".
- Two safety properties always: **reliable delivery** (no messages lost: if delivered to one node, delivered to all) and **totally ordered delivery** (same order at every node). These must hold despite node or network faults; during an interruption nothing is delivered, but the algorithm keeps retrying, still in correct order.

Using total order broadcast

- ZooKeeper and etcd implement it: a hint of a strong link to consensus.
- It is what replication needs: each message a write, every replica applying the same writes in the same order stays consistent. This is **state machine replication** (back in Chapter 11). Same for serializable transactions: messages are deterministic stored-procedure transactions.
- The order is fixed at delivery: no retroactive insertion before already-delivered messages, making it stronger than timestamp ordering.
- It is a way of creating a log (replication, transaction, write-ahead): delivering is appending, all nodes read the same sequence.
- Useful for a lock service with fencing tokens: each lock request is appended and numbered; the sequence number is the monotonic token (zxid in ZooKeeper).

Implementing linearizable storage using total order broadcast

- Same as linearizability? "Not quite, but there are close links". Total order broadcast is asynchronous: fixed order but no guarantee when, so recipients may lag. (Footnote x: a linearizable read-write register is formally easier; adding compare-and-set or increment-and-get makes it equivalent to consensus.)
- But linearizable storage can be built on it, e.g. unique usernames: a register per username, initially null, with atomic compare-and-set to the account ID provided the old value is null; of concurrent attempts one succeeds.
- Using the log: append a message tentatively claiming the username; read the log and wait for it to be delivered back; if the first message claiming the name is yours, commit the claim (perhaps by appending another message) and acknowledge the client, else abort. All nodes see entries in the same order, so agree which was first and whether to commit or abort. Serializable multi-object transactions can be built similarly.
- This gives linearizable writes, not reads: a store updated asynchronously from the log may be stale. Precisely it gives **sequential consistency** (**timeline consistency**), slightly weaker. Options for linearizable reads: sequence the read through the log, as the "position in the log thus defines the point in time at which the read happens" (etcd quorum reads); fetch the latest log position linearizably, wait for entries up to it, then read (ZooKeeper sync()); or read from a synchronously

updated replica (chain replication). (Footnote xi: acknowledging as soon as enqueued resembles x86 multi-core memory, neither linearizable nor sequentially consistent).

Implementing total order broadcast using linearizable storage

- Reverse: assume a linearizable register holding an integer with atomic increment-and-get (or compare-and-set). For each message, increment-and-get, attach the value as sequence number, send to all nodes (resending losses); recipients deliver consecutively.
- Unlike Lamport timestamps, the numbers have no gaps: a node that delivered 4 and receives 6 knows to wait for 5. That is the key difference between total order broadcast and timestamp ordering.
- Easy if nothing fails; hard is handling interruptions to that node and restoring the value after failure. If you think hard enough about linearizable sequence number generators, "you inevitably end up with a consensus algorithm". No coincidence: a linearizable compare-and-set (or increment-and-get) register and total order broadcast are both equivalent to consensus, "quite a profound and surprising insight". Now consensus head-on.

Distributed Transactions and Consensus

- Consensus is "one of the most important and fundamental problems in distributed computing": get several nodes to agree on something. "many broken systems have been built in the mistaken belief that this problem is easy to solve". Understanding "only gradually crystallized over the course of decades".
- Where agreement matters: (1) leader election: contested leadership under a network fault needs consensus to avoid a bad failover and split brain, where two leaders diverge and lose data. (2) Atomic commit: a transaction spanning nodes may fail on some and succeed on others; to keep atomicity all must agree to commit or abort. This is the **atomic commit problem** (footnote xii: it commits only if all vote to commit, while consensus may decide any proposed value; reducible to each other, nonblocking atomic commit harder).

The Impossibility of Consensus

- The FLP result (Fischer, Lynch, Paterson) proves no algorithm always reaches consensus if a node may crash. Yet algorithms exist: FLP is proved in the asynchronous model, "a very restrictive model" with deterministic algorithms and no clocks or timeouts. With timeouts or other crash suspicion (even if sometimes wrong), or even random numbers, consensus is solvable. So systems "can usually achieve consensus in practice".
- Plan: two-phase commit (2PC), the most common solution for atomic commit. 2PC "is a kind of consensus algorithm—but not a very good one"; learning from it leads to Zab (ZooKeeper) and Raft (etcd).

Atomic Commit and Two-Phase Commit (2PC)

- Outcome is a **commit** (all writes durable) or an **abort** (all rolled back), avoiding "half-finished results and half-updated state", notably for multi-object transactions and secondary indexes.

From single-node to distributed atomic commit

- Single node: the engine makes writes durable (typically in a write-ahead log), then appends a commit record; after a crash the transaction is committed if the record reached disk, else rolled back. So order matters: "first the data, then the commit record". The deciding moment is the disk finishing that record; a single device makes the commit atomic.
- Across nodes sending commit to all independently may succeed on some and fail on others (Figure 7-3): some detect a conflict and abort, some commit requests are lost and time out, some crash before the commit record is written and roll back. Most NoSQL stores don't support this.
- A committed transaction can't be retracted, so a node must commit only once certain all others will too. Commit must be irrevocable: committed data is visible and relied on (basis of read committed); retroactive abort would force dependents to revert.
- A compensating transaction can undo effects, but that is a separate transaction and cross-transaction correctness is the application's problem.

Introduction to two-phase commit

- **Two-phase commit** ensures "either all nodes commit or all nodes abort". Classic; used internally and exposed as XA transactions (e.g. Java Transaction API) or WS-AtomicTransaction. Instead of one commit request, commit/abort splits into two phases (Figure 9-9 shows success).

Don't confuse 2PC and 2PL

- 2PC gives atomic commit in a distributed database; 2PL gives serializable isolation. Best to treat them as separate and ignore "the unfortunate similarity in the names".
- 2PC adds a **coordinator** (transaction manager), often a library in the requesting process (e.g. Java EE container), or a separate service (Narayana, JOTM, BTM, MSDTC).
- The application reads and writes on multiple **participants**. To commit, the coordinator starts phase 1: a prepare request to each participant asking if it can commit, tracking responses: if all reply "yes", phase 2 sends commit; if any replies "no", it sends abort to all.
- Marriage analogy: the minister asks bride and groom individually, hears "I do" from both, pronounces them married and broadcasts it; if either doesn't say yes, the ceremony is aborted.

A system of promises

- Why does 2PC ensure atomicity if prepare and commit can be lost too? Steps: the application gets a globally unique transaction ID from the coordinator and starts a single-node transaction on each participant carrying it (any failure here lets anyone abort); at commit, the coordinator sends prepare with the ID (failure or timeout means abort); a participant must make sure it "can definitely commit the transaction under all circumstances" (data on disk, conflicts checked), and by saying "yes" it "promises to commit the transaction without error if requested", surrendering the right to abort; once all respond, the coordinator decides (commit only if all voted yes) and writes it to its disk log, the **commit point**; then it sends commit or abort and "must retry forever until it succeeds": "There is no more going back", and a recovering participant that voted yes must commit. Two "points of no return": a "yes" vote promises the participant will definitely be able to commit (though the coordinator may abort); and the coordinator's decision, once made, is irrevocable. These promises ensure atomicity; single-node commit lumps both into writing the commit record.
- Marriage: before "I do" you may say "No way!"; after, no retracting. If you faint and miss the pronouncement, ask the minister for the status of your global transaction ID, or wait for the commit retries.

Coordinator failure

- If prepare fails or times out, the coordinator aborts; if commit or abort requests fail, it retries indefinitely. A coordinator crash is less clear.
- Before sending prepare, participants can safely abort. After voting "yes" a participant can't abort unilaterally and must wait: **in doubt** or uncertain.
- Figure 9-10: the coordinator decided commit, database 2 got it, the coordinator crashed before telling database 1, which can't know. Timeout doesn't help: unilateral abort is inconsistent with database 2; unilateral commit is unsafe as another may have aborted. Participants could compare votes, but that is not part of 2PC.
- Only way to complete: wait for the coordinator to recover. So it writes its decision to a disk log before sending requests; on recovery transactions without a commit record are aborted. The commit point thus comes down to single-node atomic commit on the coordinator.

Three-phase commit

- 2PC is a **blocking** atomic commit protocol. In theory **nonblocking** is possible but hard in practice. Three-phase commit (3PC) assumes bounded delays and response times; with unbounded delay and process pauses (Chapter 8) it can't guarantee atomicity.
- Nonblocking commit needs a perfect failure detector; a timeout isn't one, as requests time out without crashes. "For this reason, 2PC continues to be used, despite the known problem with coordinator failure."

Distributed Transactions in Practice

- Mixed reputation: an important safety guarantee, but criticized for "killing performance, and promising more than they can deliver"; MySQL's are reported "over 10 times slower", partly from extra fsync and network round-trips.

- Examine rather than dismiss; two types are conflated. Database-internal: nodes of one distributed database (VoltDB, MySQL Cluster's NDB) can use any protocol and optimizations, so "can often work quite well". Heterogeneous: two or more technologies, "a lot more challenging".

Exactly-once message processing

- Heterogeneous transactions let a queue message be acknowledged if and only if the database transaction processing it committed, by committing both atomically; on failure both abort and the message may be redelivered, so it is "effectively processed exactly once".
- It needs all affected systems to share the atomic commit protocol: an email server without 2PC might send the email twice on retry. If all side effects roll back on abort, retry is safe. Chapter 11 returns to this.

XA transactions

- **X/Open XA**, introduced in 1991, standardises 2PC across heterogeneous technologies; supported by many relational databases and brokers (ActiveMQ, HornetQ, MSMQ, IBM MQ). XA "is not a network protocol—it is merely a C API for interfacing with a transaction coordinator"; in Java EE it is JTA, with JDBC and JMS drivers.
- The coordinator implements the XA API, often as a library in the application's process, tracking participants, collecting prepare responses and logging decisions on local disk.

Holding locks while in doubt

- The problem is locking: row-level exclusive locks to prevent dirty writes, plus shared locks on reads under 2PL. They can't be released until commit or abort (the shaded area in Figure 9-9), so in 2PC they're held through the in-doubt period: 20 minutes if the coordinator takes 20 to restart, forever (or until an administrator acts) if its log is lost.
- Others can't modify those rows, and may be blocked from reading; "large parts of your application" can become unavailable.

Recovering from coordinator failure

- In practice **orphaned** in-doubt transactions occur (e.g. log lost or corrupted by a bug); they can't be resolved automatically and sit holding locks. Rebooting doesn't help: correct 2PC preserves the locks across restarts. "It's a sticky situation."
- Only an administrator can resolve them manually: examine each participant, find if any committed or aborted, apply that to the rest, likely under stress during a serious outage.
- XA's emergency escape hatch is **heuristic decisions**: a participant unilaterally commits or aborts. "heuristic" is "a euphemism for probably breaking atomicity", for catastrophes only.

Limitations of distributed transactions

- XA solves a real problem but brings major operational ones. The key realization: the coordinator "is itself a kind of database", needing the same care. An unreplicated coordinator is a single point of failure (its failure blocks application servers on in-doubt locks), yet many are not highly available by default. A coordinator inside the application server makes it no longer stateless: its logs become "as important as the databases themselves". XA is "necessarily a lowest common denominator", unable to detect deadlocks across systems or work with SSI. Database-internal transactions are less limited, but 2PC needs every participant to respond, so distributed transactions have "a tendency of amplifying failures", against the goal of fault tolerance.
- Not hopeless: alternatives avoid heterogeneous distributed transactions (Chapters 11 and 12). First, finish consensus.

Fault-Tolerant Consensus

- Consensus decides which of several incompatible operations wins (last seat, theater seat, username). Formally: "one or more nodes may propose values, and the consensus algorithm decides on one of those values." In seat booking nodes propose the customer ID they serve.
- Properties: uniform agreement (no two nodes decide differently); integrity (no node decides twice); validity (a decided v was proposed by some node); termination (every non-crashing node eventually decides). Footnote xiii: this is uniform consensus, equivalent to regular consensus in asynchronous systems with unreliable failure detectors.

- Agreement and integrity are the core: “everyone decides on the same outcome, and once you have decided, you cannot change your mind”. Validity rules out trivial solutions like always deciding null.
- Without fault tolerance, a hardcoded “dictator” satisfies the first three; if it fails there are no decisions, exactly 2PC’s coordinator problem.
- Termination formalizes fault tolerance: make progress, surviving nodes decide. It is a liveness property; the other three are safety properties.
- Crashed nodes are assumed never to return (the datacenter destroyed by a landslide, node under “30 feet of mud”), so waiting for recovery fails termination: “2PC does not meet the requirements for termination”.
- Any algorithm needs at least a majority of correct nodes for termination (a quorum), so it assumes fewer than half crashed or unreachable. Most keep the safety properties even if a majority fail; an outage can stop processing but “cannot corrupt the consensus system by causing it to make invalid decisions”.
- Most assume no Byzantine faults; robustness is possible with fewer than one-third Byzantine nodes, not covered.

Consensus algorithms and total order broadcast

- Best known: Viewstamped Replication (VSR), Paxos, Raft, Zab; similar but “not the same”. Only common ideas are given; implementing one yourself “is probably not advisable—it’s hard”.
- Most decide a sequence of values rather than one, “which makes them total order broadcast algorithms”. Total order broadcast (messages exactly once, same order, all nodes) is equivalent to repeated rounds of consensus: each round nodes propose the next message and decide it. Agreement makes all deliver the same messages in the same order, integrity prevents duplicates, validity means messages are “not corrupted and not fabricated out of thin air”, termination means none is lost. VSR, Raft and Zab implement it directly, more efficiently; in Paxos this is Multi-Paxos.

Single-leader replication and consensus

- Single-leader replication looks like total order broadcast; why no consensus in Chapter 5? It depends on how the leader is chosen.
- A manually chosen leader is a “consensus algorithm” of the dictatorial variety: if it fails, writes are unavailable until operators reconfigure. Works well in practice but fails termination (needs humans).
- Automatic election and failover approaches fault-tolerant total order broadcast. But split brain means all must agree on the leader, so electing one needs consensus, which is total order broadcast, like single-leader replication, which needs a leader: “in order to elect a leader, we first need a leader.” How to break out?

Epoch numbering and quorums

- The protocols use a leader but don’t guarantee uniqueness; they guarantee it within each **epoch number** (ballot number in Paxos, view number in VSR, term number in Raft).
- When the leader is thought dead, a vote with an incremented epoch elects a new one, so epochs are totally ordered and monotonic. If leaders from two epochs conflict, the higher prevails.
- Before deciding, a leader must check no higher-epoch leader exists; it can’t trust its own judgment (recall “The Truth Is Defined by the Majority”). It collects votes from a quorum, typically but not always a majority: for every decision it sends the proposal and waits for a quorum in favour; a node votes yes only if unaware of a higher-epoch leader.
- Two rounds: choose a leader, then vote on its proposal. The key insight: the two quorums must overlap, so a successful proposal vote includes a node from the latest election. If no higher epoch is revealed, the leader knows it still leads and may decide.
- Similar to 2PC, but the coordinator is not elected; consensus needs only a majority whereas 2PC needs “yes” from every participant; and consensus defines a recovery process after a new leader. These are “key to the correctness and fault tolerance”.

Limitations of consensus

- A huge breakthrough: concrete safety properties where all else is uncertain, yet fault-tolerant while a majority works. They give total order broadcast, hence fault-tolerant linearizable atomic operations. But “the benefits come at a cost”:

- Voting is a kind of synchronous replication, whereas databases often use asynchronous replication, accepting loss on failover for performance.
- A strict majority: three nodes tolerate one failure, five tolerate two. If the network cuts nodes off, only the majority side progresses.
- Fixed membership mostly; dynamic membership extensions are “much less well understood”.
- Timeouts detect failures; with variable delays (especially geo-distributed) a transient issue causes false leader-failure beliefs. Safety is unharmed, but frequent elections give “terrible performance”.
- Sensitive to network problems: with one consistently unreliable link, Raft can bounce leadership between two nodes or force the leader to resign, never progressing; robust algorithms are an open research problem.

Membership and Coordination Services

- ZooKeeper and etcd are called “distributed key-value stores” or “coordination and configuration services”, with a database-like API. Why implement consensus; what makes them different?
- You rarely use ZooKeeper directly (not a general-purpose database); HBase, Hadoop YARN, OpenStack Nova and Kafka rely on it. It holds small in-memory-sized data replicated by fault-tolerant total order broadcast.
- Modeled after Google’s Chubby, it adds features (the book’s bullets): linearizable atomic operations, total ordering of operations, failure detection, change notifications.
- Atomic compare-and-set implements a lock, linearizably despite failures; usually a **lease** with expiry. A fencing token guards against process pauses; ZooKeeper gives each operation a zxid and cversion.
- Sessions with heartbeats: dead if heartbeats cease longer than the session timeout; locks can then be released automatically (**ephemeral nodes**). Clients can watch for changes, learning when another joins or fails, without polling.
- “Of these features, only the linearizable atomic operations really require consensus”, but the combination makes them useful.

Allocating work to nodes

- Fits choosing a leader among several instances (single-leader databases, job schedulers) and assigning partitions to nodes as they join or fail, using atomic operations, ephemeral nodes and notifications. “It’s not easy”, but better than implementing consensus from scratch, which “has a poor success record”.
- Voting among thousands of nodes would be inefficient; ZooKeeper runs on a fixed number (usually three or five), serving many clients: “outsourcing” consensus, ordering and failure detection to an external service.

Service discovery

- ZooKeeper, etcd and Consul also serve discovery: in clouds with VMs coming and going, services register endpoints in a service registry.
- It’s unclear discovery needs consensus. DNS uses caching for performance and availability; its reads are “absolutely not linearizable” and slight staleness is usually fine; reliable availability matters more.

Membership services

- A **membership service** determines which nodes are live (research since the 1980s, e.g. air traffic control). Per Chapter 8 failure detection is unreliable under unbounded delays, but coupled with consensus nodes can agree who is alive.
- A live node may be wrongly declared dead, but agreement is valuable: choosing the lowest-numbered member as leader fails if nodes disagree on membership.

Summary

- Linearizability makes replicated data appear as one copy with atomic operations, “like a variable in a single-threaded program”; appealing but slow, especially with large network delays.
- Causality orders by cause and effect; unlike a single total timeline it is weaker, history “branching and merging”. Causal consistency avoids linearizability’s coordination overhead and is much less sensitive to network problems.

- Even capturing causal order (Lamport timestamps) can't implement the unique-username check, since a node needs to know no other is concurrently registering the name. This led to **consensus**: deciding so all agree and the decision is irrevocable.
- Many problems reduce to consensus and are equivalent: compare-and-set register, atomic commit, message ordering, lock or lease acquisition, membership via failure detector, uniqueness constraints.
- Easy with a single deciding node, as in a single-leader database (linearizability, uniqueness, ordered log). But if the leader fails or is unreachable, no progress; three ways: wait for the leader to recover (many XA/JTA coordinators; fails termination, may block forever); manual failover by humans (many relational databases), consensus by "act of God", limited by human speed; or automatically choose a leader with a proven consensus algorithm. A leader still needs consensus for leadership changes: it "kicks the can down the road".
- Fault-tolerant consensus exists; ZooKeeper provides "outsourced" consensus, failure detection and membership. Not easy, but better than your own; "it is advisable to use something like ZooKeeper".
- Not every system needs consensus: leaderless and multi-leader systems typically don't; their conflicts follow, "but maybe that's okay: maybe we simply need to cope without linearizability and learn to work better with data that has branching and merging version histories".
- The theory is "incredibly valuable" for reasoning about what can be done, so the references are worth exploring. This ends Part II (replication, partitioning, transactions, failure models, consistency and consensus); Part III turns to more practical systems built from heterogeneous building blocks.

Footnotes

- Footnotes i-xiii are folded into the sections above.

References

- The chapter ends with references, not summarised here.

How this chapter builds the author's case

This chapter completes Part II. Chapters 5, 7 and 8 gave replication, transactions and the faults; here the author asks what guarantees can still be built. He descends from linearizability (simple but slow) to causal consistency (strongest model that stays available), shows by the username example that ordering is not enough, and finds total order broadcast, linearizable compare-and-set and consensus equivalent. 2PC is the flawed example; epochs and quorums fix it; the outcome is outsourcing consensus to ZooKeeper-like services. Forward links: Chapter 11 (state machine replication, exactly-once), Chapters 11 and 12 (alternatives to distributed transactions, avoiding linearizability), and Part III.

PART III

Derived Data

- The author's framing: Parts I and II "assembled from the ground up all the major considerations that go into a distributed database", but "this discussion assumed that there was only one database in the application. In reality, data systems are often more complex."
- Applications commonly combine "several different datastores, indexes, caches, analytics systems, etc." and need mechanisms for moving data between them. Integrating them "is one of the most important things that needs to be done in a nontrivial application", though vendors often overlook it.
- Key distinction: a system of record "holds the authoritative version of your data", with each fact represented exactly once; a derived data system results from taking existing data "and transforming or processing it in some way", so "If you lose derived data, you can recreate it from the original source". Caches, denormalized values, indexes and materialized views are derived.
- The distinction "depends not on the tool, but on how you use it in your application", and being clear about it "can bring clarity to an otherwise confusing system architecture". He calls this "a running theme throughout this part of the book".
- Chapters: Chapter 10 examines batch-oriented dataflow systems such as MapReduce; Chapter 11 applies those ideas "to data streams, which allow us to do the same kinds of things with lower delays"; Chapter 12 concludes by exploring how to "build reliable, scalable, and maintainable applications in the future".

The theme of the Part

- Integration of multiple data systems into one coherent architecture, organised by what is source of truth and what is derived from it.

How the chapters build the theme

- Batch (Chapter 10) gives the principles; streams (Chapter 11) reduce the delay; Chapter 12 looks ahead to applications built from these tools.

What the Part contributes to the book

- It closes the book by returning to the goals of Chapter 1 (reliable, scalable, maintainable) at the level of whole applications.

CHAPTER 10

Batch Processing

Place in the argument — why this chapter, why this title

Parts I and II dealt with requests and queries; this chapter opens the derived-data side by studying the offline, throughput-oriented kind of system, separated from services and streams. Stream processing follows in Chapter 11 because it "builds upon batch processing".

Place in the argument — why this chapter, why this title

Order of sections: Unix tools (small, familiar, with a philosophy worth borrowing), then MapReduce as the same ideas distributed, then the join and grouping algorithms, then what batch jobs output, then a comparison with MPP databases, then the alternatives. Each step answers the limit of the one before: Unix runs "only on a single machine"; MapReduce is laborious and slow for some work, so dataflow engines, graph models and high-level languages follow.

Chapter opening — untitled section before the first sub-heading in the book

- The first two parts dealt with requests and responses, with a human waiting, so "we pay a lot of attention to the response time". That is not the only way to build systems; the author distinguishes three types.
- **Service**: handles a request "as quickly as possible"; response time is usually the primary measure, availability often very important.
- **Batch processing system**: takes "a large amount of input data", runs a job, produces output data. Jobs take minutes to days, so no user waits; they are often scheduled periodically. The primary measure is usually throughput.
- **Stream processing**: between online and offline (near-real-time or nearline). It consumes inputs and produces outputs like batch, but "operates on events shortly after they happen", while a batch job operates on a fixed set of input data; hence lower latency. Left to Chapter 11.
- Batch is "an important building block in our quest to build reliable, scalable, and maintainable applications". MapReduce (2004) was, "perhaps over-enthusiastically", called "the algorithm that makes Google so massively scalable". It is "a fairly low-level programming model" but a major step in scale on commodity hardware. Its importance is declining, yet it gives "a clear picture of why and how batch processing is useful".
- Batch is very old: Hollerith punch card machines (1890 US Census) and the IBM card-sorting machines MapReduce resembles; "history has a tendency of repeating itself".
- Start with Unix tools, because "the ideas and lessons from Unix carry over to large-scale, heterogeneous distributed data systems".

Batch Processing with Unix Tools

- Example: a web server appends a line per request to a log (nginx default format: client IP, user, time, request, status, bytes sent, referrer, user agent). The sample line shows a request for /css/typography.css, status 200, 3,377 bytes; interpreting it needs the log format definition.

Simple Log Analysis

- Task: the five most popular pages. Pipeline: read the log; awk prints the seventh field (the URL); sort, so a URL requested n times appears n times in a row; uniq -c collapses adjacent repeats with a count; sort -r -n by count, largest first; head -n 5.
- It "will process gigabytes of log files in a matter of seconds" and is easy to modify (exclude CSS files; count client IPs by printing field 1). "Surprisingly many data analyses can be done in a few minutes" with awk, sed, grep, sort, uniq and xargs.

Chain of commands versus custom program

- A Ruby program does the same with a hash table of counters. Less concise but "fairly readable"; preference is "partly a matter of taste". The real difference is execution flow on a large file.

Sorting versus in-memory aggregation

- Ruby keeps an in-memory hash table; Unix has none and relies on sorting a list with repeats. Which is better "depends how many different URLs you have". The **working set** (memory needing random access) depends only on distinct URLs: a million hits on one URL still need one URL plus a counter. For most small to mid-sized sites that fits in about 1 GB, "even on a laptop".
- If the working set exceeds memory, sorting uses disks efficiently: sort chunks in memory, write segment files, merge them, as in SSTables and LSM-Trees; mergesort has sequential access. Optimizing for sequential I/O "was a recurring theme in Chapter 3"; "The same pattern reappears here".
- GNU sort spills to disk and parallelizes over cores automatically, so the simple chain scales; the bottleneck is likely the rate of reading input from disk.

The Unix Philosophy

- Easy log analysis by chaining "was in fact one of the key design ideas of Unix", "astonishingly relevant today".
- Doug McIlroy, inventor of pipes, wrote in 1964 of connecting programs like a "garden hose". The **Unix philosophy** (1978): make each program do one thing well; expect the output of every program to become the input to another, as yet unknown, program (no clutter, no stringently columnar or binary formats, no interactive input); build software to be tried early and throw away clumsy parts; prefer tools to unskilled help.
- Automation, rapid prototyping, incremental iteration, friendliness to experimentation, breaking projects into chunks "sounds remarkably like the Agile and DevOps movements of today". "Surprisingly little has changed in four decades."
- sort does one thing well, arguably better than most standard libraries (which do not spill to disk or use threads), yet is "barely useful in isolation"; it is powerful in combination, such as with uniq. What enables this composability?

A uniform interface

- Any output can feed any input only if all use the same interface. In Unix it is a file (file descriptor): "just an ordered sequence of bytes", shared by real files, process channels (stdin, stdout), device drivers, TCP sockets. "It's easy to take this for granted, but it's actually quite remarkable."
- Footnote: URLs and HTTP are another uniform interface, unlike bulletin board systems, which could not link into each other.
- By convention bytes are ASCII text with newline-separated records (an arbitrary choice; the ASCII record separator would arguably be better, but standardization lets programs interoperate). Field parsing is vaguer (whitespace, tabs, CSV).
- The text interface "mostly works, but it's not exactly beautiful": {print $7} is unreadable where {print $request_url} would be ideal. The author returns to this later.
- Still remarkable: you "can't easily pipe the contents of your email account and your online shopping history through a custom analysis tool into a spreadsheet and post the results to a social network or a wiki". Even databases with the same data model make export hard: "lack of integration leads to Balkanization of data".

Separation of logic and wiring

- Tools use stdin and stdout; by default keyboard and screen, but redirectable to files, and pipes attach one stdout to another's stdin with a small in-memory buffer rather than writing the whole stream to disk.
- Best if a program ignores file paths, so the shell user wires input and output and the program "doesn't know or care" where they go: loose coupling, late binding, inversion of control. "Separating the input/output wiring from the program logic makes it easier to compose small tools into bigger systems." You can plug in your own programs (e.g. IP address to country code).
- Limits: multiple inputs or outputs are "possible but tricky"; you can't pipe output into a network connection (footnote: except via netcat or curl; BSD sockets broke the everything-is-a-file convention, Plan 9 and Inferno keep it). If a program opens files, subprocesses or connections itself, shell wiring flexibility is reduced.

Transparency and experimentation

- Inputs are normally immutable, so you can run commands as often as you want "without damaging the input". You can end the pipeline at any point, pipe into less and inspect: "great for debugging". You can write one stage's output to a file and use it as input to the next stage, restarting the later stage "without rerunning the entire pipeline". Though "blunt, simple tools compared to a query optimizer of a relational database", they remain "amazingly useful, especially for experimentation".
- Their "biggest limitation" is a single machine, "and that's where tools like Hadoop come in".

MapReduce and Distributed Filesystems

- MapReduce is "a bit like Unix tools, but distributed across potentially thousands of machines": "blunt, brute-force, but surprisingly effective". A job is like a Unix process: inputs in, outputs out, normally no modification of input and no side effects besides output, written once and sequentially.
- Instead of stdin/stdout it uses files on a distributed filesystem: in Hadoop, HDFS, an open source reimplementation of the Google File System (footnote: unlike object stores such as S3, HDFS can schedule tasks near a file copy). HDFS is the running example.
- HDFS is shared-nothing, unlike shared-disk NAS and SAN (centralized appliance, special hardware): it needs "no special hardware, only computers connected by a conventional datacenter network". A daemon per machine serves its files; a central NameNode tracks which blocks are where, giving one big filesystem.
- Blocks are replicated (copies as in Chapter 5, or erasure coding such as Reed–Solomon with lower storage overhead), like RAID but over an ordinary network. HDFS scales to tens of thousands of machines and hundreds of petabytes, at much lower cost than dedicated storage appliances.

MapReduce Job Execution

- The pattern mirrors the log analysis (in the example the mapper is awk '{print $7}', URL as key and empty value; the reducer is uniq -c, combining adjacent equal keys without much state in memory): break files into records; mapper extracts key and value; sort by key; reducer processes sorted pairs. Steps 2 (map) and 4 (reduce) are your code; step 1 is the input format parser; step 3 is implicit: "the output from the mapper is always sorted before it is given to the reducer".
- The **mapper** is called once per record, may emit any number of pairs (including none), and keeps no state between records. The framework collects values per key and calls the **reducer** with an iterator over them.
- A second sorting stage (the ranking step) is a second job fed by the first. So "the role of the mapper is to prepare the data by putting it into a form that is suitable for sorting, and the role of the reducer is to process the data that has been sorted".

Distributed execution of MapReduce

- Main difference from Unix pipelines: parallelism across machines "without you having to write code to explicitly handle the parallelism", since mapper and reducer do not know where input comes from or output goes. Mappers and reducers are Java classes in Hadoop and JavaScript functions in MongoDB and CouchDB.
- Figure 10-1 shows a Hadoop job. Parallelism rests on partitioning (Chapter 6): each input file or block is a partition for a separate map task (m1, m2, m3), typically hundreds of megabytes. The scheduler tries to run each mapper on a machine holding a replica: "putting the computation near the data" saves network copying and increases locality.

Figure 10-1. A MapReduce job with three mappers and three reducers.

- The framework copies the code (e.g. JAR files) to the machine, then feeds records to the mapper. The reduce side is also partitioned; the job author sets the number of reduce tasks, and a hash of the key sends all pairs with the same key to the same reducer.
- The dataset is too large to sort on one machine, so sorting is staged: each map task partitions its output by reducer and writes sorted files to local disk (as in SSTables and LSM-Trees); reducers fetch these when the mapper finishes. Partitioning, sorting and copying is the **shuffle**, "a confusing term" since "there is no randomness in MapReduce".
- The reduce task merges fetched files preserving order, so equal keys are adjacent. The reducer gets a key and an iterator over its records (possibly not all fitting in memory), applies arbitrary logic, and writes output to the distributed filesystem (one local copy plus replicas).

MapReduce workflows

- One job is limited: it can count page views per URL but not find the most popular, which needs a second round of sorting. Jobs are therefore chained into **workflows**.
- Hadoop has no workflow support; chaining is implicit by directory name, so the framework sees "two independent jobs". Chaining is thus less like Unix pipes and more like commands writing temporary files; advantages and disadvantages come in "Materialization of Intermediate State".
- Output is valid only if a job completes (partial output of a failed job is discarded), so dependent jobs wait; workflow schedulers (Oozie, Azkaban, Luigi, Airflow, Pinball) manage this. Recommendation workflows of 50 to 100 jobs are common, and many teams read each other's output, so tool support matters. Pig, Hive, Cascading, Crunch and FlumeJava wire multi-stage workflows automatically.

Reduce-Side Joins and Grouping

- A join is needed whenever code accesses both sides of an association (foreign key, document reference, graph edge); denormalization reduces the need "but generally not remove it entirely".
- Databases use indexes for few records. MapReduce has no indexes "in the usual sense": it reads all input, a **full table scan**, "outrageously expensive" for a few records but reasonable for analytic aggregates over many, especially parallelized. A batch join means "resolving all occurrences of some association within a dataset", such as all users at once.

Example: analysis of user activity events

- Figure 10-2: a log of activity events (clickstream data; the fact table of a star schema) joined with a user database (a dimension). Example: which pages are popular with which age groups. Events hold only the user ID; embedding profiles would be too wasteful.

Figure 10-2. A join between a log of user activity events and a database of user profiles.

- Simplest way: query the remote user database per event. Poor: throughput limited by round-trip time, caching depends on data distribution, parallel queries "could easily overwhelm the database".
- For throughput "the computation must be (as much as possible) local to one machine"; remote random access is too slow, and a remote database also makes the job nondeterministic since its data may change.
- Better: copy the user database into the same distributed filesystem as the event log, then use MapReduce to bring relevant records together.

Sort-merge joins

- Mappers extract the user ID as key: one set over the events (event as value), another over the user database (date of birth as value), as in Figure 10-3.

Figure 10-3. A reduce-side sort-merge join on user ID. If the input datasets are partitioned into multiple files, each could be processed with multiple mappers in parallel.

- After partitioning and sorting, records with the same user ID are adjacent. A **secondary sort** can place the user record first, then events by timestamp. The reducer, called once per user ID, holds the date of birth in a local variable and emits (viewed-url, viewer-age-in-years) pairs; later jobs compute age distributions per URL.
- It keeps one user record in memory and makes no network requests. This is a **sort-merge join**: mapper output sorted by key, reducers merge the sorted lists of both sides.

Bringing related data together in the same place

- Mappers and sorting ensure one reducer call has all the data for one user ID, so the reducer can be "a fairly simple, single-threaded piece of code" with high throughput and low memory.
- Mappers "send messages" to reducers; the key acts like a destination address. This separates the physical network communication from application logic, unlike databases where a fetch sits deep in application code. It also shields code from partial failures: MapReduce "transparently retries failed tasks without affecting the application logic".

GROUP BY

- The same pattern serves grouping by key (as SQL GROUP BY) followed by aggregation: counting records per group (COUNT(*)), adding up a field (SUM), or picking the top k records by a ranking

function: mappers emit the grouping key; partitioning and sorting do the rest, so “grouping and joining look quite similar when implemented on top of MapReduce”.
- **Sessionization** collates all of a user’s events to see the sequence of actions, e.g. for A/B testing or judging marketing. Events are scattered across web server logs, so use a session cookie or user ID as grouping key, distributing different users across partitions.

Handling skew

- The pattern breaks when one key has huge data: celebrities with millions of followers, **linchpin objects** or **hot keys**. One reducer gets far more records (**skew**, **hot spots**), and since a job completes only when all reducers do, later jobs wait for the slowest.
- Pig’s skewed join samples to find hot keys, then sends their records to a random one of several reducers and replicates the other input’s records for that key to all of them: parallelism at the cost of replication. Crunch’s sharded join is similar but takes hot keys explicitly (compare “Skewed Workloads and Relieving Hot Spots”). Hive’s takes them in table metadata, stores their records separately and uses a map-side join for them.
- For grouping by a hot key, aggregate in two stages: random reducers produce compact partial aggregates, then a second job combines them.

Map-Side Joins

- The above are **reduce-side joins**; mappers only prepare data. Advantage: no assumptions about the input. Downside: sorting, copying and merging “can be quite expensive”, with data possibly written to disk several times.
- With assumptions about input, a **map-side join** is faster: “no reducers and no sorting”; each mapper reads one input block and writes one output file.

Broadcast hash joins

- For a large dataset joined with one small enough to fit in memory in each mapper: load the small input into a hash table, then scan the large input, looking up each key. (Footnote: assumes one entry per key; in general several may exist and all matches are output.)
- One map task per block of the large input, each loading the entire small input: a **broadcast hash join** (“broadcast” since the small input goes to all partitions, “hash” for the table). Supported by Pig, Hive, Cascading, Crunch, Impala. Alternative: a read-only local-disk index, almost as fast via the page cache.

Partitioned hash joins

- If both inputs are partitioned the same way (e.g. by last decimal digit of user ID, 10 partitions), mapper 3 loads only users ending in 3 and scans only events for them: less data per hash table. This needs the same number of partitions, key and hash function, a reasonable assumption if prior MapReduce jobs produced the inputs. Hive calls it a **bucketed map join**.

Map-side merge joins

- If inputs are also sorted on the same key, size no longer matters: the mapper reads both incrementally in key order and matches records, as a reducer would. Likely prior jobs produced this form; a separate map-only job may be appropriate if the data is needed for other purposes too.

MapReduce workflows with map-side joins

- A reduce-side join’s output is partitioned and sorted by the join key; a map-side join’s output is partitioned and sorted like the large input. Map-side joins assume more, so you must know the physical layout, not just “the encoding format and the name of the directory”: the number of partitions and the keys. In Hadoop this metadata is kept in HCatalog and the Hive metastore.

The Output of Batch Workflows

- Why run all these jobs? OLTP queries look up a few records by key; analytic queries scan many and output reports for analysts or managers. Batch is “not transaction processing, nor is it analytics”: closer to analytics in scanning large input, but its output is “often not a report, but some other kind of structure”.

Building search indexes

- Google's original MapReduce use was building search indexes, a workflow of 5 to 10 jobs (later abandoned); Hadoop still builds Lucene/Solr indexes. A Lucene index is a term dictionary mapping keywords to postings lists; real indexes need more (ranking, misspellings, synonyms) "but the principle holds".
- For a fixed document set: mappers partition documents, each reducer builds its partition's index, files go to the distributed filesystem. Document-partitioned indexes "parallelizes very well"; being read-only, the files are immutable.
- If documents change, rerun the whole workflow and replace index files wholesale: expensive for few changes, but easy to reason about: "documents in, indexes out". Or build incrementally, as Lucene does with new segments merged in the background (Chapter 3; Chapter 11 returns to it).

Key-value stores as batch process output

- Other outputs: classifiers (spam filters, anomaly detection, image recognition) and recommendation systems (people you may know, related products), often a database queried by user or product ID from a web application separate from Hadoop. How does it get there?
- Writing record by record from a mapper or reducer through the database client "is a bad idea for several reasons": a network request per record is "orders of magnitude slower" than batch throughput, even with client batching; many parallel tasks writing to one database can overwhelm it and hurt its queries; and MapReduce's all-or-nothing guarantee (output as if every task ran exactly once, none if the job fails) does not cover external writes, whose side effects make partial results visible and bring the complexities of task attempts and speculative execution.
- Much better: "build a brand-new database inside the batch job", written as immutable files in the output directory and bulk-loaded into read-only servers (Voldemort, Terrapin, ElephantDB, HBase bulk loading). Extracting a key and sorting is already much of index building; read-only stores need no WAL.
- Voldemort serves the old files while new ones copy over, then "atomically switches over"; if something goes wrong it can switch back, since the old files are immutable.

Philosophy of batch process outputs

- Unix is explicit about dataflow: input unchanged, previous output completely replaced, no other side effects, so you can rerun "without messing up the state of your system". MapReduce follows it: immutable inputs and no side effects such as writing to external databases give good performance and make jobs "much easier to maintain": a bug can be fixed by rolling back the code and rerunning, or by switching back to old output kept in another directory, whereas read-write databases keep bad data after a code rollback ("human fault tolerance"), so feature development proceeds faster, minimizing irreversibility as Agile favors; failed tasks are retried automatically, safe only because inputs are immutable and failed output is discarded (a bug just fails the job after a few attempts); the same files can feed other jobs, including monitoring jobs comparing output to the previous run; and logic is separated from wiring, so one team builds a job that does one thing well while others decide where and when to run it.
- Difference: Unix tools assume untyped text and must parse a lot ({print $7}); Hadoop eliminates "low-value syntactic conversions" with structured formats like Avro and Parquet (schema-based encoding with schema evolution, Chapter 4).

Comparing Hadoop to Distributed Databases

- Hadoop is "somewhat like a distributed version of Unix": HDFS the filesystem, MapReduce a "quirky implementation of a Unix process" that always runs sort between map and reduce.
- MapReduce was "in some sense—not at all new": **MPP** (massively parallel processing) databases (Gamma, Teradata, Tandem NonStop SQL) had these algorithms over a decade earlier. The biggest difference: MPP focuses on parallel analytic SQL; MapReduce plus a distributed filesystem is "something much more like a general-purpose operating system that can run arbitrary programs".

Diversity of storage

- Databases require a data model; files are byte sequences in any model or encoding (records, text, images, sensor readings, genome sequences). "To put it bluntly", Hadoop allowed dumping data indiscriminately and figuring out processing later; MPP databases demand up-front modeling before import.

- A purist prefers modeling for data quality, but in practice quick availability, even raw, "is often more valuable than trying to decide on the ideal data model up front". As with a data warehouse, centralizing data enables joins across disparate datasets; schema design slows collection ("data lake", "enterprise data hub").
- Interpretation becomes the consumer's problem (schema-on-read), an advantage when teams differ or several views are needed: the **sushi principle**, "raw data is better". Hadoop is thus used for ETL: dump raw data, clean and transform it by job, import into an MPP warehouse; modeling happens "in a separate step, decoupled from the data collection".

Diversity of processing models

- MPP databases are monolithic, tightly integrated (storage, planning, scheduling, execution), giving very good performance for designed queries, and SQL is accessible to tools like Tableau. But machine learning, recommendation, full-text relevance ranking and image analysis "inevitably require writing code, not just queries".
- MapReduce lets engineers run their own code at scale; Hive built SQL on it, but other processes do not fit SQL. MapReduce was later found too limiting and slow for some work, so more models were built on Hadoop: "even more different models were needed!" Openness made this feasible, unlike a monolithic MPP database.
- All models run on one shared cluster over the same files, so there is no importing into specialized systems and it is easier to experiment. The ecosystem includes HBase (OLTP) and Impala (MPP analytic); neither uses MapReduce, both use HDFS, and they coexist.

Designing for frequent faults

- Batch is less fault-sensitive than online systems: failure does not immediately affect users and jobs can rerun. MPP databases abort and retry the whole query on a node crash ("the cost of retrying is not too great" for queries of seconds or minutes) and keep data in memory. MapReduce retries at task granularity and is "very eager to write data to disk", partly for fault tolerance and partly because data is assumed too big for memory. This suits large jobs likely to see a task failure.
- But machine failures are "not very frequent", so why the overhead? Look at the original environment: Google's mixed-use datacenters run online services and batch jobs together, with container-enforced allocations and priorities; higher priority preempts lower and costs more. This allows overcommitting low-priority resources, but MapReduce jobs "pick up the scraps under the table" and risk preemption.
- At Google, a task running an hour has about a 5% risk of termination, over an order of magnitude above hardware-related failures; with 100 tasks of 10 minutes, the risk that at least one is terminated exceeds 50%. So MapReduce tolerates termination "not because the hardware is particularly unreliable" but because "the freedom to arbitrarily terminate processes enables better resource utilization in a computing cluster".
- Open source schedulers use preemption less. Where tasks are rarely terminated, MapReduce's design "make less sense", leading to alternatives.

Beyond MapReduce

- MapReduce is "just one among many possible programming models". It was studied because it is "a useful learning tool", simple "in the sense of being able to understand what it is doing, not in the sense of being easy to use": raw APIs are laborious (you implement join algorithms from scratch). The execution model has problems no abstraction fixes. It is very robust, handling huge data on unreliable multi-tenant systems "albeit slowly", while other tools are sometimes "orders of magnitude faster". Chapter 11's streams are another way to speed up batch.

Materialization of Intermediate State

- Jobs touch the world only via directories, with an external scheduler. That fits a dataset published by name for many jobs, including other teams' (loose coupling, as in "Separation of logic and wiring"). But often output is used by one job of the same team: **intermediate state**, plentiful in 50 to 100 job workflows. Writing it out is **materialization** (compute eagerly and write, as in materialized views). Unix pipes stream through a small buffer instead.
- Downsides against pipes: a job can start only when all tasks of preceding jobs finish, whereas piped processes start together, and a few straggler tasks (from skew or varying load) slow the whole workflow; mappers are often redundant, re-reading a reducer's output, and could be part of the

previous reducer; and replicating intermediate state across nodes is “often overkill” for temporary data.

Dataflow engines

- New engines (Spark, Tez, Flink) “handle an entire workflow as one job, rather than breaking it up into independent subjobs”; modeling data flow through stages makes them **dataflow engines**. Like MapReduce they call user functions on one record at a time, partition inputs, and copy outputs over the network; but these **operators** need not alternate map and reduce. Connection options: repartition and sort by key as in the shuffle (enabling sort-merge joins and grouping); partition without sorting (order is irrelevant for partitioned hash joins, since building the hash table randomizes it); or send one operator’s output to all partitions of a join (broadcast hash join). Advantages: sorting only where required; no unnecessary map tasks; explicit dependencies let the scheduler make locality optimizations (e.g. exchange via a shared memory buffer); intermediate state kept in memory or local disk needs less I/O than replicated HDFS; operators start as soon as input is ready; JVM processes can be reused. They descend from Dryad and Nephele.
- They run the same computations, usually significantly faster; Pig, Hive or Cascading workflows switch from MapReduce to Tez or Spark “with a simple configuration change”. Tez is a thin library using the YARN shuffle service; Spark and Flink are big frameworks with their own network layer, scheduler and APIs.

Fault tolerance

- MapReduce is easy: intermediate state is durable, so a failed task restarts elsewhere. Spark, Flink and Tez avoid writing it to HDFS, so lost state is recomputed from other available data, which requires tracking how data was computed: Spark’s resilient distributed dataset (RDD) tracks ancestry; Flink checkpoints operator state.
- Whether the computation is **deterministic** matters, since lost data may already have reached downstream operators; with nondeterminism the solution is normally to kill downstream operators and rerun. Better make operators deterministic, but nondeterminism creeps in (hash table iteration order, random numbers, system clock, external data); remove it, e.g. with a fixed seed.
- Recomputing is not always right: if intermediate data is much smaller than the source or computation is CPU-intensive, “it is probably cheaper to materialize the intermediate data to files than to recompute it”.

Discussion of materialization

- MapReduce is like temporary files; dataflow engines are like pipes. Flink is built around pipelined execution, passing output on without waiting for input to complete. Sorting must consume all input first (the last record might be the lowest key), but many parts can be pipelined.
- Final output still goes to the distributed filesystem; the gain is skipping intermediate state.

Graphs and Iterative Processing

- Chapter 2’s graphs were for OLTP-style queries; here the goal is offline processing of an entire graph (recommendation engines, ranking). PageRank estimates a page’s popularity from pages linking to it and is used in ordering search results.

Note

- Dataflow engines arrange operators as a DAG, “not the same as graph processing”: there the data flow is a graph while the data is relational-style tuples. “Another unfortunate naming confusion!”
- Graph algorithms traverse one edge at a time, repeating until a condition is met, as in Figure 2-6 (a **transitive closure**). Storing a graph as files is possible, but “repeating until done” cannot be expressed in plain MapReduce’s single pass, so it is iterated: a scheduler runs a batch process for one step, checks the completion condition (no more edges, or change below a threshold), and runs another round if unfinished. This is very inefficient, since MapReduce reads the entire input and writes a whole new output even if little of the graph changed.

The Pregel processing model

- The bulk synchronous parallel (BSP) model, implemented by Giraph, GraphX and Gelly, is also the **Pregel model**. As mappers “send a message” to a reducer call, a vertex sends messages to another, typically along edges. Each iteration calls a function per vertex with its messages, like a reducer, but a

vertex keeps state in memory between iterations, so only new messages are processed and idle parts need no work.
- Like the actor model, but vertex state and messages are fault-tolerant and durable, and communication proceeds in fixed rounds; actors normally lack that guarantee.

Fault tolerance

- Message passing only allows batching and less waiting; the only wait is between iterations. Pregel implementations guarantee messages are processed exactly once at the destination in the next iteration despite an unreliable network, recovering transparently like MapReduce.
- It checkpoints all vertex state at iteration ends. After failure, roll back everything to the last checkpoint, or, if the algorithm is deterministic and messages are logged, recover only the lost partition.

Parallel execution

- A vertex sends to a vertex ID; the framework partitions the graph and routes messages. Ideally communicating vertices are colocated, but that is hard, so graphs are often partitioned by arbitrary vertex ID. Hence heavy cross-machine communication, with intermediate state often bigger than the graph.
- So if the graph fits in memory on one computer, a single-machine algorithm will quite likely outperform a distributed one; if it fits on disk, GraphChi is viable; only if too big is a distributed approach like Pregel "unavoidable".

High-Level APIs and Languages

- Engines have matured to many petabytes on over 10,000 machines; with physical operation "more or less solved", attention turned to the programming model, efficiency and breadth. Hive, Pig, Cascading and Crunch exist because hand-written MapReduce is laborious, and moved to Tez without rewriting. Spark and Flink have dataflow APIs inspired by FlumeJava, with relational building blocks (join, group, filter, aggregate) implemented by this chapter's algorithms.
- They also allow interactive shell use, helpful for exploration and "reminiscent of the Unix philosophy", and improve machine-level efficiency.

The move toward declarative query languages

- With joins as relational operators, the framework can pick a join algorithm automatically; Hive, Spark and Flink have cost-based optimizers that can reorder joins to minimize intermediate state. It is "nice not to have to understand and remember all the various join algorithms" (see "Query Languages for Data").
- Yet MapReduce is built on callbacks free to run arbitrary code, giving access to libraries for parsing, language and image analysis, statistics. This "has long distinguished" it from MPP databases, whose user-defined functions are "often cumbersome".
- Dataflow engines still gain from declarative features: simple filter or field-selection callbacks add CPU overhead per record; declared, the optimizer can read only required columns (column-oriented storage) and use vectorized execution in a tight, cache-friendly loop; Spark generates JVM bytecode, Impala uses LLVM. So batch frameworks "begin to look more like MPP databases (and can achieve comparable performance)" while keeping flexibility.

Specialization for different domains

- Recurring patterns deserve reusable building blocks beyond business intelligence: machine learning algorithms (Mahout on MapReduce, Spark, Flink; MADlib in Apache HAWQ), spatial algorithms such as k-nearest neighbors, and approximate search for genome analysis. As batch gains declarative operators and MPP databases become programmable, the two look alike: "in the end, they are all just systems for storing and processing data".

Summary

- Unix tools' philosophy carries into MapReduce and dataflow engines: inputs immutable, outputs meant to become input to another (as yet unknown) program, complex problems solved by composing small tools that "do one thing well". Unix's uniform interface is files and pipes; MapReduce's is a distributed filesystem; dataflow engines add pipe-like transport, but initial input and final output are still usually HDFS.

- Two main problems for frameworks (partitioning and fault tolerance). MapReduce partitions mappers by input block, then repartitions, sorts and merges output into reducer partitions to bring related data together; dataflow engines avoid sorting unless required. MapReduce writes to disk often, easing task recovery but slowing fault-free runs; dataflow engines keep more in memory and recompute more, and deterministic operators reduce recomputation.
- Join algorithms (also used in MPP databases and engines): sort-merge (mappers extract the key, equal keys meet in one reducer call); broadcast hash (small input in a hash table per mapper); partitioned hash (same key, hash function and partition count, per partition).
- The restricted model (stateless callbacks without external side effects) lets frameworks retry tasks safely, discard failed output, and let only one successful task's output show: the final output is "the same as if no faults had occurred", far stronger than online services that write to databases.
- A batch job reads input and produces derived output without modifying the input. Input is **bounded**, so a job knows when it has finished. Next, stream processing, with **unbounded** input and never-complete jobs: similar in some respects, but that "also changes a lot about how we build systems".

Footnotes

- Footnote i: cat is unnecessary but makes the pipeline's linear shape more apparent. The others are noted where they apply.

References

- The chapter ends with references (not summarised).

How this chapter builds the author's case

The chapter opens the batch half of derived data by carrying Unix ideas (uniform interface, logic separate from wiring, immutable inputs, replaceable outputs) into distributed form. It leans on earlier chapters: sequential I/O and SSTables (Chapter 3), joins and graphs (Chapter 2), replication (Chapter 5), partitioning and hot spots (Chapter 6), Avro and Parquet (Chapter 4). MapReduce's costs motivate dataflow engines, Pregel and declarative APIs, converging with MPP databases. It ends on bounded input, the assumption Chapter 11 drops.

CHAPTER 11

Stream Processing

Place in the argument — why this chapter, why this title

the title names the unbounded counterpart of Chapter 10. The chapter opens by saying Chapter 10 rested on one assumption, bounded input, which real data violates. The author announces the route: how streams are represented, stored and transmitted (Transmitting Event Streams); then the relationship between streams and databases (Databases and Streams); then "approaches and tools for processing those streams continually" (Processing Streams). Each needs the previous: first a stream and a way to move it, then the insight that database writes are a stream, then what to do with streams (time, joins, fault tolerance: Chapter 10 topics redone for data that never ends).

Place in the argument — why this chapter, why this title

The Gall epigraph frames the method: extend familiar batch ideas to streams.

Chapter opening — untitled section before the first sub-heading in the book

- Chapter 10: batch processing reads files and produces new output files; the output is "derived data", recreatable by rerunning. Unspoken assumption: input is bounded, "of a known and finite size". MapReduce sorting must read all input first, since the last record might need to be the first output.
- Real data is unbounded: users will produce more tomorrow, and "this process never ends". Batch must therefore cut data into chunks of fixed duration (a day, an hour). Daily batches reflect changes a day later, "too slow for many impatient users". Shorten the slices, or go continuous, "abandoning the fixed time slices entirely and simply processing every event as it happens". That is stream processing.
- A stream is data incrementally made available over time (Unix stdin/stdout, TCP, audio and video). The chapter treats event streams as the unbounded counterpart to batch data: how they are represented, stored and transmitted; streams and databases; then processing them continually.

Transmitting Event Streams

- The streaming counterpart of a record is the **event**: "a small, self-contained, immutable object containing the details of something that happened at some point in time", usually with a time-of-day timestamp. It may come from a user (viewing a page, a purchase) or a machine (sensor, CPU metric); each web server log line is one. It is encoded as text, JSON or binary (Chapter 4) so it can be stored or sent over the network.
- As a file is written once and read by many jobs, an event is generated once by a **producer** (publisher, sender) and processed by potentially multiple **consumers** (subscribers, recipients). Related events are grouped into a **topic** or **stream**, as a filename groups records.
- A datastore with polling consumers would do in principle (that is a batch process), but at low delay "The more often you poll, the lower the percentage of requests that return new events". Better that consumers are notified. Database triggers are "very limited" and "somewhat of an afterthought in database design", so specialized tools arose. This sets up the next section.

Messaging Systems

- A **messaging system**: a producer sends a message containing the event, pushed to consumers. Pipes and TCP link exactly one sender to one recipient; messaging systems allow many producers and consumers on a topic (publish/subscribe). There is "no one right answer for all purposes", so the author differentiates systems with two questions. (1) What if producers send faster than consumers process? "Broadly speaking, there are three options": drop messages, buffer in a queue, or apply backpressure ("flow control"); Unix pipes and TCP block the sender when a small fixed-size buffer fills. If buffering, what happens as the queue grows: a crash when it no longer fits in memory, or disk writes and their cost to performance? (2) What if nodes crash or go offline: are messages lost? As with databases, durability may need disk writes and/or replication, "which has a cost"; if you can sometimes lose messages, "you can probably get higher throughput and lower latency on the same hardware".

- Message loss: acceptable for periodic sensor readings (an updated value follows), though many dropped messages may go unnoticed; unacceptable when counting events, since "every lost message means incorrect counters".
- Batch gives a strong guarantee (failed tasks retried, partial output discarded, so output is as if no failures occurred). He will later show similar guarantees for streams.

Direct messaging from producers to consumers

- Some systems use direct network communication without intermediaries: UDP multicast (widely used in finance for stock market feeds, where low latency matters; application-level protocols recover lost packets, so the producer must remember what it sent); brokerless libraries such as ZeroMQ and nanomsg (publish/subscribe over TCP or IP multicast); StatsD and Brubeck (unreliable UDP for metrics, so counters are "at best approximate"); and webhooks, where a callback URL of one service is registered with another, which requests it whenever an event occurs. They "generally require the application code to be aware of the possibility of message loss" and assume producers and consumers are constantly online: an offline consumer may miss messages, and a producer that crashes loses its retry buffer.

Message brokers

- A **message broker** (**message queue**) is "essentially a kind of database that is optimized for handling message streams": a server to which producers and consumers connect as clients. Centralizing data tolerates clients that come and go; durability moves to the broker (memory only, or disk depending on configuration). Slow consumers generally mean unbounded queueing rather than dropping or backpressure.
- So consumers are **asynchronous**: the producer waits only for the broker to confirm buffering, and delivery happens at an undetermined later time, sometimes significantly later with a backlog.

Message brokers compared to databases

- Some brokers join two-phase commit via XA or JTA, making them quite similar to databases, though with important practical differences. Databases usually keep data until explicitly deleted, whereas most brokers delete a message once delivered, so they "are not suitable for long-term data storage". Because they delete quickly they assume a small working set (short queues); if many messages are buffered, each takes longer and throughput may degrade. Databases offer secondary indexes and search, brokers subscription to a subset of topics matching a pattern: both let a client select the data it wants. A database query gives a point-in-time snapshot and the client is not told when the result becomes outdated; brokers support no arbitrary queries but do notify clients when data changes. This is the traditional view, encapsulated in JMS and AMQP (RabbitMQ, ActiveMQ and others).

Multiple consumers

- Two patterns for several consumers on one topic (Figure 11-1), combinable. Load balancing: each message goes to one consumer, to share work when messages are expensive to process. Fan-out: each message goes to all consumers, letting independent consumers "tune in" to the same broadcast, "the streaming equivalent of having several different batch jobs that read the same input file".

Figure 11-1. (a) Load balancing: sharing the work of consuming a topic among consumers; (b) fan-out: delivering each message to multiple consumers.

- Combination: two consumer groups each receive all messages, but within a group only one node gets each message.

Acknowledgments and redelivery

- Consumers may crash mid-processing, so brokers use **acknowledgments**: the client tells the broker when it has finished so the message can be removed. With no acknowledgment before a close or timeout, the broker redelivers to another consumer. The message may have been fully processed with the acknowledgment lost; handling that needs an atomic commit protocol.
- Redelivery with load balancing reorders messages (Figure 11-2): consumer 2 crashes on m3 while consumer 1 handles m4; m3 is redelivered to consumer 1, which processes m4, m3, m5.

Figure 11-2. Consumer 2 crashes while processing m3, so it is redelivered to consumer 1 at a later time.

- Even with JMS and AMQP ordering, load balancing plus redelivery "inevitably leads to messages being reordered". Remedy: a separate queue per consumer. Harmless for independent messages, but important with causal dependencies, met later in the chapter.

Partitioned Logs

- Network calls leave no permanent trace, and even disk-writing brokers delete messages once delivered, built around a "transient messaging mindset". Databases and filesystems expect everything written to be recorded until explicitly deleted.
- This matters for derived data. Batch can be rerun and experimented with since input is read-only; with AMQP/JMS, receiving is destructive, so you cannot rerun a consumer and get the same result. A new consumer sees only messages sent after it registered.
- The motivating question: why not combine the durable storage of databases with the low-latency notification of messaging? That is the idea behind **log-based message brokers**.

Using logs for message storage

- A log is "simply an append-only sequence of records on disk" (Chapters 3 and 5). As a broker, producers append and consumers read sequentially, waiting for a notification at the end; tail -f works this way.
- To scale beyond one disk, partition the log (Chapter 6); each partition is an independent log, possibly on a different machine; a topic is a group of partitions carrying the same type of message (Figure 11-3). The broker gives each message in a partition a monotonically increasing **offset**; as partitions are append-only, messages within one are totally ordered, but "There is no ordering guarantee across different partitions."

Figure 11-3. Producers send messages by appending them to a topic-partition file, and consumers read these files sequentially.

- Shows offsets numbering messages in partition files.
- Kafka, Kinesis Streams and DistributedLog work like this. Despite writing everything to disk they reach millions of messages per second by partitioning, with fault tolerance by replication.

Logs compared to traditional messaging

- Fan-out is trivial, since reading does not delete. For load balancing the broker assigns entire partitions to nodes in a consumer group, each consuming its partitions sequentially, single-threaded. This coarse-grained approach has downsides: the number of nodes sharing a topic is at most the number of partitions, and one slow message holds up later ones in its partition (head-of-line blocking). Footnote i: single-threaded processing is generally preferable and parallelism comes from more partitions.
- Verdict: for expensive per-message processing, per-message parallelism and unimportant ordering, JMS/AMQP style is preferable; with high throughput, fast messages and important ordering, "the log-based approach works very well".

Consumer offsets

- Sequential consumption makes progress easy to tell: messages below the consumer's offset are processed, above it unseen. The broker only periodically records offsets, with less bookkeeping and more batching and pipelining, raising throughput.
- This resembles the log sequence number of single-leader replication, letting a follower resume without skipping writes: "the message broker behaves like a leader database, and the consumer like a follower". On failure, another node resumes at the last recorded offset; messages processed after it are processed twice, a problem treated later in the chapter.

Disk space usage

- The log is divided into segments; old ones are deleted or archived. A consumer too slow, with its offset in a deleted segment, misses messages: the log is a bounded buffer, a "circular buffer or ring buffer", but on disk and so large.
- A 6 TB drive with 150 MB/s writes fills in about 11 hours; in practice deployments keep days or weeks. Throughput stays constant whatever the retention, unlike in-memory brokers that slow down when they spill to disk.

When consumers cannot keep up with producers

- Of the three choices (drop, buffer, backpressure), the log is "buffering with a large but fixed-size buffer". You can monitor how far a consumer lags and alert, leaving time for a human to fix it before it misses messages.
- Only the lagging consumer is affected, "a big operational advantage": you can consume a production log for testing or debugging without disrupting production. A stopped consumer leaves only its offset, unlike traditional brokers where queues of dead consumers pile up and take memory.

Replaying old messages

- AMQP/JMS acknowledging is destructive; log consumption is "more like reading from a file", read-only. Only the offset moves, and it is under the consumer's control: start a copy at yesterday's offsets, write elsewhere, reprocess, vary the code.
- This makes log-based messaging like batch, with derived data separated from input by a repeatable transformation, enabling experimentation and easier recovery from bugs, "a good tool for integrating dataflows within an organization".

Databases and Streams

- Brokers and databases were separate categories, but log-based brokers took ideas from databases; now the reverse. A write to a database is an event too, so the connection is "quite fundamental".
- A replication log is a stream of write events from the leader. State machine replication (Total Order Broadcast): if every event is a write and every replica processes the same events in the same order, deterministically, replicas end in the same state. "It's just another case of event streams!"
- Plan: a problem in heterogeneous systems, then solving it with event-stream ideas.

Keeping Systems in Sync

- No single system serves all needs, so applications combine an OLTP database, a cache, a full-text index and a warehouse, each with its own copy in its own representation, which must be kept in sync. Warehouses use ETL, a batch process. If dumps are too slow, one alternative is **dual writes**: the application writes to each system itself.
- Problem 1, a race (Figure 11-4): clients 1 and 2 set X to A and B; the database sees A then B, the search index B then A. They are "permanently inconsistent with each other, even though no error occurred", and without version vectors you will not notice.

Figure 11-4. In the database, X is first set to A and then to B, while at the search index the writes arrive in the opposite order.

- Shows the interleaved writes.
- Problem 2: one write fails while the other succeeds, a fault-tolerance problem; making both succeed or fail is atomic commit, "expensive to solve".
- With one single-leader database the leader fixes order and state machine replication works. In Figure 11-4 the database and index each may have a leader but neither follows the other. Better: one leader (the database) and the index its follower. "But is this possible in practice?"

Change Data Capture

- Replication logs were long "an internal implementation detail of the database, not a public API"; many databases had no documented way to get the change log. Now interest grows in **change data capture** (CDC): "the process of observing all data changes written to a database and extracting them in a form in which they can be replicated to other systems", best as a stream. Apply the changes in order to a search index and it matches the database; the index and other derived systems are just consumers of the change stream (Figure 11-5).

Figure 11-5. Taking data in the order it was written to one database, and applying the changes to other systems in the same order.

- Shows one database's writes applied in order elsewhere.

Implementing change data capture

- The consumers are **derived data systems** (Part III introduction): other views onto the system of record. "Essentially, change data capture makes one database the leader … and turns the others into followers." A log-based broker suits transport since it preserves order (avoiding Figure 11-2).

- Triggers writing to a changelog table "tend to be fragile and have significant performance overheads"; parsing the replication log is more robust though schema changes are hard. Used at scale by LinkedIn Databus, Facebook Wormhole, Yahoo! Sherpa, among others.
- CDC is usually asynchronous: a slow consumer barely affects the system of record, but all replication lag issues apply.

Initial snapshot

- Replaying the whole log would rebuild the database, but keeping all changes takes too much disk and replay too long, so the log is truncated. A new full-text index needs a full copy, so start from a consistent snapshot tied to a known log position or offset. Some CDC tools integrate this; others leave it manual.

Log compaction

- Alternative to repeated snapshots: **log compaction** (hash indexes, Figure 3-2). The engine finds records with the same key, discards duplicates and keeps the most recent update; a null update (a **tombstone**) deletes. Space depends only on current contents, "not the number of writes".
- Same for brokers and CDC when every change has a primary key: start a new consumer at offset 0 of the compacted topic and scan; the log has "the most recent value for every key", a full copy without a new snapshot. Kafka supports it; this lets the broker serve as "durable storage, not just for transient messaging".

API support for change streams

- Databases increasingly offer change streams "as a first-class interface, rather than the typical retrofitted and reverse-engineered CDC efforts" (RethinkDB, Firebase, CouchDB, Meteor). VoltDB exports a stream through a table that transactions insert into but that cannot be queried. Kafka Connect integrates CDC tools with Kafka.

Event Sourcing

- **Event sourcing**, from the domain-driven design (DDD) community, also stores all changes to application state as a log of change events; "The biggest difference is that event sourcing applies the idea at a different level of abstraction". In CDC the application uses the database mutably, updating and deleting at will, and the log is extracted at a low level (e.g. by parsing the replication log), so write order matches actual order, avoiding the race of Figure 11-4, and the application need not know CDC is occurring. In event sourcing the application logic is built explicitly on immutable events in an append-only event store, where updates and deletes are discouraged or prohibited; events reflect things that happened at the application level, not low-level state changes.
- It is "a powerful technique for data modeling": record user actions as immutable events, not their effect on mutable state. It eases evolution, debugging and guards against bugs. "student cancelled their course enrollment" states intent neutrally, whereas the side effects embed assumptions; a new feature (offering the place to the next on the waiting list) chains off the existing event.

Deriving current state from the event log

- Users expect current state (the cart contents), not the history, so the log must be transformed into application state. The transform should be "deterministic" so replay gives the same state. Log compaction differs: a CDC update event typically holds the entire new record, so the latest event per key determines the value and compaction can discard earlier ones. Event sourcing events express the intent of a user action, not the mechanics of the state update, so later events typically do not override earlier ones, you need the full history, and compaction is "not possible in the same way". Snapshots are only "a performance optimization"; the intention is to store all raw events forever and reprocess on demand (see Limitations of immutability).

Commands and events

- A request first arrives as a **command**, which may fail an integrity condition, so it is validated first; if accepted it becomes an event, "durable and immutable" (username, seat reservation). The event is then a **fact**: a later cancellation is a separate event.
- A consumer cannot reject an event, so validation must be synchronous, before it becomes an event (e.g. a serializable transaction). Alternative: a tentative reservation event followed by a confirmation event, allowing asynchronous validation.

State, Streams, and Immutability

- Immutability of batch input is “also what makes event sourcing and change data capture so powerful”. Yet databases store mutable current state. Resolution: state is the result of the events that mutated it (seats from reservations, balance from credits and debits, a response time graph from request times); “there was always a sequence of events”.
- “mutable state and an append-only log of immutable events do not contradict each other: they are two sides of the same coin” The **changelog** is the evolution of state over time. State is the integral of the event stream over time, the change stream its derivative (Figure 11-6): “a useful starting point for thinking about data”.

Figure 11-6. The relationship between the current application state and an event stream.

- Shows state and event stream as integral and derivative.
- Treat the durable log as system of record and mutable state as derived. Pat Helland:

> *“The truth is the log. The database is a cache of a subset of the log.”*

- Log compaction bridges the two: it keeps only the latest version of each record.

Advantages of immutable events

- Accountants have long used an append-only **ledger**; accounts are derived by adding up transactions. Mistakes are corrected by compensating entries, never erased, for auditing; published figures are corrected in the next period. “This process is entirely normal in accounting”
- Auditability helps other systems: after buggy code writes bad data, an append-only log makes diagnosis and recovery much easier. Events also capture more than state: adding an item to a cart and removing it cancels out for fulfilment, but analytics may want the fact.

Deriving several views from the same event log

- Separating mutable state from the log yields several read-oriented views, like multiple stream consumers. A new feature can run a new view alongside existing systems, easier than a complicated schema migration.
- Complexities of schema design, indexing and storage engines come from query patterns, so separating the written form from the read form gives flexibility: **command query responsibility segregation** (CQRS). “The traditional approach to database and schema design is based on the fallacy that data must be written in the same form as it will be queried.” Normalization debates become largely irrelevant, and denormalized views stay consistent via translation.

Concurrency control

- Biggest downside: consumers are usually asynchronous, so a user may not see their own write in a derived view. Fixes: update the view synchronously with the append, via one storage system or a distributed transaction, or use total order broadcast linearizable storage.
- Upside: a self-contained event needs a single write, “easy to make atomic”, replacing multi-object transactions. If log and state are partitioned alike, a single-threaded consumer needs no concurrency control: “The log removes the nondeterminism of concurrency by defining a serial order of events in a partition” Events touching several partitions are deferred to Chapter 12.

Limitations of immutability

- Many non-event-sourced systems rely on immutability (multi-version snapshots, Git, Mercurial, Fossil). Whether forever is feasible depends on churn: mostly-adding workloads are easy; heavy updates and deletes on a small dataset make history grow prohibitively, fragment, and make compaction and garbage collection crucial.
- Data may need deletion for privacy regulation, data protection law or leaks. Appending a deletion event is not enough; you want to “rewrite history and pretend that the data was never written in the first place” (Datomic excision, Fossil shunning). “Truly deleting data is surprisingly hard”; deletion is more “making it harder to retrieve the data” than “making it impossible to retrieve the data”. Yet you sometimes must try (Legislation and self-regulation).

Processing Streams

- Having covered sources and transport, the author turns to processing, of three options, focusing on the third. (1) Write the events to a database, cache, search index or similar store for other clients to query: a good way to keep a database in sync with the rest of the system, especially if the consumer

is its only writer, and the streaming equivalent of the output of batch workflows. (2) Push the events to users (email alerts, push notifications, a real-time dashboard), a human being the ultimate consumer. (3) Process input streams into output streams, possibly through a pipeline of stages ending at option 1 or 2: producing derived streams. Such code is an **operator** or **job**, like Unix processes and MapReduce jobs: input read-only, output append-only. Partitioning, parallelization, mapping and filtering are as in Chapter 10.

- "The one crucial difference to batch jobs is that a stream never ends": sorting is meaningless, so sort-merge joins are out, and a long-running job cannot restart from the beginning after a crash.

Uses of Stream Processing

- Long used for monitoring and alerting: fraud detection (card usage patterns unexpectedly changed), trading systems (execute trades by rules on price changes), manufacturing (machine status, quickly identifying malfunctions), military and intelligence (tracking a potential aggressor). These need "quite sophisticated pattern matching and correlations"; other uses are compared below.

Complex event processing

- **Complex event processing** (CEP), from the 1990s, searches for event patterns, like a regular expression over events. Patterns are written declaratively (SQL-like or GUI); the engine keeps a state machine and emits a **complex event** on a match. Roles are reversed compared to databases: queries are stored long-term and events flow past them.

Stream analytics

- Analytics favors aggregations and statistical metrics over many events rather than specific sequences; the boundary is blurry. Examples: event rates, rolling averages, comparison with earlier intervals (e.g. last week) to detect trends. Typically over fixed intervals, e.g. the average rate over 5 minutes and 99th percentile response time; smoothing irrelevant fluctuation. The interval is a **window** (see Reasoning About Time).
- Probabilistic algorithms (Bloom filters, HyperLogLog, percentile estimators) save memory, but "there is nothing inherently approximate about stream processing, and probabilistic algorithms are merely an optimization". Frameworks include Storm, Spark Streaming, Flink, Samza and Kafka Streams.

Maintaining materialized views

- Keeping derived systems updated from a change stream is maintaining **materialized views**; in event sourcing application state is one too. Unlike analytics, it needs events over "a window that stretches all the way back to the beginning of time", minus obsolete ones removed by log compaction. Some analytics frameworks assume limited windows; Samza and Kafka Streams support this via log compaction.

Search on streams

- Searching individual events by complex criteria (full-text): media monitoring, real estate alerts, Elasticsearch's percolator. Search "turns the processing on its head": queries are stored and documents run past them, as in CEP. Test every document against every query, or index the queries too.

Message passing and RPC

- Actor-style message passing is not normally thought of as stream processing: actor frameworks mainly manage concurrency and distributed execution of communicating modules, whereas stream processing is "primarily a data management technique"; actor communication is often ephemeral and one-to-one, event logs are durable and multi-subscriber; actors can communicate arbitrarily (even cyclically), stream processors are usually acyclic pipelines where each stream derives from a well-defined set of inputs. There is crossover though: Storm's distributed RPC interleaves user queries with event streams. Actor frameworks often do not guarantee delivery after crashes, so they are not fault-tolerant without retry logic.

Reasoning About Time

- Windows like "the average over the last five minutes" seem clear but "the notion is surprisingly tricky". A batch process uses timestamps embedded in events, never its own clock, which also makes it deterministic. Many stream frameworks use the local system clock (**processing time**): simple, and fine when delay is negligible, but it "breaks down if there is any significant processing lag".

Event time versus processing time

- Delays come from queueing, network faults, contention, restarts, reprocessing. They also reorder: server B's event may reach the broker before A's though it occurred later. Star Wars analogy: release order differs from narrative order; episode number is the event timestamp, viewing date the processing time.
- "Confusing event time and processing time leads to bad data": after a redeploy, the backlog looks like a spike in request rate though the real rate was steady (Figure 11-7).

Figure 11-7. Windowing by processing time introduces artifacts due to variations in processing rate.

- Shows the artifact spike from backlog processing.

Knowing when you're ready

- With event-time windows, you can never be sure all events for a window have arrived. A timeout can fail because events are buffered elsewhere; **straggler** events after completion must be handled, with two options: ignore the stragglers, probably a small percentage, tracking the dropped count as a metric and alerting if significant; or publish a **correction**, an updated window value, perhaps retracting earlier output. A special message can promise no more events earlier than t, but with several producers each needs tracking and adding or removing producers is trickier.

Whose clock are you using, anyway?

- Offline mobile apps send events hours or days later, appearing as "extremely delayed stragglers". The device clock is right but untrusted; the server's is accurate but less meaningful. Fix: log three timestamps: when the event occurred (device clock), when it was sent (device clock), when the server received it (server clock); subtracting the second from the third estimates the clock offset (assuming negligible network delay and a stable offset), applied to the event timestamp.
- "This problem is not unique to stream processing"; batch has it too, just less noticeable.

Types of windows

- Four types: tumbling, hopping, sliding, session. **Tumbling**: fixed length, each event in exactly one window (round timestamps down to the minute). **Hopping**: fixed length but overlapping for smoothing (aggregate adjacent tumbling windows). **Sliding**: events within an interval of each other (keep a time-sorted buffer). **Session**: no fixed duration; groups a user's events close in time and ends after inactivity; sessionization is common in website analytics.

Stream Joins

- Joins are as needed on streams as in batch pipelines, but harder since new events appear anytime. Three types: stream-stream, stream-table and table-table.

Stream-stream join (window join)

- Example: click-through rate per URL, joining search and click events by session ID. The click may never come, may come days later, or even before the search; choose a window (e.g. one hour). Embedding the search in the click event is not equivalent, as it loses unclicked searches.
- The processor keeps state (the last hour's events indexed by session ID); on a match it emits which result was clicked; on expiry, which were not clicked.

Stream-table join (stream enrichment)

- Like the batch join of activity events and user profiles (Figure 10-2), done continuously: **enriching** each event with profile data. A remote lookup is slow and risks overloading the database, so load a local copy (like map-side hash joins). Since the database changes, keep the copy current via CDC, subscribing to the profile changelog: a join of two streams.
- Biggest difference from stream-stream: the changelog join uses a window back to the "beginning of time", with newer versions overwriting older.

Table-table join (materialized view maintenance)

- Twitter timelines (Describing Load): iterating over all followed people is too expensive, so keep a per-user cache "inbox". Maintaining it needs streams of tweets (sending, deleting) and follow relationships, plus a follower set per user. The cache needs four kinds of processing: a new tweet by u goes into every follower's timeline; a deleted tweet is removed from all timelines; when u1 follows

u2, u2's recent tweets are added to u1's timeline; on unfollow they are removed. Equivalent to a materialized view of a join query:

```
SELECT follows.follower_id AS timeline_id, array_agg(tweets.* ORDER BY tweets.timestamp DESC) FROM tweets JOIN follows ON follows.followee_id = tweets.sender_id GROUP BY follows.follower_id
```

- Timelines are "effectively a cache of the result of this query, updated every time the underlying tables change" (footnote iii: the product rule $(u \cdot v)' = u'v + uv'$; each change of tweets joins current followers and vice versa).

Time-dependence of joins

- All three keep state from one input and query it on messages from the other; the order of state-maintaining events matters (follow then unfollow), but order across streams or partitions is not guaranteed. "if state changes over time, and you join with some state, what point in time do you use for the join" Example: the tax rate at the time of sale.
- Undetermined ordering makes the join nondeterministic, so reruns may differ. Warehouses call this a **slowly changing dimension** (SCD), addressed by giving each version a unique identifier (the invoice stores the tax rate's identifier). That makes the join deterministic but prevents log compaction, as all versions must stay.

Fault Tolerance

- Batch retries failed tasks because input is immutable, outputs go to separate files and become visible only on success; the output is as if every record were processed once: **exactly-once semantics**, "though effectively-once would be a more descriptive term". In streams you cannot wait for a task to finish, since a stream is infinite.

Microbatching and checkpointing

- **Microbatching**: small blocks treated as miniature batches, about one second, a compromise between scheduling overhead and delay; it implies a tumbling window of the batch size by processing time. Flink instead writes rolling checkpoints of state, triggered by barriers in the stream; a crashed operator restarts from the last checkpoint, discarding later output.
- Both give exactly-once within the framework, but once output leaves it (databases, external brokers, emails) a restart makes the side effect happen twice, so they are "not sufficient".

Atomic commit revisited

- To appear exactly-once, all outputs and side effects (downstream messages, database writes, operator state, acknowledgments and offset moves) must take effect "if and only if the processing is successful": atomic commit again. Chapter 9 criticized XA, but in restricted environments it can be efficient (Google Cloud Dataflow, VoltDB), by keeping state changes and messaging inside the framework and amortizing overhead over several messages per transaction.

Idempotence

- Alternative to transactions: an **idempotent** operation "has the same effect as if you performed it only once". Setting a key is idempotent; incrementing is not. Metadata can fix this: write the Kafka offset of the message along with the value so you can tell an update was applied. Assumptions: same messages replayed in the same order, deterministic processing, no concurrent updates; fencing on failover. "Despite all those caveats, idempotent operations can be an effective way of achieving exactly-once semantics with only a small overhead."

Rebuilding state after a failure

- Windowed aggregates and join tables need recoverable state. Keep it remote and replicated (slow per message), or local and replicated periodically: Flink snapshots to HDFS; Samza and Kafka Streams write changes to a log-compacted Kafka topic, like CDC.
- "There is no universally ideal trade-off for all situations"; it depends on infrastructure performance and may shift as technology evolves.

Summary

- Stream processing is like batch (Chapter 10) but continuous on unbounded streams; brokers and event logs are the streaming filesystem.

- Two broker types: AMQP/JMS-style and log-based. AMQP/JMS style: messages assigned individually, acknowledged and deleted; suited as asynchronous RPC (a task queue) where order is unimportant and no rereading is needed. Log-based: partition messages go to one consumer in order; parallelism from partitioning; consumers checkpoint offsets; messages retained on disk for rereading. The latter resembles replication logs and log-structured storage and suits systems producing derived state or streams.
- Sources: user activity, sensors, data feeds; also database writes, captured implicitly by CDC or explicitly by event sourcing; log compaction retains a full copy. This integrates systems: keep indexes, caches and analytics up to date from the change log, and build fresh views by consuming the log from the beginning. The same facilities underlie joins and fault tolerance.
- Purposes: event patterns (CEP), windowed aggregations (stream analytics), materialized views. Time: processing time versus event timestamps, and stragglers.
- Joins (stream-stream, stream-table, table-table): stream-stream (related events within a window, perhaps a self-join), stream-table (activity events plus a changelog-fed local copy, giving enriched events), table-table (two changelogs, giving changes to the materialized join).
- Fault tolerance: discard partial output of failed tasks, but a stream cannot discard everything, so use finer-grained recovery based on “microbatching, checkpointing, transactions, or idempotent writes”.

Footnotes

- i: see Logs compared to traditional messaging. ii: credits the Star Wars analogy. iii: see Table-table join.

References

- The chapter ends with references (not summarised).

How this chapter builds the author’s case

it lifts Chapter 10’s batch ideas (immutable input, derived data, repeatable transformation) to unbounded data, so that “message brokers and event logs serve as the streaming equivalent of a filesystem”. It leans on Chapters 3 (logs, compaction), 5 (replication logs), 6 (partitioning) and 9 (atomic commit). The log is the hinge: log-based brokers give durable, ordered, replayable delivery; CDC and event sourcing turn database writes into the same kind of log; and “The database is a cache of a subset of the log.” justifies deriving indexes, caches and views from it, with joins and exactly-once recovery built on the same replay. Forward, events touching several state partitions are deferred to Chapter 12, and deletion to Legislation and self-regulation.

CHAPTER 12

The Future of Data Systems

Place in the argument — why this chapter, why this title

the title turns from the present to the future. The author says “this book has been mostly about describing things as they are at present”, and here shifts to “how things should be”, with ideas he believes “may fundamentally improve the ways we design and build applications”; so he uses the first person and invites disagreement.

Place in the argument — why this chapter, why this title

Why it is last: the Chapter 1 goal was applications that are “reliable, scalable, and maintainable”; here those themes are brought together to ask how to build applications that are “robust, correct, evolvable, and ultimately beneficial to humanity”. The sections follow that goal: Data Integration, Unbundling Databases (a design for combining tools), Aiming for Correctness, Doing the Right Thing, Summary.

Chapter opening — untitled section before the first sub-heading in the book

- The ideas are his opinions and speculation; first person is used; readers may disagree, but he hopes to give a starting point for productive discussion and “bring some clarity to concepts that are often confused”.

Data Integration

- “I want to store some data and look it up again later” has no one right solution. An implementation must pick one approach, and “trying to do everything in one piece of software almost guarantees that the implementation will be poor.”
- So the best tool depends on circumstances: “Every piece of software, even a so-called “general-purpose” database, is designed for a particular usage pattern.” The first challenge is mapping products to the circumstances they fit; vendors are reluctant to say which workloads suit them poorly, so the earlier chapters supply questions to read between the lines.

Combining Specialized Tools by Deriving Data

- Example: an OLTP database plus a full-text search index. Some databases (PostgreSQL) have full-text indexing, enough for simple applications; sophisticated search needs specialist information retrieval tools. Search indexes are generally not suitable as a durable system of record, so two tools are needed.

Reasoning about dataflows

- When copies of the same data live in several systems for different access patterns, be clear about inputs and outputs: where is data written first, and which representations derive from which sources?
- Good arrangement: write to a system of record database, capture changes (change data capture, CDC), apply them to the search index in the same order. If CDC is the only way of updating the index, the index is entirely derived from the system of record and consistent with it (barring bugs); writing to the database is the only input.
- Bad arrangement: the application writes directly to both index and database. This is the Figure 11-4 problem: two clients send conflicting writes concurrently, the two stores process them in different order, neither is “in charge” of ordering, and they become permanently inconsistent.
- If all user input is funnelled through one system that decides a write order, other representations are derived by processing writes in the same order. This is state machine replication (Total Order Broadcast). CDC versus an event sourcing log matters less than “simply the principle of deciding on a total order”.

Derived data versus distributed transactions

- Classic alternative: distributed transactions (atomic commit, 2PC). Both aim at the same goal by different means: transactions order writes with locks (2PL) while CDC/event sourcing use a log;

transactions get exactly-once effect with atomic commit, log-based systems with deterministic retry and idempotence.

- Biggest difference: transaction systems usually provide linearizability, hence guarantees such as reading your own writes; derived data systems are often asynchronous and "do not by default offer the same timing guarantees".
- Distributed transactions work in limited environments willing to pay their cost, but "I think that XA has poor fault tolerance and performance characteristics". A better protocol might be possible, but widespread adoption "unlikely to happen soon".
- Hence "I believe that log-based derived data is the most promising approach for integrating different data systems." Yet he does not think it productive to tell everyone "eventual consistency is inevitable —suck it up and learn to deal with it" without good guidance.

The limits of total ordering

- Total order broadcast is equivalent to consensus. Most consensus algorithms assume one node's throughput can process the whole stream and give no way for several nodes to share the work of ordering; scaling beyond one node and working geographically distributed "is still an open research problem". The limits as scale grows: a totally ordered log usually needs all events through a single leader; beyond one machine's throughput you partition, and "The order of events in two different partitions is then ambiguous"; with several datacenters you typically have a leader in each, which "implies an undefined ordering of events that originate in two different datacenters"; microservices that each keep their own durable state give no defined order between events from different services; clients that update state at once on user input and work offline will see events in different orders from servers.

Ordering events to capture causality

- Without causal links, concurrent events can be ordered arbitrarily. Easy case: updates to one object can be totally ordered by routing all updates for an object ID to the same log partition. But causal dependencies "sometimes arise in more subtle ways".
- Example: two users break up; one removes the other as a friend and then sends the remaining friends a rude message. If friendship and messages are stored in different places, the ordering dependency may be lost and the notification service may notify the ex-partner. The notifications are effectively a join of messages and friend list (Time-dependence of joins).
- "Unfortunately, there does not seem to be a simple answer to this problem". Starting points: logical timestamps give total ordering without coordination, but recipients must still handle out-of-order delivery and extra metadata is passed around; log an event recording the state the user saw before a decision, with a unique identifier, so later events reference it to record the causal dependency (returns in "Reads are events too"); conflict resolution algorithms help with unexpectedly ordered events but not when actions have external side effects such as notifications. Hope: patterns will emerge to capture causality "without forcing all events to go through the bottleneck of total order broadcast".

Batch and Stream Processing

- His goal statement: "the goal of data integration is to make sure that data ends up in the right form in all the right places". That requires consuming inputs, transforming, joining, filtering, aggregating, training models, evaluating and writing outputs; batch and stream processors are the tools.

Maintaining derived state

- Deterministic functions with well-defined inputs and outputs help fault tolerance (idempotence) and simplify reasoning about organization-wide dataflows. Whether the derived thing is a search index, a statistical model or a cache, think in "data pipelines that derive one thing from another".

Reprocessing data for application evolution

- Stream processing reflects input changes in derived views with low delay; batch reprocesses accumulated history to derive new views onto an existing dataset.
- Reprocessing is "a good mechanism for maintaining a system, evolving it to support new features and changed requirements" (Ch. 4). Without it, schema evolution is limited to simple changes such as adding an optional field or a new record type, in both schema-on-write and schema-on-read. With it, a dataset can be restructured into a completely different model.

Schema Migrations on Railways

- Lesson: derived views allow gradual evolution. Keep old and new schema side by side as two independently derived views, shift a small number of users to the new view to test performance and bugs, increase the share gradually, then drop the old view.

The lambda architecture

- Question: if batch reprocesses history and streaming handles recent updates, how to combine them? The lambda architecture is a proposal that has gained much attention.
- Core idea: record incoming data by appending immutable events to an always-growing dataset (like event sourcing) and derive read-optimized views. Run two systems in parallel, batch (e.g. Hadoop MapReduce) and stream (e.g. Storm). The stream side quickly produces an approximate update; the batch side later consumes the same events and produces a corrected view.
- Credit: “an influential idea that shaped the design of data systems for the better”, by popularizing views derived from streams of immutable events and reprocessing when needed. But “a number of practical problems”: (1) the same logic must be maintained in both batch and stream frameworks; libraries such as Summingbird abstract this, but the operational complexity of debugging, tuning and maintaining two systems remains; (2) the two pipelines produce separate outputs that must be merged for user requests, easy for a simple aggregation over a tumbling window but “significantly harder” for joins, sessionization or non-time-series output; (3) reprocessing everything is expensive, so the batch layer is set up to process incremental batches (an hour’s data every hour), raising stragglers and windows crossing batch boundaries, which makes it “more akin to the streaming layer” against the goal of keeping batch simple.

Unifying batch and stream processing

- More recent work allows lambda’s benefits without its downsides, by implementing batch computation (reprocessing history) and stream computation (events as they arrive) in the same system. This needs features “becoming increasingly widely available”: replaying historical events through the same engine that handles recent ones (log-based brokers can replay; some stream processors read from HDFS); exactly-once semantics for stream processors, which like batch means discarding partial output of failed tasks; and windowing by event time, not processing time, which is meaningless when reprocessing (Apache Beam expresses this, running on Flink or Google Cloud Dataflow).

Unbundling Databases

- Abstractly, databases, Hadoop and operating systems “all perform the same functions”: they store data and let you process and query it. A database stores records in a data model; a filesystem stores files; both are “information management” systems, and Hadoop is somewhat like a distributed version of Unix.
- Two philosophies: Unix offers a logical but fairly low-level hardware abstraction (pipes and files as sequences of bytes); relational databases offer a high-level abstraction hiding disk structures, concurrency and crash recovery (SQL and transactions). Each is “simpler” in its own sense: Unix is a thin wrapper around hardware; a short declarative query draws on query optimization, indexes, join methods, concurrency control and replication without the author understanding them.

Composing Data Storage Technologies

- Database features discussed in this book: secondary indexes (efficient search by field value), materialized views (“a kind of precomputed cache of query results”), replication logs (keep copies on other nodes up to date), and full-text search indexes (built into some relational databases). Chapters 10 and 11 raised the same themes: full-text indexes built in batch, materialized view maintenance, replicating database changes to derived systems (CDC). So built-in database features parallel the derived data systems built with batch and stream processors.

Creating an index

- CREATE INDEX: scan a consistent snapshot of the table, pick out and sort the indexed values, write the index; process the backlog of writes made since the snapshot (if the table was not locked); then keep the index up to date on every write.

The meta-database of everything

- So “the dataflow across an entire organization starts looking like one huge database”. Any batch, stream or ETL process moving data from one place and form to another acts like the subsystem that keeps indexes or materialized views up to date. Batch and stream processors are like elaborate triggers, stored procedures and materialized view maintenance routines; derived data systems are like different index types (B-tree, hash, spatial). The difference: these facilities come from different software, on different machines, run by different teams.
- Avenue one, federation: a unified query interface over many storage engines and processing methods, a **federated database** or **polystore** (e.g. PostgreSQL’s foreign data wrapper). Specialized applications can still access engines directly. It follows the relational tradition: one integrated system with a high-level query language and elegant semantics, “but a complicated implementation”.
- Avenue two, unbundling: making it easier to reliably plug storage systems together (CDC, event logs) is “like unbundling a database’s index-maintenance features”. It follows the Unix tradition: small tools that do one thing well, communicating through a uniform low-level API (pipes), composed with a higher-level language (the shell).

Making unbundling work

- The traditional route, distributed transactions across heterogeneous systems, is “the wrong solution”. Transactions inside one storage or stream system are feasible (some stream processors use them for exactly-once and it works well), but when systems are written by different groups, the lack of a standardized transaction protocol makes integration much harder. Instead: “an asynchronous event log with idempotent writes is a much more robust and practical approach”; an ordered log with idempotent consumers is a simpler abstraction, more feasible across heterogeneous systems.
- Log-based integration brings loose coupling in two ways. System level: asynchronous streams make the system more robust to outages or slowness of components; the log buffers messages so a slow or failed consumer catches up later while producer and other consumers carry on, whereas distributed transactions “tends to escalate local faults into large-scale failures”. Human level: different teams can develop and maintain components independently, each doing one thing well with well-defined interfaces; event logs are powerful enough to capture fairly strong consistency (durability, ordering) yet general enough for almost any data.

Unbundled versus integrated systems

- Unbundling will not replace databases in current form: they are still needed for state in stream processors and to serve queries on batch/stream output; specialized engines (e.g. MPP warehouses for exploratory analytics) stay important.
- The goal is not to beat individual databases on one workload but to combine several for good performance over a much wider range: “It’s about breadth, not depth”. So if one technology does everything you need, “you’re most likely best off simply using that product”. Unbundling matters only when no single piece of software satisfies all requirements.

What’s missing?

- Composition tools are improving, but missing is “the unbundled-database equivalent of the Unix shell”: a high-level declarative language for composing storage and processing systems.
- His wish: declare mysql | elasticsearch, the unbundled equivalent of CREATE INDEX: index all documents of a MySQL database in an Elasticsearch cluster, then continually capture changes and apply them to the index, with no custom application code; possible for almost any storage or indexing system.

Designing Applications Around Dataflow

- The unbundling approach is also called the “database inside-out” approach, after a 2014 conference talk of his. Calling it a “new architecture” is “too grandiose”; it is “a design pattern, a starting point for discussion”, named only to talk about it better. The ideas are an amalgamation of others’ (dataflow languages Oz and Juttle, functional reactive programming such as Elm, logic programming such as Bloom); the term unbundling was proposed by Jay Kreps.
- Spreadsheets have dataflow capabilities ahead of most mainstream languages: change a cell’s input and the formula result is automatically recalculated. That is what we want of data systems: update indexes, cached views and aggregations automatically, and “simply trust that it works correctly”. Most data systems “still have something to learn” from VisiCalc in 1979.

Application code as a derivation function

- Derivations: a secondary index picks out the indexed values of each row or document and sorts by them (B-tree/SSTable, Ch. 3); a full-text index applies language detection, word segmentation, stemming or lemmatization, spelling correction and synonym identification, then builds e.g. an inverted index; a machine learning model is derived from training data by feature extraction and statistical analysis (and its output derives from input plus model); a cache holds data aggregated in the form displayed in the UI, so UI changes may require redefining and rebuilding the cache.

Separation of application code and state

- Databases could in theory deploy arbitrary application code like an OS, but in practice are poorly suited: dependency and package management, version control, rolling upgrades, evolvability, monitoring, metrics, network service calls, external integration. Tools like Mesos, YARN, Docker, Kubernetes are built to run application code and do it much better by doing one thing well.
- Most web apps are stateless services: any request can go to any server, which forgets it; the state goes to a database. The trend is stateless logic separate from state: "not putting application logic in the database and not putting persistent state in the application". The joke: "We believe in the separation of Church and state".

Dataflow: Interplay between state changes and application code

- Dataflow thinking renegotiates the relation of code and state: not a passive variable manipulated by the application, but "the interplay and collaboration between state, state changes, and code that processes them". Code responds to state changes in one place by triggering changes in another.
- Key caution: "maintaining derived data is not the same as asynchronous job execution", for which messaging systems are traditionally designed. First, order of state changes often matters: views derived from one event log must process events in the same order to stay consistent; many brokers lack this when redelivering unacknowledged messages, and dual writes are ruled out. Second, fault tolerance is key: losing one message leaves the derived dataset permanently out of sync, so delivery and state updates must both be reliable (many actor systems keep state and messages in memory, lost on a crash). These demands are stringent but "much less expensive and more operationally robust than distributed transactions"; modern stream processors provide them at scale and run application code as stream operators.

Stream processors and services

- Trend: services communicating via synchronous requests such as REST. Advantage over a monolith is "organizational scalability through loose coupling": teams deploy services independently.
- Example: a purchase priced in one currency and paid in another needs the exchange rate. Microservices: query an exchange-rate service or database for the current rate. Dataflow: subscribe ahead of time to a stream of exchange rate updates, record the current rate in a local database, and at purchase time query only that. This replaces a synchronous request with a local query (maybe same process): faster and "more robust to the failure of another service". "The fastest and most reliable network request is no network request at all!" Instead of RPC there is a stream join between purchase events and exchange rate update events.
- The join is time-dependent: on reprocessing, the rate will have changed, so reconstructing original output needs the historical rate. This applies whether you query a service or subscribe.

Observing Derived State

- Dataflow gives a process to create and update derived datasets: the **write path**. Whenever information is written it may pass through batch and stream stages until every derived dataset incorporates it (Figure 12-1 shows a search index update).
- Derived datasets exist to be queried later: the **read path**: serve a request from the derived dataset, maybe process further, build the response.
- Together they cover the whole journey from collection to consumption (probably by another human). The write path is precomputed, done eagerly whether or not anyone asks; the read path happens only on request. Analogy: eager versus lazy evaluation.

Figure 12-1. In a search index, writes (document updates) meet reads (queries).

Materialized views and caching

- Search index: writes update index entries for every term in a document; reads search each query word and apply Boolean logic (AND, or OR over synonyms).
- No index: less write work, but a read must scan all documents like grep, very expensive at scale. Precompute results for all possible queries: cheap reads, but the set of queries is infinite, so infinite time and space; "That wouldn't work so well." Middle option: precompute only a fixed set of common queries, with the rest from the index: a cache of common queries, or a materialized view, since it must be updated when new documents affect those results.
- Conclusion: an index is not the only boundary; "caches, indexes, and materialized views" simply "shift the boundary between the read path and the write path": more work on writes by precomputing, to save effort on reads.

Stateful, offline-capable clients

- The boundary idea lets us discuss shifting it, here in another context. The client/server model (stateless clients, servers authoritative over data) is so common we forget alternatives; it is worth questioning the status quo.
- Browsers were stateless and useful only online; single-page JavaScript apps and mobile apps now keep local state and need no round-trip for most interactions. Hence renewed interest in **offline-first** applications: use a local database, sync in the background when connected. On slow, unreliable mobile connections this is a big advantage.

Pushing state changes to clients

- A typical page reads data once, assuming it is static, so device state is "a stale cache" unless you poll (RSS is basic polling). Server-sent events (EventSource) and WebSockets keep an open TCP connection so the server can push changes, reducing staleness.

End-to-end event streams

- Elm and React/Flux/Redux manage client state by subscribing to a stream of events (user input, server responses), structured like event sourcing. Natural extension: let the server push state-change events into this pipeline. Then changes flow end to end from one device through event logs, derived data systems and stream processors to another person's UI, "say, under one second end to end".

Reads are events too

- A store written by a stream processor is the boundary between write and read paths, allowing random-access reads that would otherwise need a scan of the whole log. Usually storage is separate from the streaming system, but processors keep state for aggregations and joins; some frameworks let outside clients query it, making the processor "a kind of simple database".
- That is a stream-table join between read queries and the database. The read must be routed to the partition holding the data, like copartitioning in joins. "This correspondence between serving requests and performing joins is quite fundamental": a one-off read passes through the join and is forgotten; a subscribe request is a persistent join with past and future events.
- Logging read events helps track causal dependencies and data provenance: it lets you reconstruct what the user saw before deciding (e.g. the shown shipping date and inventory status in an online shop). It costs extra storage and I/O, and reducing that "is still an open research problem".

Multi-partition data processing

Aiming for Correctness

- Stateless read-only services: fix the bug, restart, everything returns to normal. Stateful systems "are designed to remember things forever (more or less)", so errors can last forever and need more careful thought.
- Goal: applications that are reliable and correct, "programs whose semantics are well defined and understood, even in the face of various faults". For about four decades atomicity, isolation and durability (Ch. 7) were the tools, but "those foundations are weaker than they seem" (weak isolation levels). Some areas abandon transactions for models with "much messier semantics" (leaderless replication). Consistency is poorly defined; some say "embrace weak consistency" without a clear idea what it means.

- Our engineering methods are “surprisingly flaky”: it is very hard to tell whether an application is safe at a given isolation level or replication configuration. Simple solutions often seem right at low concurrency and no faults, then show subtle bugs.
- Kyle Kingsbury’s Jepsen experiments show “stark discrepancies” between claimed guarantees and behavior under network problems and crashes. Even with flawless infrastructure, application code must use features correctly, which is error-prone when configuration is hard to understand (weak isolation, quorums).

The End-to-End Argument for Databases

- Strong safety properties (e.g. serializable transactions) do not guarantee freedom from data loss or corruption: if a bug writes incorrect data or deletes data, serializability won’t save you. He used this before (State, Streams, and Immutability) to argue for immutable, append-only data, since recovery is easier if faulty code cannot destroy good data. But immutability “is not a cure-all by itself”; a subtler corruption follows.

Exactly-once execution of an operation

- On a processing failure you either give up (data loss) or retry, risking that the first attempt succeeded unnoticed, so the message is processed twice. Double processing is corruption: charging twice, incrementing a counter twice.
- Exactly-once means “the final effect is the same as if no faults had occurred”, even with retries. One of the most effective approaches is making the operation idempotent (same effect executed once or many times). That takes effort: possibly metadata such as the set of operation IDs that updated a value, and fencing on failover.

Duplicate suppression

- The pattern recurs outside stream processing. TCP uses packet sequence numbers to order packets and detect loss or duplication; the stack retransmits losses and removes duplicates before handing data to the application.

Example 12-1. A nonidempotent transfer of money from one account to another

- Retrying on a new connection is outside TCP’s duplicate suppression; being non-idempotent, “$22 is transferred instead of the desired $11”. Though a standard atomicity example, “real banks do not work like this”.
- 2PC breaks the 1:1 mapping of connection and transaction, yet is not sufficient: the network between end-user device and application server matters too. A user on weak cellular connection sends an HTTP POST, loses the response, retries; the browser asks “Are you sure you want to submit this form again?” and the user says yes. To the server the retry is a separate request, to the database a separate transaction: “The usual deduplication mechanisms don’t help.”

Operation identifiers

- Idempotence across several network hops cannot rely on the database’s transaction mechanism alone; “you need to consider the end-to-end flow of the request”. Generate a unique operation ID (e.g. UUID) as a hidden form field, or hash the relevant form fields; a double submit carries the same ID. Pass it all the way to the database and execute only one operation per ID.

Example 12-2. Suppressing duplicate requests using a unique ID

- A duplicate ID makes the INSERT fail and aborts the transaction, so it cannot take effect twice. Databases generally maintain uniqueness constraints correctly even at weak isolation, unlike application-level check-then-insert (write skew and phantoms).

The end-to-end argument

- Duplicate suppression is one example of the end-to-end argument, articulated by Saltzer, Reed and Clark in 1984:

  > *“The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the endpoints of the communication system.”*

- It also applies to integrity checking: checksums in Ethernet, TCP and TLS detect network corruption but not bugs at sender/receiver or corruption on disks, so end-to-end checksums are needed. And to encryption: home WiFi passwords protect against local snooping but not remote attackers; TLS

protects against network attackers but not server compromise; “Only end-to-end encryption and authentication can protect against all of these things.”

Applying end-to-end thinking in data systems

- His thesis returns: strong guarantees such as serializable transactions do not make the application free of loss or corruption; it needs its own end-to-end measures such as duplicate suppression.
- “That is a shame, because fault-tolerance mechanisms are hard to get right.” Low-level mechanisms like TCP work well so remaining faults are rare; ideally the rest would be wrapped in an abstraction, “but I fear that we have not yet found the right abstraction”.
- They are expensive, especially across heterogeneous storage. Refusing them for cost means reimplementing fault tolerance in application code; reasoning about concurrency and partial failure is difficult and counterintuitive, so “most application-level mechanisms do not work correctly”, leading to lost or corrupted data.
- Hence it is worth exploring fault-tolerance abstractions that give application-specific end-to-end correctness while keeping good performance and operational characteristics at scale.

Enforcing Constraints

- Correctness through the unbundling lens: end-to-end duplicate suppression works with a request ID from client to the database that records the write. What of other constraints, especially uniqueness (as in Example 12-2)? Examples from Ch. 7: unique username or email, one file per name in a storage service, no double booking of a flight seat or theater seat.

Uniqueness constraints require consensus

- In a distributed setting, uniqueness “requires consensus”: for concurrent requests with the same value, the system must decide which is accepted and reject the others.
- Scale out by partitioning on the value that must be unique (route same request ID to the same partition; partition by hash of username). But asynchronous multi-master replication is ruled out, since masters may accept conflicting writes. To reject violations immediately, “synchronous coordination is unavoidable”.

Uniqueness in log-based messaging

- A log gives all consumers the same order (total order broadcast, equivalent to consensus), so a similar approach enforces uniqueness in the unbundled approach.
- A stream processor consumes a log partition sequentially on a single thread. If the log is partitioned by the value that must be unique, it can “unambiguously and deterministically” decide which of several conflicting operations came first. Usernames: every request is a message appended to a partition chosen by hash of the username; the processor reads requests in order, using a local database of taken names, recording an available name as taken and emitting success, or emitting rejection for a taken one; the requesting client watches the output stream for its success or rejection. This is basically the algorithm for linearizable storage from total order broadcast, and scales by adding partitions. Works for many constraints: “any writes that may conflict are routed to the same partition and processed sequentially”; the definition of conflict depends on the application and the processor can use arbitrary validation logic (cf. Bayou, 1990s).

Multi-partition request processing

- Traditionally: atomic commit across all three, forcing a total order with respect to other transactions on those partitions; with cross-partition coordination, throughput likely suffers.
- Equivalent correctness is possible with partitioned logs and no atomic commit: (1) the client gives the transfer request a unique request ID and appends it to a log partition by request ID; (2) a stream processor reads the requests and, for each, emits a debit instruction to the payer account A (partitioned by A) and a credit instruction to the payee B (partitioned by B), carrying the request ID; (3) further processors consume those streams, deduplicate by request ID and apply the balance changes. Steps 1 and 2 exist because sending credit and debit directly would need atomic commit across two partitions; instead the request is logged as one message, and single-object writes are atomic almost everywhere.
- Result: by splitting the transaction into two differently partitioned stages and using the request ID, “every request is applied exactly once to both the payer and payee accounts”, despite faults and without atomic commit.

Timeliness and Integrity

- Transactions are typically linearizable: the writer waits for commit and then its writes are immediately visible. Not so when an operation is unbundled across stream stages: consumers are asynchronous, so the sender does not wait. A client can wait for a message to appear on an output stream, as in the uniqueness check; but there correctness did not depend on waiting, which only informs the sender and can be decoupled from processing effects.
- **Timeliness**: users observe the system up to date. Stale reads give temporary inconsistency, resolved by waiting and retrying. CAP's consistency is linearizability, a strong way to get timeliness; weaker ones such as read-after-write consistency can also be useful.
- **Integrity**: absence of corruption: no data loss, no contradictory or false data. A derived dataset must be a correct view of underlying data; an index missing records is not very useful. If violated, inconsistency is permanent: waiting won't fix it, "explicit checking and repair is needed". ACID consistency is usually application-specific integrity; atomicity and durability preserve it.
- Slogan: "violations of timeliness are "eventual consistency," whereas violations of integrity are "perpetual inconsistency.""
- Assertion: "in most applications, integrity is much more important than timeliness". Timeliness violations are annoying; integrity violations "can be catastrophic". Credit card example: a transaction from the last 24 hours not yet on the statement is normal (banks reconcile and settle asynchronously). But a statement balance that is not the previous balance plus transactions, or a charge never paid to the merchant ("disappearing money"), would violate integrity.

Correctness of dataflow systems

- ACID gives both timeliness (linearizability) and integrity (atomic commit), so the distinction seems inconsequential. Event-based dataflow systems "decouple timeliness and integrity": no timeliness guarantee unless consumers wait, but integrity is "central to streaming systems".
- Reliable stream processing can preserve integrity without distributed transactions and atomic commit, with comparable correctness and better performance and operational robustness, through a combination of mechanisms: representing a write as a single message (written atomically, fitting event sourcing); deriving all other state updates from it with deterministic derivation functions, like stored procedures; passing a client-generated request ID through all levels for end-to-end duplicate suppression and idempotence; and immutable messages that allow derived data to be reprocessed, easing recovery from bugs. "a very promising direction for building fault-tolerant applications in the future".

Loosely interpreted constraints

- Uniqueness needs consensus, usually by funnelling a partition through one node; stream processing cannot avoid that for the traditional constraint. But many real applications "can actually get away with much weaker notions of uniqueness": concurrent registrations of one username or seat can be fixed by apologizing and asking one person to choose again, a **compensating transaction**; oversold stock can be handled by ordering more, apologizing and offering a discount, the same as when a forklift runs over items, so the apology workflow already exists; airlines and hotels deliberately overbook, violating "one person per seat", with compensation (refunds, upgrades, a neighboring hotel), which is needed anyway for cancelled flights; a bank can charge overdraft fees and bound risk by limiting daily withdrawals.
- In many business contexts it is acceptable to violate a constraint temporarily and fix it by apologizing. Costs vary but are often low: you cannot unsend an email but can send a correction; a double charge can be refunded (cost: fees and perhaps a complaint); cash from an ATM cannot be directly recovered, though debt collectors can pursue an overdrawn account.
- Whether the apology cost is acceptable is "a business decision". If so, checking all constraints before writing is "unnecessarily restrictive", linearizability is not needed, and one may write optimistically and check afterward, while still validating before anything expensive to recover from.
- Such applications need integrity (no lost reservation, no vanishing money) but not timeliness of enforcement: overselling can be patched by apologizing, like conflict resolution in "Handling Write Conflicts".

Coordination-avoiding data systems

- Two observations: (1) "Dataflow systems can maintain integrity guarantees on derived data without atomic commit, linearizability, or synchronous cross-partition coordination." (2) Strict uniqueness

needs timeliness and coordination, but many applications are fine with loose constraints that may be temporarily violated and fixed later, provided integrity is preserved. Together they mean dataflow systems can serve many applications "without requiring coordination, while still giving strong integrity guarantees"; such **coordination-avoiding data systems** have better performance and fault tolerance than systems needing synchronous coordination.
- Serializable transactions still help maintain derived state, at small scope where they work well; heterogeneous XA transactions are not required. Synchronous coordination can be added where needed (e.g. strict constraints before an unrecoverable operation), but not everything should pay for coordination that only a small part needs.

Trust, but Verify

- All of the discussion assumed some things may go wrong and others won't: our **system model** (processes crash, power lost, network delays or drops messages; but fsync data not lost, memory not corrupted, CPU multiplication correct). These are reasonable since true most of the time.
- Anecdote: an application he worked on collected client crash reports, some explicable only by random memory bit-flips; with enough devices, very unlikely things happen. Besides hardware faults or radiation, pathological access patterns can flip bits in healthy memory, usable to break OS security (rowhammer). "hardware isn't quite the perfect abstraction that it may seem". Bit-flips are still very rare on modern hardware, "they deserve some attention".

Maintaining integrity in the face of software bugs

- Lower-level checksums do not catch software bugs. He has personally seen MySQL fail to maintain a uniqueness constraint and PostgreSQL's serializable isolation show write skew anomalies, though both are robust and battle-tested; less mature software is likely worse. Bugs are rare and eventually fixed, but corrupt data meanwhile.

Don't just blindly trust what they promise

- Since hardware and software fall short, corruption is "inevitable sooner or later", so we need a way to find it, fix it and trace the source: Checking the integrity of data is known as **auditing**. Not only financial (see Advantages of immutable events), but finance prizes it because everyone knows mistakes happen.
- "If you want to be sure that your data is still there, you have to actually read it and check." Likewise restore from backups from time to time, or find the backup broken too late. "Don't just blindly trust that it is all working."

A culture of verification

- HDFS and S3 still assume disks work most of the time, which is not the same as always. Few systems continually audit themselves in a "trust, but verify" way; many treat correctness guarantees as absolute. He hopes for more "self-validating or self-auditing systems".
- He fears ACID culture led us to build on blind trust in technology like transactions and neglect auditability, since audit mechanisms seemed not worth the investment. Then NoSQL made weaker guarantees the norm and immature storage widespread, yet we kept the blind trust, now more dangerous.

Designing for auditability

- A multi-object transaction is hard to interpret afterward: even with transaction logs, the changes do not show why they happened, and the application logic invocation is "transient and cannot be reproduced".
- Event-based systems audit better: with event sourcing, input is one immutable event and state updates derive from it, deterministically and repeatably, so the same log through the same derivation code gives the same state.

The end-to-end argument again

- If we cannot trust every component, we must "at least periodically check the integrity of our data"; unchecked corruption is found only after downstream damage, when it is harder and costlier to trace.
- Integrity checking is best end to end: the more systems included, the fewer places for unnoticed corruption. Checking a whole derived pipeline implicitly covers disks, networks, services and algorithms along it.

Tools for auditable data systems

- Few systems make auditability top-level. Some log changes to an audit table, but guaranteeing the integrity of the audit log and the database state is hard. Signing a transaction log with a hardware security module makes it tamper-proof, but "does not guarantee that the right transactions went into the log in the first place".
- Cryptographic tools could prove integrity robustly against hardware, software and even malicious issues. Cryptocurrencies, blockchains and distributed ledgers explore this: distributed databases whose replicas can be hosted by mutually untrusting organizations that continually check each other and agree via consensus.

Doing the Right Thing

- Stepping back: the book has examined architectures and techniques for reliable, scalable, maintainable applications but "left out an important and fundamental part of the discussion". Every system is built for a purpose and has intended and unintended consequences; "We, the engineers building these systems, have a responsibility to carefully consider those consequences and to consciously decide what kind of world we want to live in." This ties back to the epigraph: systems serve ends beyond themselves.
- Many datasets are about people, so "We must treat such data with humanity and respect. Users are humans too, and human dignity is paramount." Guidelines exist (ACM code of ethics) but are rarely discussed, applied or enforced, so engineers and product managers are sometimes cavalier about privacy.
- "A technology is not good or bad in itself—what matters is how it is used and how it affects people", true of a search engine as of a gun. Ethics is difficult "but it is too important to ignore".

Predictive Analytics

- Predicting weather or disease spread differs from predicting whether a convict will reoffend, an applicant default or an insurance customer claim: the latter directly affect individual lives. Organizations are naturally cautious (missed opportunity is cheap, a bad loan costly): "If in doubt, they are better off saying no."
- Someone labeled risky (accurately or falsely) may suffer many "no" decisions, excluded from jobs, travel, insurance, rental, finance: "algorithmic prison". Courts presume innocence; automated systems can exclude people without proof of guilt and with little chance of appeal.

Bias and discrimination

- Algorithmic decisions are not necessarily better or worse than human ones; people are biased, and data might be fairer to the overlooked. But the rules are inferred from data and the learned patterns are opaque (correlation without known why); systematic bias in input is likely learned and amplified.
- Anti-discrimination law protects traits; but other features may correlate with them (postal code or IP address predicting race in segregated neighborhoods). Believing biased input yields impartial output is ridiculous, yet implied by proponents: "machine learning is like money laundering for bias".

Responsibility and accountability

- When a human errs they can be held accountable and the decision appealed; who is accountable when an algorithm errs (self-driving car, discriminatory credit scoring, judicial review where you must explain the decision)?
- Credit scores rest on relevant borrowing history and errors can be corrected; ML scoring uses wider, more opaque inputs. A credit score asks "How did you behave in the past?"; predictive analytics asks "Who is similar to you, and how did people like you behave in the past?", i.e. stereotyping (by where you live, a proxy for race and class); with erroneous data, recourse is almost impossible.
- Statistical data can be right overall and wrong for individuals (average life expectancy of 80 does not mean dying on your 80th birthday). "A blind belief in the supremacy of data for making decisions is not only delusional, it is positively dangerous." We must make algorithms accountable and transparent, avoid reinforcing biases, fix mistakes, and prevent harm: analytics can target aid to those in need, or be used by predatory businesses to sell vulnerable people high-cost loans and worthless degrees.

Feedback loops

- Self-reinforcing loops: employers use credit scores; misfortune causes missed payments, a worse score, less chance of work, poverty, worse scores. “It’s a downward spiral due to poisonous assumptions, hidden behind a camouflage of mathematical rigor and data.”

Privacy and Tracking

- Beyond automated decisions, data collection raises ethical problems: what is the relationship between collectors and the people collected from? If a system stores only what users explicitly enter, it serves them: the user is the customer. When activity is logged as a side effect, the service “takes on interests of its own, which may conflict with the user’s interests”.
- Tracking improves features (click-based ranking, “people who liked X also liked Y”, A/B tests), and users benefit. But with an advertising model the advertisers are the customers, tracking grows more detailed and retained long, building profiles. The user is “coaxed into engaging with it as much as possible”; the relationship is best described as “surveillance”.

Surveillance

- Thought experiment: replace “data” with “surveillance” (“surveillance warehouse”, “surveillance scientists”). Polemic for this book, but strong words are needed: “we have built the greatest mass surveillance infrastructure the world has ever seen.”
- Even totalitarian regimes could only dream of it, yet we adopt it voluntarily; the difference is that corporations collect rather than governments. Having “nothing to hide” means being in line with power structures; the purpose seems benign, but linked to predictive analytics it blurs: insurance tied to tracking devices, smartwatch sensors revealing what you type.

Consent and freedom of choice

- The claim that users consent via terms and get a valuable service has problems: users know little of what data they feed in or how it is processed (“most privacy policies do more to obscure than to illuminate”), so consent is not meaningful; one user’s data also says things about non-users; derived datasets combine data in ways users cannot understand.

Privacy and use of data

- “Privacy is dead” is false: privacy is not secrecy but “the freedom to choose which things to reveal to whom”. It is “a decision right”, part of freedom and autonomy.
- Surveillance infrastructure transfers this right to the data collector (“trust us”). Companies keep outcomes secret because revealing them would seem creepy, exposing intimate information only indirectly (e.g. ad targeting by illness). Users lose agency: the company exercises the privacy right to maximize profit. Companies manage perceptions rather than intrusiveness, and do so poorly (a correct fact can still trigger painful memories).

Data as assets and power

- Behavioral data is called “data exhaust”, suggesting waste to recycle. More correct is the reverse: with targeted advertising paying for the service, behavioral data is its core asset, the application “merely a means to lure users into feeding more and more personal information”.

Remembering the Industrial Revolution

- Data defines the information age; comparisons to the Industrial Revolution come to mind. That brought growth and better living standards in the long run, but also terrible pollution, poor housing, harsh conditions and child labor. Safeguards (environmental regulation, workplace safety, outlawing child labor, food inspection) took long and raised costs, but “society as a whole benefited hugely”.

> *“Data is the pollution problem of the information age, and protecting privacy is the environmental challenge.”*
>
> *“We should try to make them proud.”*

Legislation and self-regulation

- Data protection law might help: the 1995 European Data Protection Directive requires data “collected for specified, explicit and legitimate purposes” and “adequate, relevant and not excessive”. Doubtful it is effective today: it runs against Big Data’s aim to maximize collection and explore, which means unforeseen purposes, the opposite of “specified and explicit”. Updated regulations are being developed.

- His view: a “culture shift in the tech industry”. “We should stop regarding users as metrics to be optimized”; self-regulate to keep trust, and educate users. Control over our data is like a national park: unprotected, “It will be the tragedy of the commons”. “Ubiquitous surveillance is not inevitable —we are still able to stop it.”
- How is open: do not retain data forever, purge when no longer needed (which conflicts with immutability, but that can be solved); a promising approach is access control enforced by cryptographic protocols, not just policy. “Overall, culture and attitude changes will be necessary.”

Summary

- Author’s wrap-up: no single tool serves all use cases, so applications must compose several; data integration is solved with batch processing and event streams letting changes flow between systems. Some systems are systems of record and other data is derived (indexes, materialized views, ML models, statistical summaries). Asynchronous, loosely coupled derivation stops a problem spreading, increasing robustness and fault tolerance.
- Correctness: strong integrity can be achieved scalably with asynchronous event processing, end-to-end operation identifiers for idempotence and asynchronous constraint checking. Clients can wait for the check or proceed and risk apologizing. This is “much more scalable and robust” than distributed transactions and fits how business works. Avoiding most coordination yields systems that keep integrity and perform well, even geographically distributed with faults; audits verify integrity and detect corruption.
- Ethics: data can do good but also significant harm (hard-to-appeal decisions, discrimination, exploitation, normalized surveillance, exposed intimate information, breaches, unintended consequences). Engineers “carry a responsibility to work toward the kind of world that we want to live in: a world that treats people with humanity and respect.”

Footnotes

- Three footnotes: the joke on Church and state (lambda calculus has no mutable state), the microservice exchange-rate cache needing polling or a stream, and the finite-but-exponential number of distinct search queries.

References

- The chapter ends with references (not summarised).

How this chapter builds the author’s case

it opens from Chapter 1’s goals (reliable, scalable, maintainable) and the recurring theme that every problem has several solutions with trade-offs (Ch. 3, Ch. 5). Data Integration argues for a totally ordered log (CDC, event sourcing; Ch. 11, Ch. 9) over distributed transactions (2PC/XA); Unbundling Databases recasts Ch. 10 and 11 as index maintenance and the Unix philosophy; Observing Derived State extends write and read paths (cf. the Ch. 1 Twitter example) to clients. Aiming for Correctness uses Ch. 7 and 9 ideas (idempotence, consensus, linearizability) to keep integrity without coordination, plus auditing; Doing the Right Thing asks what the systems are for, echoing the epigraph.